

1.1 - Introduction to Azure Cloud Shell

Azure Cloud Shell is a browser-accessible command-line experience for managing Azure resources. It provides the flexibility of choosing the shell experience that best suits the way you work, either Bash or PowerShell. Traditionally, to interact with Azure resources via command-line, you need to install the necessary components into your local computer (PC, Mac, Linux).

How does Azure Cloud Shell work?

You have a few different options for accessing Azure Cloud Shell:

- From a direct link: <https://shell.azure.com>
- From the Azure portal
- From code snippets when accessing Microsoft Learn:

Cloud Shell sessions terminate after 20 minutes of inactivity.

When a session terminates, files on your CloudDrive are persisted, but you need to start a new session to access the Cloud Shell environment.

Access your own scripts and files

When using Cloud Shell, you might also need to run scripts or use files for different actions. You can persist files on Cloud Shell by using the Azure CloudDrive:

If you need to edit scripts hosted on the CloudDrive or File Share, you can use the Cloud Shell editor.

Category	Name
Linux tools	bash zsh sh tmux dig
Azure tools	Azure CLI AzCopy Azure Functions CLI Service Fabric CLI Batch Shipyard blobxfer
Text editors	code (Cloud Shell editor) vim nano emacs
Source control	git
Build tools	make maven npm pip
Containers	Docker Machine Kubectl Helm DC/OS CLI

Category	Name
Databases	MySQL client PostgreSql client sqlcmd Utility mssql-scripter
Other	iPython Client Cloud Foundry CLI Terraform Ansible Chef InSpec Puppet Bolt HashiCorp Packer Office 365 CLI

When should you use Azure Cloud Shell?

You can use Azure Cloud Shell to:

- Open a secure command-line session from any browser-based device.
- Interact with Azure resources without the need to install plug-ins or add-ons to your device.
- Persist files between sessions for later use.
- Use either Bash or PowerShell, whichever you prefer, to manage Azure resources.
- Edit files (such as scripts) via the Cloud Shell editor.

You shouldn't use Azure Cloud Shell if:

- You intend to leave a session open for more than 20 minutes for long running scripts or activities. In these cases, your session is disconnected without warning, and the current state is lost.
- You need admin permissions, such as sudo access, from within the Azure CLI or PowerShell environment.
- You need to install tools that aren't supported in the limited Cloud Shell environment, but instead require an environment such as a custom virtual machine or container.
- You need storage from different regions. You might need to back up and synchronize this content since only one region can have the storage allocated to Azure Cloud Shell.
- You need to open multiple sessions at the same time. Azure Cloud Shell allows only one instance at time and isn't suitable for concurrent work across multiple subscriptions or tenants.

1.2 - Deploy Azure infrastructure by using JSON ARM templates

JSON Azure Resource Manager templates (ARM templates) allow you to specify your project's infrastructure in a declarative and reusable way. You can version and save the templates in the same source control as your development project.

Bicep is a language for defining your Azure resources. It has a simpler authoring experience than JSON, along with other features that help improve the quality of your infrastructure as code.

Explore Azure Resource Manager template structure

What is infrastructure as code?

Infrastructure as code allows you to describe, through code, the infrastructure that you need for your application.

With infrastructure as code, you can maintain both your application code and everything you need to deploy your application in a central code repository. The advantages to infrastructure as code are:

- Consistent configurations
- Improved scalability
- Faster deployments
- Better traceability

What is an ARM template?

ARM templates are JavaScript Object Notation (JSON) files that define the infrastructure and configuration for your deployment. The template uses a *declarative syntax*. The declarative syntax is a way of building the structure and elements that outline what resources look like without describing the control flow. *Declarative syntax is different than imperative syntax, which uses commands for the computer to perform. Imperative scripting focuses on specifying each step in deploying the resources.*

Benefits of using ARM templates

ARM templates allow you to automate deployments and use the practice of infrastructure as code (IaC). The template code becomes part of your infrastructure and development projects. Just like application code, you can store the IaC files in a source repository and version it.

ARM template file structure

When you're writing an ARM template, you need to understand all the parts that make up the template and what they do. ARM template files are made up of the following elements:

Element	Description
schema	A required section that defines the location of the JSON schema file that describes the structure of JSON data. The version number you use depends on the scope of the deployment and your JSON editor.
contentVersion	A required section that defines the version of your template (such as 1.0.0.0). You can use this value to document significant changes in your template to ensure you're deploying the right template.
apiProfile	An optional section that defines a collection of API versions for resource types. You can use this value to avoid having to specify API versions for each resource in the template.
parameters	An optional section where you define values that are provided during deployment. You can provide these values in a parameter file, by command-line parameters, or in the Azure portal.
variables	An optional section where you define values that are used to simplify template language expressions.
functions	An optional section where you can define user-defined functions that are available within the template. User-defined functions can simplify your template when complicated expressions are used repeatedly in your template.
resources	A required section that defines the actual items you want to deploy or update in a resource group or a subscription.
output	An optional section where you specify the values that are returned at the end of the deployment.

You can deploy an ARM template to Azure in one of the following ways:

- Deploy a local template (you need to have either [Azure PowerShell](#) or the [Azure CLI](#) installed locally.)
- Deploy a linked template
- Deploy in a continuous deployment pipeline

Use linked templates to deploy complex solutions. You can break a template into many templates and deploy these templates through a main template. When you deploy the main template, it triggers the linked template's deployment. You can store and secure the linked template by using a SAS token.

```
# login using azure CLI
az login

# login using azure powershell
Connect-AzAccount

# obtain available location value using azure CLI / azure powershell
az account list-locations
Get-AzLocation # powershell

# configure default location using the below command
az configure --defaults location=<location>

az group create --name {name of your resource group} --location "{location}"

New-AzResourceGroup -Name {name of your resource group} -Location "{location}"

# start template deployment using azure CLI / powershell
az deployment group create # new command to use
az group deployment create # deprecated

templateFile="{provide-the-path-to-the-template-file}"
az deployment group create --name blanktemplate --resource-group myResourceGroup --
template-file $templateFile

$templateFile = "{provide-the-path-to-the-template-file}"
New-AzResourceGroupDeployment -Name blanktemplate -ResourceGroupName myResourceGroup
-TemplateFile $templateFile
```

Add resources to the template

To add a resource to your template, you need to know the resource provider and its types of resources. The syntax for this combination is in the form of *{resource-provider}/{resource-type}*.

find provider list: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/azure-services-resource-providers>

example of a template for storageAccount

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-
```

```
01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.1",
  "apiProfile": "",
  "parameters": {},
  "variables": {},
  "functions": [],
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2025-01-01",
      "name": "learntemplatestorage123",
      "location": "westus",
      "sku": {
        "name": "Standard_LRS"
      },
      "kind": "StorageV2",
      "properties": {
        "supportsHttpsTrafficOnly": true
      }
    }
  ],
  "outputs": {}
}
```

Create and deploy an Azure Resource Manager template

1. Open Visual Studio Code and create a new file called *azuredeploy.json*.

2.


```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {},
  "functions": [],
  "variables": {},
  "resources": [],
  "outputs": {}
}
```

Deploy the ARM template to Azure

To deploy this template to Azure, you need to sign in to your Azure account from the Visual Studio Code terminal. Be sure that Azure PowerShell Tools are installed from the Visual Studio Code Extensions.

1. In the command bar, select **Terminal > New Terminal** to open a PowerShell window.
2. Change directory to the folder containing your ARM template files.

Sign in to Azure by using Azure PowerShell

From the terminal in Visual Studio Code, run the following command to sign in to Azure.

```

Connect-AzAccount # powershell
az login

# deploy the template to azure
New-AzResourceGroup -Name <ResourceGroupName> -Location <Location> # $ powershell

az group create --name <resource-group-name> --location <location>

# set the resource group
Set-AzDefault -ResourceGroupName <ResourceGroupName> # powershell

az configure --defaults group="<resource-group-name>"

```

Deploy the template to Azure by running the following commands.

```

// powershell
$templateFile="azuredeploy.json"
$today=Get-Date -Format "MM-dd-yyyy"
$deploymentName="blanktemplate-"+$today
New-AzResourceGroupDeployment -Name $deploymentName -TemplateFile $templateFile

// bash
templateFile="azuredeploy.json"
today=$(date +"%d-%b-%Y")
DeploymentName="blanktemplate-"$today

az deployment group create --name $DeploymentName --template-file $templateFile

```

check if deployment was successful

1. In the resource menu, select **Resource groups**.
2. Select the resource group you created in this exercise.
3. On the **Overview** pane, you see that one deployment succeeded.
4. Select **1 Succeeded** to see the details of the deployment.
5. select the deployment name to see what resources was deployed, if any.

Add a resource to the ARM template

update the previous file to add resources to the deployment

```

{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {},
  "functions": [],
  "variables": {},
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",

```

```

    "apiVersion": "2025-01-01",
    "name": "storageaccount1",
    "tags": {
      "displayName": "storageaccount1"
    },
    "location": "[resourceGroup().location]",
    "kind": "StorageV2",
    "sku": {
      "name": "Standard_LRS"
    }
  },
  "outputs": {}
}

```

Deploy the code using Azure CLI and Powershell

```

# Azure CLI
templateFile="azuredeploy.json"
today=$(date +%d-%b-%Y)
DeploymentName="addstorage-"$today

az deployment group create --name $DeploymentName --template-file $templateFile

# powershell
$templateFile="azuredeploy.json"
$today=Get-Date -Format "MM-dd-yyyy"
$deploymentName="addstorage-"+"$today"
New-AzResourceGroupDeployment -Name $deploymentName -TemplateFile $templateFile

```

Add flexibility to your Azure Resource Manager template by using parameters and outputs

You can make your template more reusable by adding a parameter

ARM-template parameters

ARM-template parameters let you customize the deployment by providing values that are tailored for a particular environment. For example, you pass in different values based on whether you're deploying to an environment for development, test, production, or others. For example, the previous template uses the *Standard_LRS* storage account SKU. You can reuse this template for other deployments that create a storage account by making the name of the storage account SKU a parameter. Then, you pass in the name of the SKU you want for this particular deployment when the template is deployed. You can do this step either at the command line or by using a parameter file.

In the `parameters` section of the template, you specify which values you can input when you deploy the resources. You're limited to 256 parameters in a template. Parameter definitions can use most template functions.

available properties for a parameter and The allowed types of parameters are:

- string
- secureString
- integers
- boolean
- object
- secureObject
- array

```
"parameters": {
  "<parameter-name>": {
    "type": "<type-of-parameter-value>",
    "defaultValue": "<default-value-of-parameter>",
    "allowedValues": [
      "<array-of-allowed-values>"
    ],
    "minValue": <minimum-value-for-int>,
    "maxValue": <maximum-value-for-int>,
    "minLength": <minimum-length-for-string-or-array>,
    "maxLength": <maximum-length-for-string-or-array-parameters>,
    "metadata": {
      "description": "<description-of-the-parameter>"
    }
  }
}
```

Recommendations for using parameters

Use parameters for settings that vary according to the environment; for example, SKU, size, or capacity. Also use parameters for resource names that you want to specify yourself for easy identification or to comply with internal naming conventions. Provide a description for each parameter, and use default values whenever possible.

For security reasons, never hardcode or provide default values for usernames and/or passwords in templates. Always use parameters for usernames and passwords (or secrets). Use secureString for all passwords and secrets. If you pass sensitive data in a JSON object, use the secureObject type.

use the parameter in the resource definition. The syntax is `[parameters('name of the parameter')]`. Then, when you deploy you use the `parameters` function. In the next module, you learn more about functions.

```
"resources": [
  {
    "type": "Microsoft.Storage/storageAccounts",
    "apiVersion": "2025-01-01",
    "name": "learntemplatestorage123",
    "location": "[resourceGroup().location]",
    "sku": {
      "name": "[parameters('storageAccountType')]"
    },
    "kind": "StorageV2",
    "properties": {
      "supportsHttpsTrafficOnly": true
    }
  }
]
```

```
    }
  }
]
```

When you deploy the template, you can provide a value for the parameter.

```
# powershell
$templateFile="azuredeploy.json"

New-AzResourceGroupDeployment -Name testdeployment1 -TemplateFile $templateFile -
storageAccountType Standard_LRS

# azure CLI
templateFile="azuredeploy.json"
az deployment group create --name testdeployment1 --template-file $templateFile --
parameters storageAccountType=Standard_LRS
```

ARM template outputs

In your ARM template's outputs section, you can specify the values that are returned after a successful deployment. Here are the elements that make up the outputs section.

Element	Description
output-name	Must be a valid JavaScript identifier.
condition	(Optional) A Boolean value that indicates whether this output value is returned. When true, the value is included in the output for the deployment. When false, the output value is skipped for this deployment. When not specified, the default value is true.
type	The type of the output value.
value	(Optional) A template language expression to be evaluated and returned as an output value.
copy	(Optional) Copy is used to return more than one value for an output.

```
"outputs": {
  "<output-name>": {
    "condition": "<boolean-value-whether-to-output-value>",
    "type": "<type-of-output-value>",
    "value": "<output-value-expression>",
    "copy": {
      "count": <number-of-iterations>,
      "input": <values-for-the-variable>
    }
  }
}

// Example
"outputs": {
  "storageEndpoint": {
    "type": "object",
    "value": "[reference('learntemplatestorage123').primaryEndpoints]"
  }
}
```

```
}
}
```

Deploy an ARM template again

Recall that ARM templates are *idempotent*, which means you can deploy the template to the same environment again, and if nothing changes in the template, nothing changes in the environment. If a change is made to the template (for example, you change a parameter value), only that change is deployed. Your template can contain all of the resources you need for your Azure solution, and you can safely execute a template again. Resources are created only if they don't already exist, and updated only if there's a change.

Add parameters and outputs to your Azure Resource Manager template

Here, you make your ARM template more flexible by adding parameters that can be set at runtime. Create a parameter for the `storageName` value.

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "storageName": {
      "type": "string",
      "minLength": 3,
      "maxLength": 24,
      "metadata": {
        "description": "The name of the Azure storage resource"
      }
    },
    "storageSKU": {
      "type": "string",
      "defaultValue": "Standard_LRS",
      "allowedValues": [
        "Standard_LRS",
        "Standard_GRS",
        "Standard_RAGRS",
        "Standard_ZRS",
        "Premium_LRS",
        "Premium_ZRS",
        "Standard_GZRS",
        "Standard_RAGZRS"
      ]
    }
  },
  "functions": [],
  "variables": {},
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2025-01-01",
```

```

    "name": "[parameters('storageName')]",
    "tags": {
      "displayName": "[parameters('storageName')]"
    },
    "location": "[resourceGroup().location]",
    "kind": "StorageV2",
    "sku": {
      "name": "[parameters('storageSKU')]"
    }
  },
],
"outputs": {
  "storageEndpoint": {
    "type": "object",
    "value": "[reference(parameters('storageName')).primaryEndpoints]"
  }
}
}

```

1 - Understanding Entra ID

Microsoft Entra ID is a cloud-based identity and access management service provided by Microsoft. Microsoft Entra ID is a comprehensive solution for managing identities, enforcing access policies, and securing your applications and data in the cloud and on-premises.

Microsoft Entra ID is part of the platform as a service (PaaS) offering and operates as a Microsoft-managed directory service in the cloud. It's not a part of the core infrastructure that customers own and manage, nor is it an Infrastructure as a service offering.

With Microsoft Entra ID, you also have access to a set of features that aren't natively available in AD DS, such as support for multi-factor authentication, identity protection, and self-service password reset.

You can use Microsoft Entra ID to provide more secure access to cloud-based resources for organizations and individuals by:

- Configuring access to applications
- Configuring single sign-on (SSO) to cloud-based SaaS applications
- Managing users and groups
- Provisioning users
- Enabling federation between organizations
- Providing an identity management solution
- Identifying irregular sign-in activity
- Configuring multi-factor authentication
- Extending existing on-premises Active Directory implementations to Microsoft Entra ID
- Configuring Application Proxy for cloud and local applications
- Configuring Conditional Access for users and devices

Microsoft Entra tenants

Unlike AD DS, Microsoft Entra ID is multi-tenant by design and is implemented specifically to ensure isolation between its individual directory instances. It's the world's largest multi-tenant directory, hosting over a million directory services instances, with billions of authentication requests per week. The term tenant in this context typically represents a company or organization

that signed up for a subscription to a Microsoft cloud-based service such as Microsoft 365, Intune, or Azure, each of which uses Microsoft Entra ID.

At any given time, an Azure subscription must be associated with one, and only one, Microsoft Entra tenant. This association allows you to grant permissions to resources in the Azure subscription (via RBAC) to users, groups, and applications that exist in that particular Microsoft Entra tenant.

A single Microsoft Entra tenant can support multiple Azure subscriptions.

Microsoft Entra schema

The Microsoft Entra schema contains fewer object types than that of AD DS. Most notably, it doesn't include a definition of the computer class, although it does include the device class. The process of joining devices to Microsoft Entra differs considerably from the process of joining computers to AD DS.

Microsoft Entra ID and its services define a concept of modern management. Microsoft Entra ID's primary strength lies in providing directory services; storing and publishing user, device, and application data; and handling the authentication and authorization of the users, devices, and applications.

Microsoft Entra ID doesn't include the organizational unit (OU) class, which means that you can't arrange its objects into a hierarchy of custom containers, which is frequently used in on-premises AD DS deployments.

Compare Microsoft Entra ID and Active Directory Domain Services

Characteristics of AD DS

AD DS is the traditional deployment of Windows Server-based Active Directory on a physical or virtual server. Although AD DS is commonly considered being primarily a directory service, it's only one component of the Windows Active Directory suite of technologies, which also includes Active Directory Certificate Services (AD CS), Active Directory Lightweight Directory Services (AD LDS), Active Directory Federation Services (AD FS), and Active Directory Rights Management Services (AD RMS).

When comparing AD DS with Microsoft Entra ID, it's important to note the following characteristics of AD DS:

- AD DS is a true directory service, with a hierarchical X.500-based structure.
- AD DS uses Domain Name System (DNS) for locating resources such as domain controllers.
- You can query and manage AD DS by using Lightweight Directory Access Protocol (LDAP) calls.
- AD DS primarily uses the Kerberos protocol for authentication.
- AD DS uses OUs and GPOs for management.
- AD DS includes computer objects, representing computers that join an Active Directory domain.
- AD DS uses trusts between domains for delegated management.

Characteristics of Microsoft Entra ID

When comparing Microsoft Entra ID with AD DS, it's important to note the following characteristics of Microsoft Entra ID:

- Microsoft Entra ID is primarily an identity solution, and it's designed for internet-based applications by using HTTP (port 80) and HTTPS (port 443) communications.

- Microsoft Entra ID is a multi-tenant directory service.
- Microsoft Entra users and groups are created in a flat structure, and there are no OUs or GPOs.
- You can't query Microsoft Entra ID by using LDAP; instead, Microsoft Entra ID uses the REST API over HTTP and HTTPS.
- Microsoft Entra ID doesn't use Kerberos authentication; instead, it uses HTTP and HTTPS protocols such as SAML, WS-Federation, and OpenID Connect for authentication, and uses OAuth for authorization.
- Microsoft Entra ID includes federation services, and many third-party services such as Facebook are federated with and trust Microsoft Entra ID.

Examine Microsoft Entra ID as a directory service for cloud apps

When you deploy cloud services such as Microsoft 365 or Intune, you also need to have directory services in the cloud to provide authentication and authorization for these services. Because of this, each cloud service that needs authentication will create its own Microsoft Entra tenant. When a single organization uses more than one cloud service, it's much more convenient for these cloud services to use a single cloud directory instead of having separate directories for each service.

Compare Microsoft Entra ID P1 and P2 plans

The Microsoft Entra ID P1 or P2 tier provides extra functionality as compared to the Free and Office 365 editions. However, premium versions require additional cost per user provisioning. Microsoft Entra ID P1 or P2 comes in two versions P1 and P2. You can procure it as an extra license or as a part of the Microsoft Enterprise Mobility + Security, which also includes the license for Azure Information Protection and Intune.

The following features are available with the Microsoft Entra ID P1 edition:

- **Self-service group management.** It simplifies the administration of groups where users are given the rights to create and manage the groups. End users can create requests to join other groups, and group owners can approve requests and maintain their groups' memberships.
- **Advanced security reports and alerts.** You can monitor and protect access to your cloud applications by viewing detailed logs that show advanced anomalies and inconsistent access pattern reports. Advanced reports are machine learning based and can help you gain new insights to improve access security and respond to potential threats.
- **Multi-factor authentication.** Full multi-factor authentication (MFA) works with on-premises applications (using virtual private network [VPN], RADIUS, and others), Azure, Microsoft 365, Dynamics 365, and third-party Microsoft Entra gallery applications. It doesn't work with non-browser off-the-shelf apps, such as Microsoft Outlook. Full multi-factor authentication is covered in more detail in the following units in this lesson.
- **Microsoft Identity Manager (MIM) licensing.** MIM integrates with Microsoft Entra ID P1 or P2 to provide hybrid identity solutions. MIM can bridge multiple on-premises authentication stores such as AD DS, LDAP, Oracle, and other applications with Microsoft Entra ID. This provides consistent experiences to on-premises line-of-business (LOB) applications and SaaS solutions.
- **Enterprise SLA of 99.9%.** You're guaranteed at least 99.9% availability of the Microsoft Entra ID P1 or P2 service. The same SLA applies to Microsoft Entra Basic.
- **Password reset with writeback.** Self-service password reset follows the Active Directory on-premises password policy.

- **Cloud App Discovery feature of Microsoft Entra ID.** This feature discovers the most frequently used cloud-based applications.
- **Conditional Access based on device, group, or location.** This lets you configure conditional access for critical resources, based on several criteria.
- **Microsoft Entra Connect Health.** You can use this tool to gain operational insight into Microsoft Entra ID. It works with alerts, performance counters, usage patterns, and configuration settings, and presents the collected information in the Microsoft Entra Connect Health portal.

In addition to these features, the Microsoft Entra ID P2 license provides extra functionalities:

- **Microsoft Entra ID Protection.** This feature provides enhanced functionalities for monitoring and protecting user accounts. You can define user risk policies and sign-in policies. In addition, you can review users' behavior and flag users for risk.
- **Microsoft Entra Privileged Identity Management.** This functionality lets you configure additional security levels for privileged users such as administrators. With Privileged Identity Management, you define permanent and temporary administrators. You also define a policy workflow that activates whenever someone wants to use administrative privileges to perform some task.

Examine Microsoft Entra Domain Services

Microsoft provides Microsoft Entra Domain Services. This service, which runs as part of the Microsoft Entra ID P1 or P2 tier, provides domain services such as Group Policy management, domain joining, and Kerberos authentication to your Microsoft Entra tenant. These services are fully compatible with locally deployed AD DS, so you can use them without deploying and managing additional domain controllers in the cloud.

Because Microsoft Entra ID can integrate with your local AD DS, when you implement Microsoft Entra Connect, users can utilize organizational credentials in both on-premises AD DS and in Microsoft Entra Domain Services. Even if you don't have AD DS deployed locally, you can choose to use Microsoft Entra Domain Services as a cloud-only service.

Microsoft Entra Domain Services provides several benefits for organizations, such as:

- Administrators don't need to manage, update, and monitor domain controllers.
- Administrators don't need to deploy and manage Active Directory replication.
- There's no need to have Domain Admins or Enterprise Admins groups for domains that Microsoft Entra ID manages.

If you choose to implement Microsoft Entra Domain Services, you need to be aware of the service's current limitations. These include:

- Only the base computer Active Directory object is supported.
- It's not possible to extend the schema for the Microsoft Entra Domain Services domain.
- The organizational unit (OU) structure is flat and nested OUs aren't currently supported.
- There's a built-in Group Policy Object (GPO), and it exists for computer and user accounts.
- It's not possible to target OUs with built-in GPOs. Additionally, you can't use Windows Management Instrumentation filters or security-group filtering.

You can enable Microsoft Entra Domain Services by using the Azure portal. This service charges per hour based on the size of your directory.

2 - Create, configure and manage identities

Every user who needs access to resources needs a user account in Microsoft Entra ID. A user account contains all the information needed to authenticate the user during the sign-on process. Once authenticated, Microsoft Entra ID builds an access token to authorize the user and determine what resources they can access and what they can do with those resources.

*You use the **Microsoft Entra admin center** to work with user objects. Keep in mind that you can only work with a single directory at a time. You can use the **Directory + Subscription** panel to switch directories. The admin center also has a **Switch directory** button in the toolbar, which makes it easy to switch to another available directory.*

View users

To view the Microsoft Entra users, select the **Users** entry under **Identity**, then open the **All Users** view.

Typically, Microsoft Entra ID defines users in three ways:

- **Cloud identities** - These users exist only in Microsoft Entra ID. Examples are administrator accounts and users that you manage yourself. Their source is **Microsoft Entra ID** or **External Microsoft Entra directory** if the user is defined in another Microsoft Entra instance but needs access to subscription resources controlled by this directory. When these accounts are removed from the primary directory, they're deleted.
- **Directory-synchronized identities** - These users exist in an on-premises Active Directory. A synchronization activity brings these users into Microsoft Entra ID. **Microsoft Entra Cloud Sync** is the recommended synchronization tool for most organizations—it uses a lightweight cloud-managed agent and supports multiple disconnected forests. **Microsoft Entra Connect Sync** remains available for complex scenarios such as device synchronization or groups with more than 50,000 members. Their source is **Windows Server AD**.
- **Guest users** - These users exist outside your organization. Examples are accounts from other cloud providers and Microsoft accounts. Their source is **Invited user**. This type of account is useful when external vendors or contractors need access to your organization's resources. Once their help is no longer necessary, you can remove the account and all of their access.

assign licenses to users

Create a new user in Microsoft Entra ID

You can skip creating this user if you created the same user in the earlier module.

1. Browse to the Identity menu in the [Microsoft Entra admin center](#).
2. In the left navigation, under select **Users**, then **All Users**.
3. Within the Users page, on the menu, select + **New user** and **Create new user**.

Create a security group in Microsoft Entra ID

1. Browse to the Microsoft Entra admin center screen.
2. In the left navigation, under **Identity**, select **Groups** and then **All groups**.
3. In the Groups screen, on the menu, select **New group**.

Assign a license to a group

License assignment to groups is managed through the Microsoft 365 admin center.

1. Go to the Microsoft 365 admin center at <https://admin.microsoft.com>.
2. Select **Billing** from the menu on the left.
3. Select **Licenses**.

4. From the list of licenses you have available, select one.
5. Select **Groups** from the list near the top of the screen.
6. On the Groups page, select **+ Assign license**.
7. Search for and select the **Marketing** group you created earlier.
8. Select the **Assign** button at the bottom of the dialog.
9. You should get a message that licenses were successfully assigned.

Restore or remove a recently deleted user with Microsoft Entra ID

After you delete a user, the account remains in a suspended state for 30 days. During that 30-day window, the user account can be restored, along with all its properties. After that 30-day window passes, the permanent deletion process is automatically started.

Required permissions

You must have one of the following roles to restore or permanently delete users.

- Global administrator
- Partner Tier-1 Support
- Partner Tier-2 Support
- User administrator

A Microsoft Entra group helps organize users, which makes it easier to manage permissions. Using groups lets the resource owner (or Microsoft Entra directory owner), assign a set of access permissions to all the members of the group, instead of having to provide the rights one-by-one. Groups let you define a security boundary and then add and remove specific users to grant or deny access with a minimum amount of effort.

Microsoft Entra ID allows you to define two different types of groups.

- **Security groups** - the most common type of groups and are used to manage access to shared resources. Members of a security group can include users, devices, and service principals. For example, you can create a security group for a specific security policy. By doing it this way, you can give a set of permissions to all the members at once, instead of having to add permissions to each member individually. This option requires a Microsoft Entra administrator.
- **Microsoft 365 groups** - provide collaboration opportunities by giving members access to a shared mailbox, calendar, files, SharePoint site, and more. This option also lets you give people outside of your organization access to the group. This option is available to users and admins.

The second characteristic of a group that you need to be aware of is the **Membership Type**. This specifies how individual members are added to the group. The three types are:

- **Assigned** - members are added and maintained manually.
- **Dynamic User** - users are added and removed automatically based on rules that evaluate user attributes such as department, job title, or location.
- **Dynamic Device** - devices are added and removed automatically based on rules that evaluate device attributes. Applies to security groups only; Microsoft 365 groups support dynamic users but not dynamic devices.

Dynamic groups

With dynamic membership, Microsoft Entra ID automatically adds or removes users or devices from a group based on rules you define. When a member's attributes change—for example, a user moves to a different department—all dynamic membership rules in the tenant are reevaluated, and the user is added to or removed from groups accordingly.

*Dynamic membership requires a **Microsoft Entra ID P1** license (or Intune for Education for device-based rules).*

Configure and manage device registration

With the proliferation of devices of all shapes and sizes and the proliferation of bring-your-own-device (BYOD), IT professionals are faced with two somewhat opposing goals:

- Allow end users to be productive wherever and whenever and on any device
- Protect the organization's assets

To protect these assets, IT-staff needs to first manage the device identities. IT-staff can build on the device identity with tools like Microsoft Intune to ensure standards for security and compliance are met. Microsoft Entra ID enables single sign-on to devices, apps, and services from anywhere through these devices.

- Your users get access to your organization's assets they need.
- Your IT-staff gets the controls they need to secure your organization.

Microsoft Entra registered devices

The goal of Microsoft Entra registered devices is to provide your users with support for the BYOD or mobile device scenarios. In these scenarios, a user can access your organization’s Microsoft Entra ID controlled resources using a personal device.

Microsoft Entra registered	Description
Definition	Registered to Microsoft Entra ID without requiring organizational account to sign in to the device
Primary audience	Applicable to Bring your own device (BYOD), and Mobile devices
Device ownership	User or Organization
Operating systems	Windows 10 or newer, macOS 10.15 or newer, iOS 15 or newer, Android, Linux (Ubuntu 20.04/22.04/24.04 LTS, Red Hat Enterprise Linux 8/9 LTS)
Device sign in options	End-user local credentials, Password, Windows Hello, PIN, Biometrics
Device management	Mobile Device Management (example: Microsoft Intune), Mobile Application Management
Key capabilities	SSO to cloud resources, Conditional Access when enrolled in Intune, Conditional Access via App protection policy

Microsoft Entra registered devices are signed in to using a local account like a Microsoft account on a Windows 10 or newer device, but additionally have a Microsoft Entra account attached for access to organizational resources. Access to resources in the organization can be further limited based on that Microsoft Entra account and Conditional Access policies applied to the device identity.

Administrators can secure and further control these Microsoft Entra registered devices using Mobile Device Management (MDM) tools like Microsoft Intune. MDM provides a means to enforce

organization-required configurations like requiring storage to be encrypted, password complexity, and security software kept updated

Microsoft Entra joined devices

Microsoft Entra joined is intended for organizations that want to be cloud-first or cloud-only. Any organization can deploy Microsoft Entra joined devices no matter the size or industry. Microsoft Entra joined enables access to both cloud and on-premises apps and resources.

Microsoft Entra joined	Description
Definition	Joined only to Microsoft Entra ID requiring organizational account to sign in to the device
Primary audience	Suitable for both cloud-only and hybrid organizations
Device ownership	Organization
Operating systems	All Windows 10 and Windows 11 devices (except Home editions); Windows Server 2019 and newer VMs in Azure (Server Core not supported); macOS 13 or newer (preview)
Device management	Mobile Device Management (example: Microsoft Intune)
Key capabilities	SSO to both cloud and on-premises resources, Conditional Access, Self-service Password Reset and Windows Hello PIN reset

Administrators can secure and further control Microsoft Entra joined devices using Mobile Device Management (MDM) tools like Microsoft Intune or in co-management scenarios using Microsoft Endpoint Configuration Manager.

These tools provide a means to enforce organization-required configurations like requiring storage to be encrypted, password complexity, software installations, and software updates. Administrators can make organization applications available to Microsoft Entra joined devices using Configuration Manager.

Although Microsoft Entra joined is primarily intended for organizations that don't have an on-premises Windows Server Active Directory infrastructure, you can certainly use it in scenarios where:

- You want to transition to cloud-based infrastructure using Microsoft Entra ID and MDM like Intune.
- You can't use an on-premises domain join, for example, if you need to get mobile devices such as tablets and phones under control.
- Your users primarily need to access Microsoft 365 or other SaaS apps integrated with Microsoft Entra ID.
- You want to manage a group of users in Microsoft Entra ID instead of in Active Directory. This scenario can apply, for example, to seasonal workers, contractors, or students.
- You want to provide joining capabilities to workers in remote branch offices with limited on-premises infrastructure.

The goal of Microsoft Entra joined devices is to simplify:

- Windows deployments of work-owned devices
- Access to organizational apps and resources from any Windows device
- Cloud-based management of work-owned devices

- Users to sign in to their devices with their Microsoft Entra ID or synced Active Directory work or school accounts.

Hybrid Microsoft Entra joined devices

For more than a decade, many organizations have used the domain join to their on-premises Active Directory to enable:

- IT departments to manage work-owned devices from a central location.
- Users to sign in to their devices with their Active Directory work or school accounts.

If your environment has an on-premises AD footprint and you also want benefit from the capabilities provided by Microsoft Entra ID, you can implement hybrid Microsoft Entra joined devices. These devices are devices that are joined to your on-premises Active Directory and registered with your Microsoft Entra directory.

Hybrid Microsoft Entra joined	Description
Definition	Joined to on-premises AD and Microsoft Entra ID requiring organizational account to sign in to the device
Primary audience	Suitable for hybrid organizations with existing on-premises AD infrastructure
Device ownership	Organization
Operating systems	Windows 10, Windows 11 (except Home editions), Windows Server 2016, 2019, and 2022
Device sign in options	Password or Windows Hello for Business
Device management	Group Policy, Configuration Manager standalone, or co-management with Microsoft Intune
Key capabilities	SSO to both cloud and on-premises resources, Conditional Access, Self-service Password Reset and Windows Hello PIN reset

Use Microsoft Entra hybrid joined devices if:

- You have Win32 apps deployed to these devices that rely on Active Directory machine authentication.
- You want to continue to use Group Policy to manage device configuration.
- You want to continue to use existing imaging solutions to deploy and configure devices.

For organizations planning new hybrid deployments, use Cloud Kerberos Trust to enable on-premises SSO and Windows Hello for Business in hybrid environments.

Manage licenses

Microsoft paid cloud services, such as Microsoft 365, Enterprise Mobility + Security, Dynamics 365, and other similar products, require licenses. These licenses are assigned to each user who needs access to these services. To manage licenses, administrators use the [Microsoft 365 admin center](#) or PowerShell and Microsoft Graph API. Microsoft Entra ID is the underlying infrastructure that supports identity management for all Microsoft cloud services. Microsoft Entra ID stores information about license assignment states for users.

License requirements

You must have one of the following licenses to use group-based licensing:

- Paid or trial subscription for Microsoft Entra ID Premium P1 and greater

- Paid or trial edition Office 365 Enterprise E3 or greater

Features

Here are the main features of group-based licensing:

- Licenses can be assigned to any security group in Microsoft Entra ID. Security groups can be synced from on-premises by using Microsoft Entra Cloud Sync (recommended) or Microsoft Entra Connect Sync. You can also create security groups directly in Microsoft Entra ID (also called cloud-only groups), or automatically via the Microsoft Entra dynamic group feature.
- When a product license is assigned to a group, the administrator can disable one or more service plans in the product. Typically, this assignment is done when the organization isn't yet ready to start using a service included in a product. For example, the administrator might assign Microsoft 365 to a department, but temporarily disable the Viva Engage service.
- All Microsoft cloud services that require user-level licensing are supported. This support includes all Microsoft 365 products, Enterprise Mobility + Security, and Dynamics 365.
- Group-based licensing is currently available only through the [Microsoft 365 admin center](#).
- Microsoft Entra ID automatically manages license modifications that result from group membership changes. Typically, license modifications are effective within minutes of a membership change.
- A user can be a member of multiple groups with license policies specified. A user can also have some licenses that were directly assigned, outside of any groups. The resulting user state is a combination of all assigned product and service licenses. If a user is assigned same license from multiple sources, the license is consumed only once.
- In some cases, licenses can't be assigned to a user. For example, there might not be enough available licenses in the tenant, or conflicting services are assigned at the same time. Administrators have access to information about users for whom Microsoft Entra ID couldn't fully process group licenses. They can then take corrective action based on that information.

change group license assignments

1. Open <https://entra.microsoft.com> to get to the Microsoft Entra admin center.
2. In the left navigation, open **Groups**.
3. Select **All groups**, then select one of the available groups.
4. In the left navigation, under **Manage**, select **Licenses**.
5. Review the current assignments and then, on the menu, select **+ Assignments**.
6. Open <https://admin.microsoft.com> to open the Microsoft 365 admin center.
7. Select **Billing**. Then select **Licenses**.
8. Select an available license from the list.
9. Select **Groups** from the menu near the top of the page.
10. Select the **+ Assign licenses** option.
11. Pick the group you were looking at earlier in Microsoft Entra. Then select the **Assign** button at the bottom of the page.
12. On the group's Licenses page, review the change. You should be able to see the change in both the Microsoft Entra admin center and the Microsoft 365 admin center.

*To see how many licenses are available, go to **Microsoft Entra - Identity - Billing** , then **Licenses**, then **All products**.*

Identify and resolve license assignment problems for a group in Microsoft Entra ID

Group-based licensing in Microsoft Entra ID introduces the concept of users in a licensing error state.

When you assign licenses directly to individual users, without using group-based licensing, the assignment operation might fail. For example, when you execute the PowerShell cmdlet `Set-MgUserLicense` on a user object, the cmdlet can fail for many reasons that are related to business logic.

When you're using group-based licensing the same errors can occur, but they happen in the background while the Microsoft Entra service is assigning licenses. For this reason, the errors can't be communicated to you immediately. Instead, they're recorded on the user object and then reported via the administrative portal.

Not enough licenses

Problem: There aren't enough available licenses for one of the products specified in the group. You need to either purchase more licenses for the product or free up unused licenses from other users or groups.

To see how many licenses are available, go to **Microsoft Entra - Identity - Billing**, then **Licenses**, then **All products**.

PowerShell: PowerShell cmdlets report this error as *CountViolation*.

Service plans that conflict

Problem: One of the products specified in the group contains a service plan that conflicts with another service plan that's already assigned to the user via a different product. Some service plans are configured in a way that they can't be assigned to the same user as another, related service plan.

To solve this conflict, you need to disable two of the plans. You can disable the E1 license directly assigned to the user. Or, you need to modify the entire group license assignment and disable the plans in the E3 license. Alternatively, you might decide to remove the E1 license from the user if it's redundant in the context of the E3 license.

PowerShell: PowerShell cmdlets report this error as *MutuallyExclusiveViolation*.

Other products depend on this license

Problem: One of the products specified in the group contains a service plan that must be enabled for another service plan, in another product, to function. This error occurs when Microsoft Entra ID attempts to remove the underlying service plan. For example, this can happen when you remove the user from the group.

To solve this problem, you need to make sure that the required plan is still assigned to users through some other method or that the dependent services are disabled for those users. After doing that, you can properly remove the group license from those users.

PowerShell: PowerShell cmdlets report this error as *DependencyViolation*.

Usage location isn't allowed

Problem: Some Microsoft services aren't available in all locations because of local laws and regulations. Before you can assign a license to a user, you must specify the **Usage location** property for the user. You can specify the location under the **User**, then **Profile**, then **Edit** the section in the Azure portal.

To solve this problem, remove users from unsupported locations from the licensed group. Alternatively, if the current usage location values don't represent the actual user location, you can modify them so that the licenses are correctly assigned next time (if the new location is supported).

PowerShell: PowerShell cmdlets report this error as *ProhibitedInUsageLocationViolation*.

Duplicate proxy addresses

If you use Exchange Online, some users in your organization might be incorrectly configured with the same proxy address value. When group-based licensing tries to assign a license to such a user, it fails and shows “Proxy address is already being used.”

Microsoft Entra Mail and ProxyAddresses attribute change

Problem: While updating license assignment on a user or a group, you might see that the Microsoft Entra Mail and ProxyAddresses attribute of some users are changed.

Updating license assignment on a user causes the proxy address calculation to be triggered, which can change user attributes.

LicenseAssignmentAttributeConcurrencyException in audit logs

Problem: User has LicenseAssignmentAttributeConcurrencyException for license assignment in audit logs. When group-based licensing tries to process concurrent license assignment of the same license to a user, this exception is recorded on the user. This typically happens when a user is a member of more than one group with same assigned license. Microsoft Entra ID retries processing the user license and will resolve the issue. There's no action required from the customer to fix this issue.

How to migrate users with individual licenses to group licenses

You can have existing licenses deployed to users in the organizations via direct assignment; that is, using PowerShell scripts or other tools to assign individual user licenses. Before you begin using group-based licensing to manage licenses in your organization, you can use this migration plan to seamlessly replace existing solutions with group-based licensing.

Recommended migration process

1. You have existing automation (for example, PowerShell) managing license assignment and removal for users. Leave it running as is.
2. Create a new licensing group (or decide which existing groups to use) and make sure that all required users are added as members.
3. Assign the required licenses to those groups; your goal should be to reflect the same licensing state your existing automation (for example, PowerShell) is applying to those users.
4. Verify that licenses are applied to all users in those groups. This application can be done by checking the processing state on each group and by checking Audit Logs.
 - You can perform a random check of a few individual users by looking at their license details. You see that they have the same licenses assigned “directly” and “inherited” from groups.
 - You can run a PowerShell script to [verify how licenses are assigned to users](#).
 - When the same product license is assigned to the user both directly and through a group, only one license is consumed by the user. Hence no more licenses are required to perform migration.
5. Verify that no license assignments failed by checking each group for users in error state.

change user license assignments

Create a new user in Microsoft Entra ID

1. Browse to the [Microsoft Entra admin center](#).
2. In the left navigation, under **Identity**, select **Users**.
3. In the Users page, on the menu, select **+ New user** and then **Create new user**.

Update user license assignments

License assignment to individual users is managed through the Microsoft 365 admin center.

1. Open the [Microsoft 365 admin center](#).
2. Select **Billing**, then select **Licenses**.
3. Select an available license from the list.
4. Select **Licensed users** from the menu near the top of the page.
5. Select **+ Assign licenses**.
6. Search for and select **Dominique Koch**, then select **Assign** at the bottom of the page.
7. When complete, verify the license is listed on Dominique Koch's profile in the Microsoft Entra admin center under **Identity** > **Users** > select the user > **Licenses**.

Create custom security attributes

What is a custom security attribute?

Custom security attributes in Microsoft Entra ID are business-specific attributes (key-value pairs) that you can define and assign to Microsoft Entra objects. These attributes can be used to store information, categorize objects, or enforce fine-grained access control over specific Azure resources.

Why use custom security attributes?

- Extend user profiles, such as add Hourly Salary to all my employees.
- Ensure only administrators can see the Hourly Salary attribute in my employees' profiles.
- Categorize hundreds or thousands of applications to easily create a filterable inventory for auditing.
- Grant users access to the Azure Storage blobs belonging to a project.

What can I do with custom security attributes?

- Define business-specific information (attributes) for your tenant.
- Add custom security attributes to Microsoft Entra users and enterprise applications (service principals).
- Manage Microsoft Entra objects using custom security attributes with queries and filters.
- Provide attribute governance so attributes determine who can get access.

Features of custom security attributes

- Available tenant-wide
- Include a description
- Support different data types: Boolean, integer, string
- Support single value or multiple values
- Support user-defined free-form values or predefined values

- Assign custom security attributes to directory synced users from an on-premises Active Directory

Explore automatic user creation

Components of SCIM (System for Cross-Domain Identity Management)

- **HCM system** - Applications and technologies that enable Human Capital Management process and practices that support and automate HR processes throughout the employee lifecycle.
- **Microsoft Entra Provisioning Service** - Uses the SCIM 2.0 protocol for automatic provisioning. The service connects to the SCIM endpoint for the application, and uses the SCIM user object schema and REST APIs to automate provisioning and deprovisioning of users and groups.
- **Microsoft Entra ID** - User repository used to manage the lifecycle of identities and their entitlements.
- **Target system** - Application or system that has SCIM endpoint and works with the Microsoft Entra provisioning to enable automatic provisioning of users and groups.

Why use SCIM?

System for Cross-Domain Identity Management (SCIM) is an open standard protocol for automating the exchange of user identity information between identity domains and IT systems. SCIM ensures that employees added to the Human Capital Management (HCM) system automatically have accounts created in Microsoft Entra ID or Windows Server Active Directory. User attributes and profiles are synchronized between the two systems, updating, or removing users based on the user status or role change.

The key is keeping your identity systems up to date. If a user can be automatically deprovisioned from Microsoft Entra ID as soon as they're removed from your HR systems, you have less worry about a possible breach.

3 - Core architectural components of azure

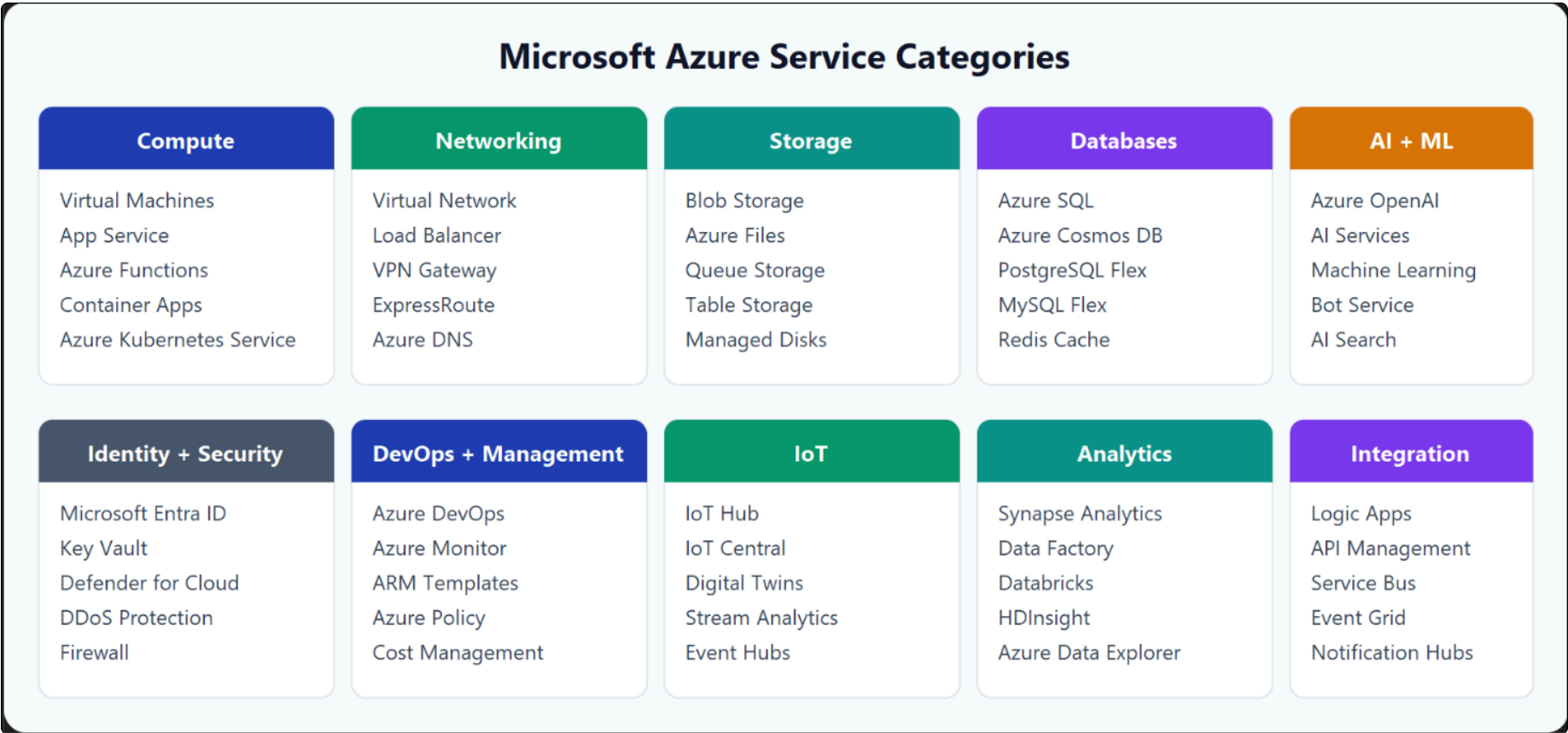
What is Microsoft Azure

Azure is a continually expanding set of cloud services that help you meet current and future IT challenges. Azure gives you the freedom to build, manage, and deploy applications on a massive global network using your favorite tools and frameworks.

What does Azure offer?

Limitless innovation. Build intelligent apps and solutions with advanced technology, tools, and services to take your operations to the next level. Seamlessly unify your technology to simplify platform management and deliver innovations efficiently and securely on a trusted cloud.

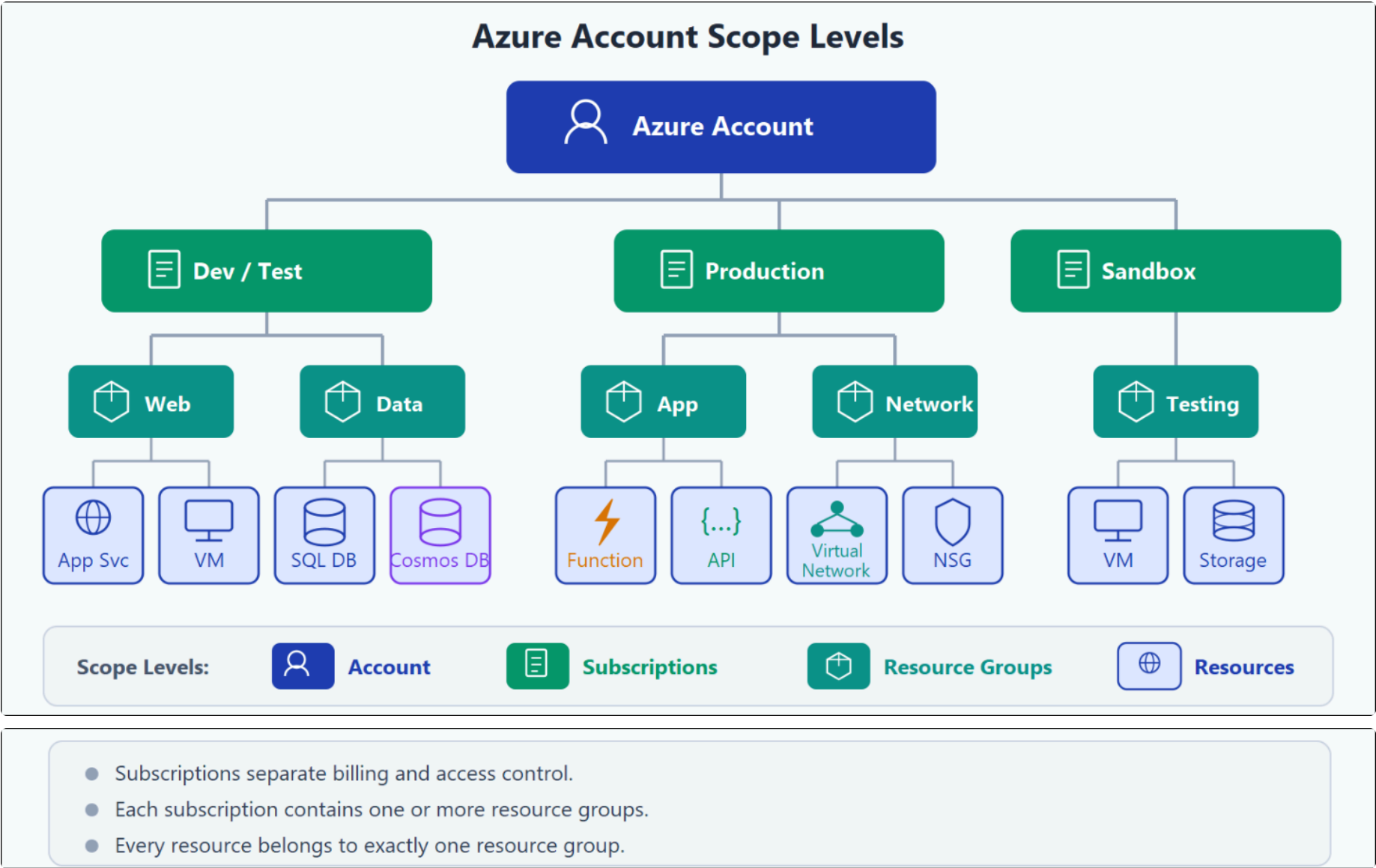
- **Bring ideas to life:** Build on a trusted platform to advance your team's capabilities with industry-leading AI and cloud services.
- **Seamlessly unify:** Efficiently manage all your infrastructure, data, analytics, and AI solutions across an integrated platform.
- **Innovate on trust:** Rely on trusted technology from a partner who's dedicated to security and responsibility.



Get started with Azure accounts

To create and use Azure services, you need an Azure subscription. When you're working with your own applications and workloads, you create an Azure account, and a subscription is created for you. After you've created an Azure account, you're free to create additional subscriptions.

After you've created an Azure subscription, you can start creating Azure resources within each subscription.



Create an Azure account

You can purchase Azure access directly from Microsoft by signing up on the Azure website or through a Microsoft representative. You can also purchase Azure access through a Microsoft partner. Cloud Solution Provider partners offer a range of complete managed-cloud solutions for Azure.

What is the Azure free account?

The Azure free account includes:

- Free access to popular Azure products for 12 months.
- A credit to use for the first 30 days.
- Access to more than 65 services that are always free.

What is the Azure free student account?

The Azure free student account offer includes:

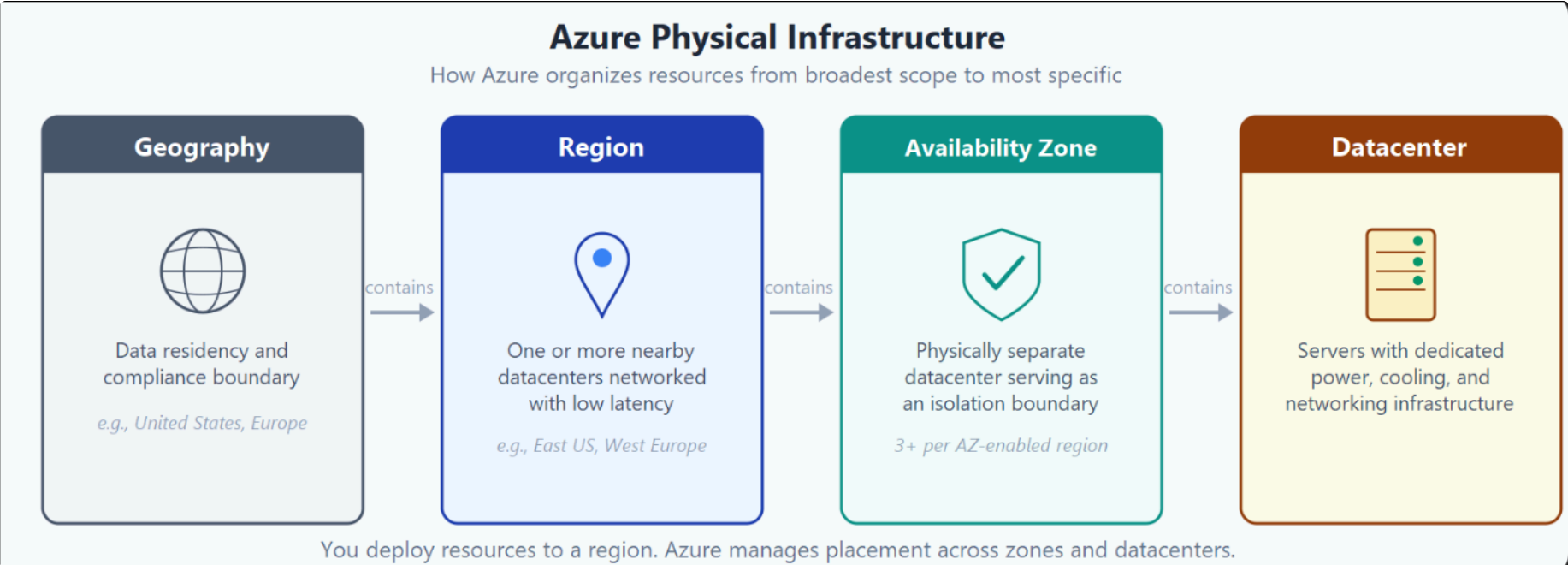
- Free access to certain Azure services for 12 months.
- A credit to use in the first 12 months.
- Free access to certain software developer tools.

Describe Azure physical infrastructure

Azure's core architectural components can be broken down into two main groupings: the physical infrastructure and the management infrastructure.

Physical infrastructure

The physical infrastructure for Azure starts with datacenters. These datacenters are facilities with servers arranged in racks, with dedicated power, cooling, and networking infrastructure — similar to an on-premises datacenter, but at a much larger scale.



Regions

A region is a geographical area on the planet that contains at least one, but potentially multiple datacenters that are nearby and networked together with a low-latency network. Azure intelligently assigns and controls the resources within each region to ensure workloads are appropriately balanced.

Availability Zones

Availability zones are physically separate datacenters within an Azure region. Each availability zone is made up of one or more datacenters equipped with independent power, cooling, and networking. An availability zone is set up to be an isolation boundary. If one zone goes down, the other continues working. Availability zones are connected through high-speed, private fiber-optic networks.

To ensure resiliency, a minimum of three separate availability zones are present in all availability zone-enabled regions. However, not all Azure Regions currently support availability zones.

Use availability zones for your workloads

With Azure, you can protect your workloads by spreading them across availability zones within a region.

You place your VMs, storage, databases, and other resources in one availability zone and replicate them to other zones within the same region. Keep in mind that there could be a cost to duplicating your services and transferring data between zones.

Azure services that support availability zones fall into three categories:

- Zonal services: You pin the resource to a specific zone (for example, VMs, managed disks, IP addresses).
- Zone-redundant services: The platform replicates automatically across zones (for example, zone-redundant storage, SQL Database).
- Non-regional services: Services are always available from Azure geographies and are resilient to zone-wide outages as well as region-wide outages.

Region pairs

Most Azure regions are paired with another region within the same geography (such as US, Europe, or Asia) at least 300 miles away. This approach allows for the replication of resources across a geography that helps reduce the likelihood of interruptions because of events such as natural disasters, civil unrest, power outages, or physical network outages that affect an entire region.

Not all Azure services automatically replicate data or automatically fall back from a failed region to cross-replicate to another enabled region. In these scenarios, recovery and replication must be configured by the customer.

Sovereign Regions

In addition to regular regions, Azure also has sovereign regions. Sovereign regions are instances of Azure that are isolated from the main instance of Azure. You may need to use a sovereign region for compliance or legal purposes.

Azure sovereign regions include:

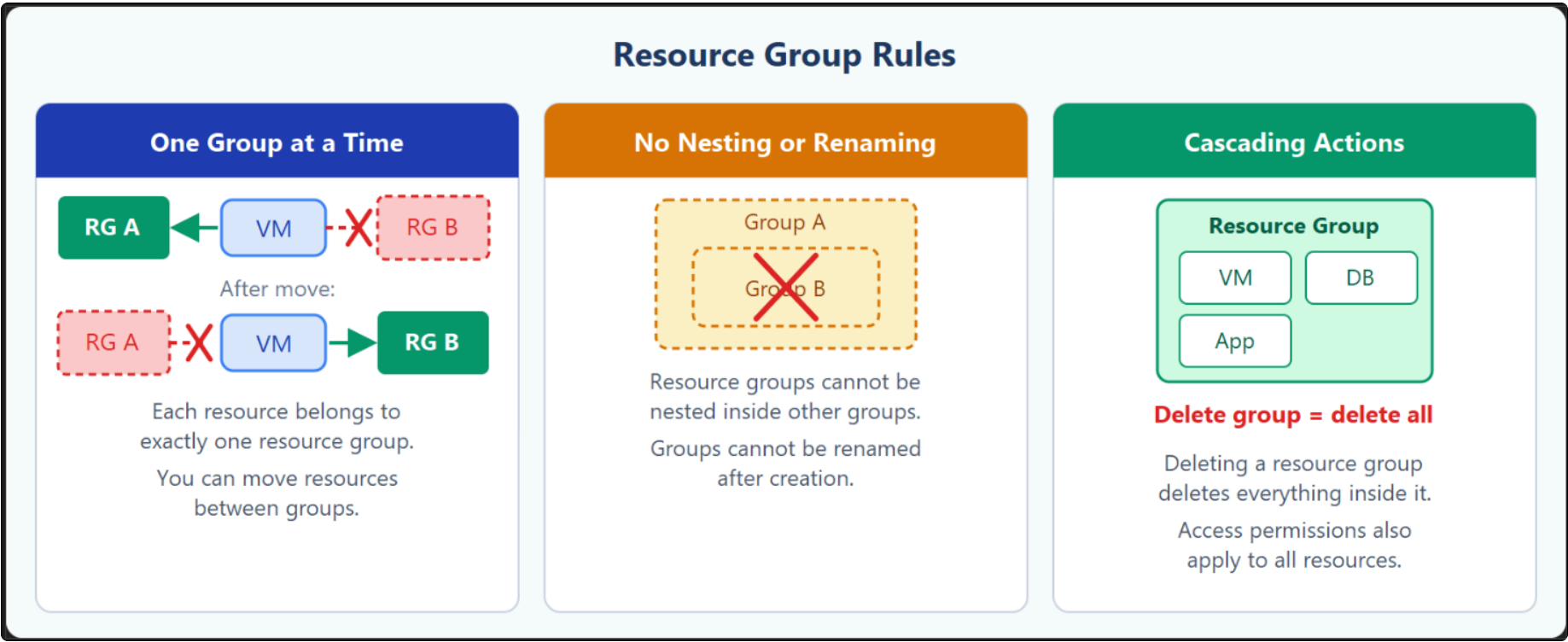
- US DoD Central, US Gov Virginia, US Gov Arizona, and more: These regions are physical and logical network-isolated instances of Azure for U.S. government agencies and partners. These datacenters are operated by screened U.S. personnel and include additional compliance certifications.
- China East, China North, and more: These regions are available through a unique partnership between Microsoft and 21Vianet, whereby Microsoft doesn't directly maintain the datacenters.

Describe Azure management infrastructure

The management infrastructure includes Azure resources and resource groups, subscriptions, and accounts. Understanding this hierarchy helps you organize resources, control who can access what, and manage costs as your Azure usage grows.

Azure resources and resource groups

A resource is the basic building block of Azure. Anything you create, provision, or deploy is a resource. VMs, virtual networks, databases, and Azure AI services are all examples of resources.



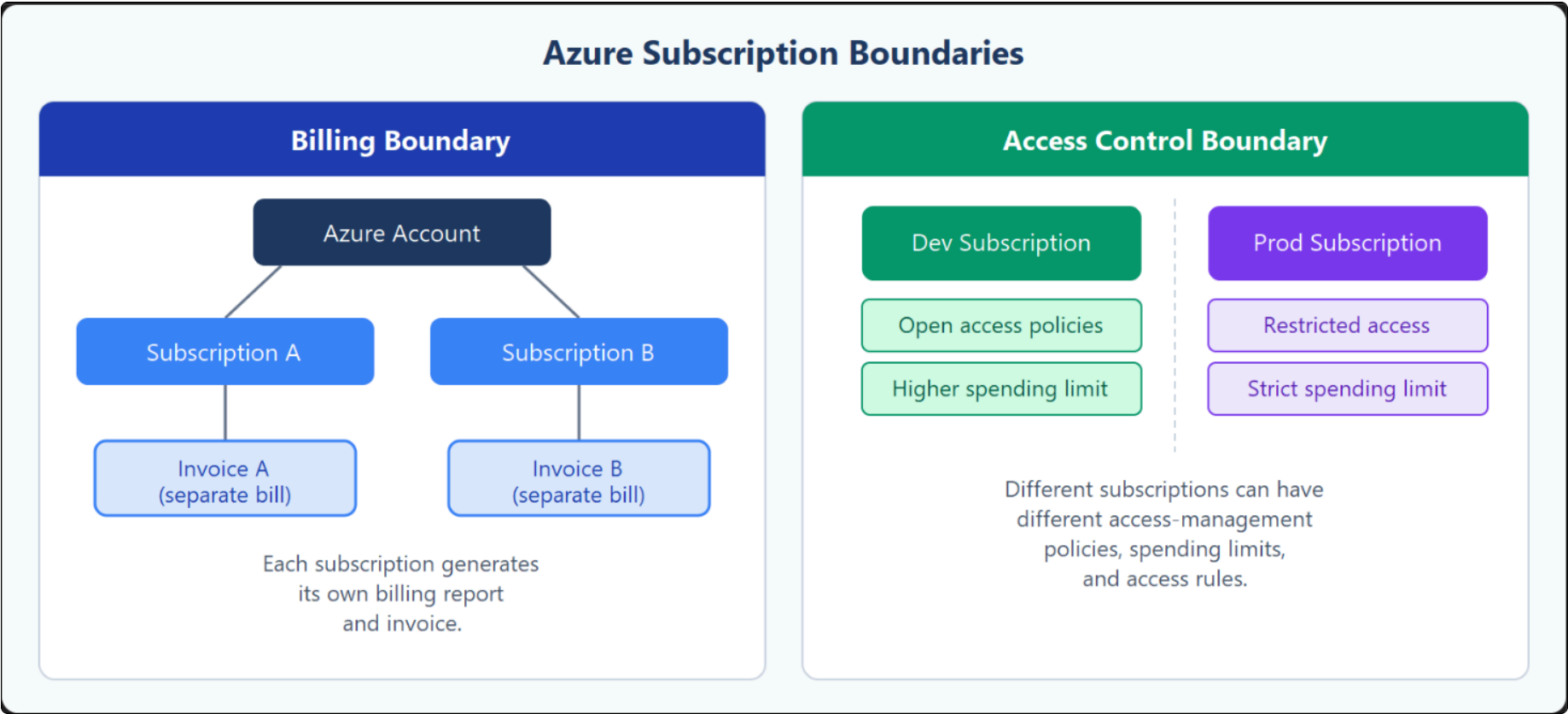
Actions you apply to a resource group affect all resources inside it. Deleting a resource group deletes everything in it. Granting or denying access applies to all its resources.

For example, if you're setting up a temporary dev environment, grouping all the resources together lets you delete the entire group when you're done. If you're running multiple projects, create a separate resource group for each so each team only sees and manages its own resources.

There are no hard rules for structuring resource groups — choose the approach that works best for your situation.

Azure subscriptions

In Azure, subscriptions are a unit of management, billing, and scale. Subscriptions let you organize resource groups and control billing separately from access.



An account can have multiple subscriptions, but only one is required. In a multi-subscription account, you can configure different billing models and access policies. There are two types of subscription boundaries:

- **Billing boundary:** Determines how an Azure account is billed. You can create multiple subscriptions for different billing requirements. Azure generates separate billing reports and invoices for each subscription.
- **Access control boundary:** Azure applies access-management policies at the subscription level. For example, you might create one subscription for your development work and another for production, each with different spending limits and access rules.

Create additional Azure subscriptions

You might create additional subscriptions to separate:

- **Environments:** Subscriptions for lifecycle stages such as sandbox, development, test, and production. Access control occurs at the subscription level, making this a natural boundary.
- **Team and workload boundaries:** Give each project its own subscription so costs are easy to track, or separate sandbox environments from production.
- **Billing:** Create subscriptions to track costs separately — for instance, one for production workloads and another for development and testing.

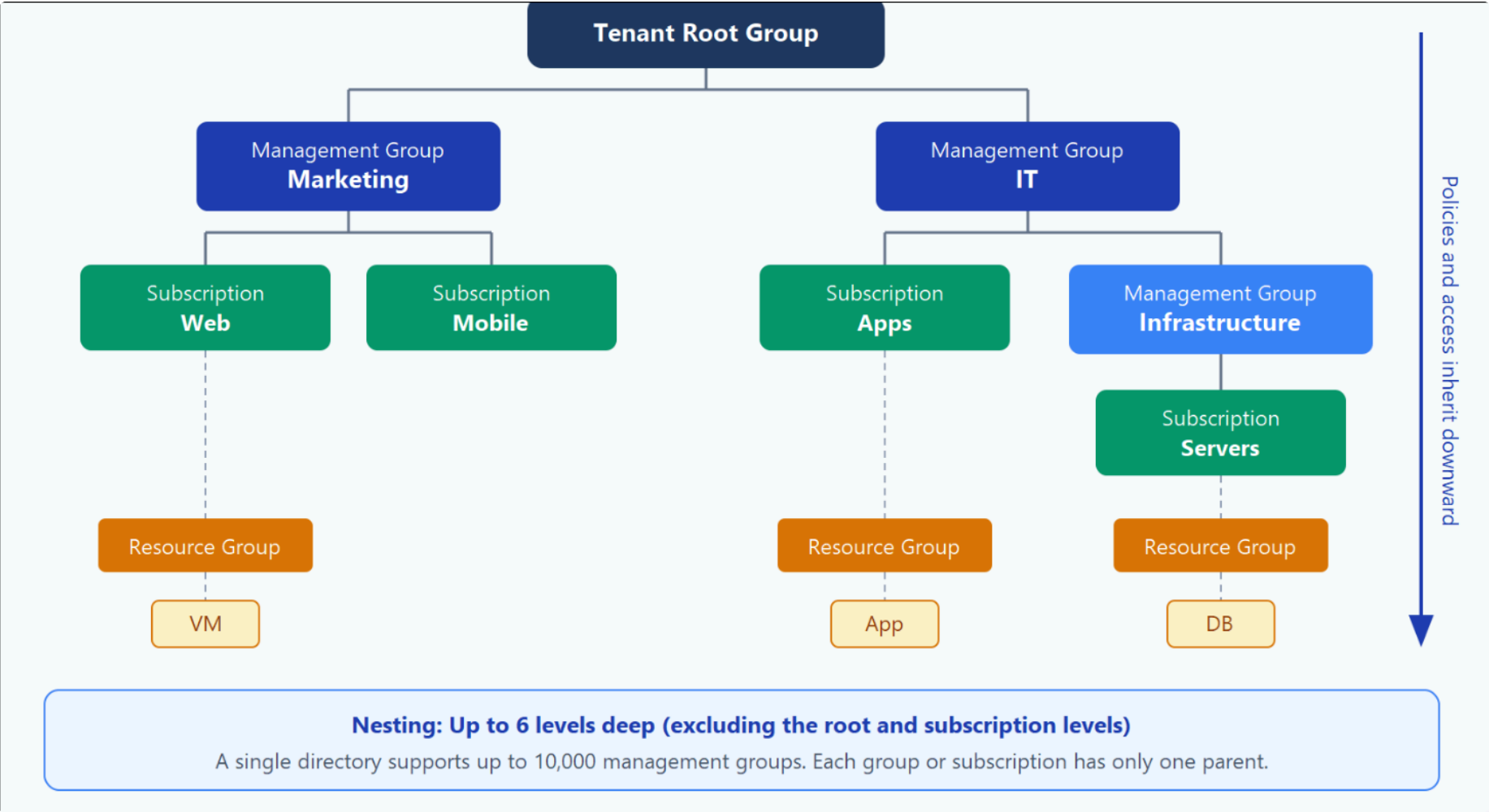
Azure management groups

Resources go into resource groups, and resource groups go into subscriptions. For a small environment, that's enough. But when you have many subscriptions across multiple teams or geographies, you need a way to manage access and policies at a higher level.

Azure management groups sit above subscriptions. You organize subscriptions into management groups and apply governance conditions — like access policies or compliance rules — to the group. All subscriptions in a management group automatically inherit those conditions, just as resources inherit settings from their resource group. Management groups can be nested up to six levels deep (not counting the root level or the subscription level), letting you build a hierarchy that mirrors your organization.

Management group, subscriptions, and resource group hierarchy

You can build a flexible structure of management groups and subscriptions to organize your resources into a hierarchy for unified policy and access management.



Examples of how you could use management groups:

- **Apply a policy across subscriptions.** You could limit VM locations to the US West Region in a group called Production. This policy inherits to all subscriptions under that management group and applies to all VMs in those subscriptions. The resource or subscription owner can't override it, which strengthens governance.

- **Grant access to multiple subscriptions at once.** By placing subscriptions under a management group, you can create one Azure RBAC assignment on the group. All sub-management groups, subscriptions, resource groups, and resources underneath inherit those permissions — no need to script Azure RBAC across individual subscriptions.

Important facts about management groups:

- A single directory supports up to 10,000 management groups.
- Each management group and subscription can have only one parent.

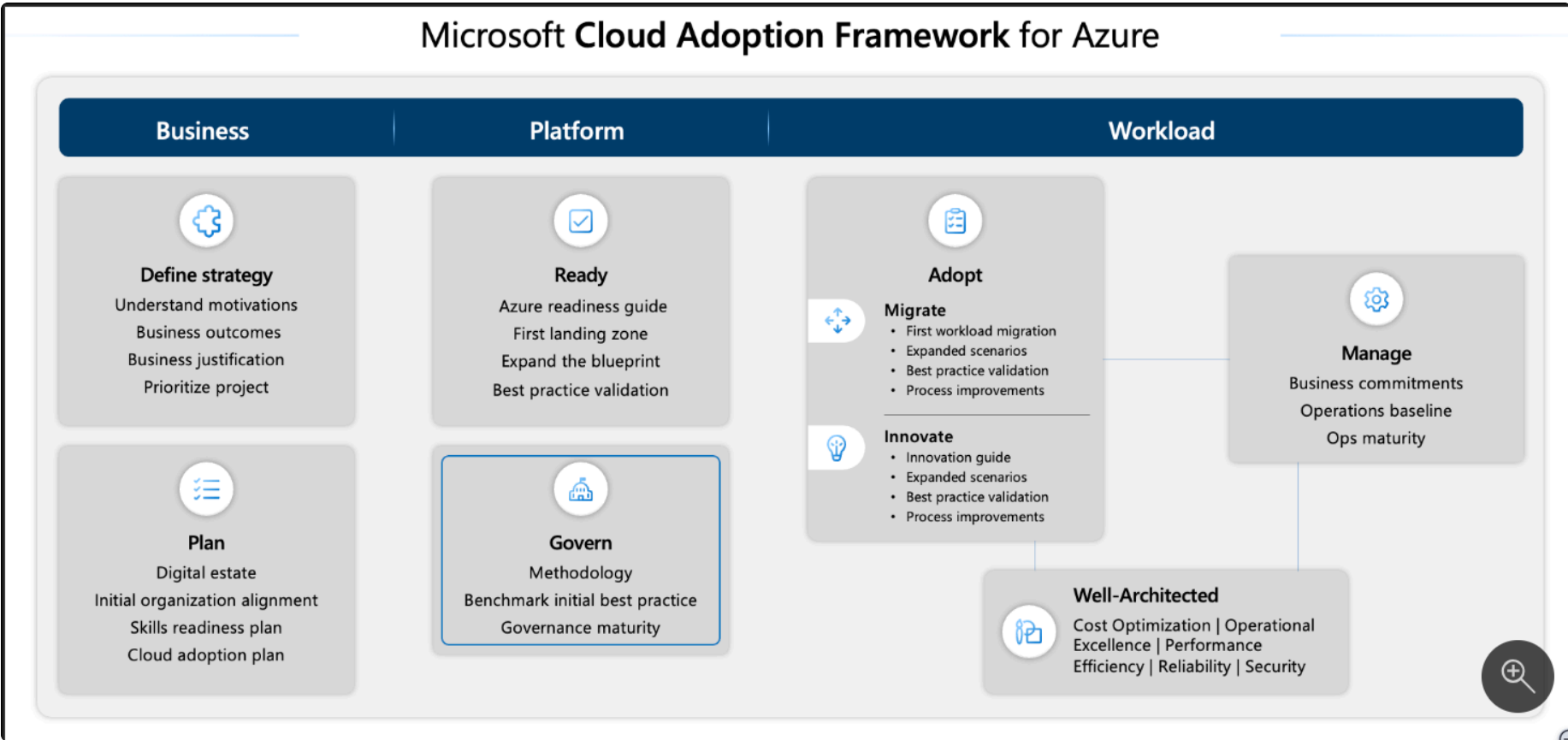
4 - Azure Policy initiatives

Azure Policy is a service that allows you to create, assign, and manage governance policies that enforce rules and effects over Azure resources to ensure that they stay compliant with your IT governance standards. These policies enforce various rules and effects on resources, ensuring that they adhere to corporate standards and service-level agreements. These policies are described in JSON format and are known as policy definitions. *Azure Policy is crucial for enforcing organizational standards and assessing compliance on a large scale.*

Azure Policy initiatives are a collection of Azure Policy definitions that are grouped together toward a specific goal or purpose. By consolidating multiple Azure policies into a single item, Azure Policy initiatives allow centralized control and enforcement of configurations across Azure resources.

Cloud Adoption Framework for Azure

The Microsoft Cloud Adoption Framework for Azure offers comprehensive technical guidance for Microsoft Azure. This end-to-end framework helps cloud architects, IT experts, and business leaders reach their cloud adoption objectives. Cloud Adoption Framework includes best practices, documentation, and tools that Microsoft employees, partners, and customers contribute to formulate and implement effective business and technology strategies for the cloud.



Steps for cloud governance

The Cloud Adoption Framework - Govern methodology divides cloud governance into five steps.

1. **Build a governance team** - Establish a dedicated cloud governance team that's responsible for defining, maintaining, and reporting on the progress of cloud governance policies.
2. **Assess cloud risks** - Conduct a thorough risk assessment that's unique to your organization, addressing all risk categories, including regulatory compliance, security, operations, costs,

data management, resource management, and AI-related risks.

- 3. **Document cloud governance policies** - Clearly document cloud governance policies that dictate acceptable cloud usage and outline the rules and guidelines that mitigate identified risks.
- 4. **Enforce cloud governance policies** - Implement a systematic approach to ensure compliance with cloud governance policies. Use automated tools alongside manual oversight to enforce compliance. These tools help set guardrails, monitor configurations, and ensure adherence to policies.
- 5. **Monitor cloud governance** - Regularly monitor cloud usage and the governance teams to ensure ongoing compliance with the established cloud governance policies.

Considerations for defining a cloud governance policy

The key considerations when defining a corporate cloud governance policy are as follows:

- **Business risk** – You must document the evolving business risks and the business's tolerance for risk based on data classification and application criticality.
- **Policy and compliance** – You must convert risk decisions into policy statements to establish cloud adoption boundaries efficiently.
- **Process** – You must establish processes to monitor violations and adherence to corporate policies.



The five core disciplines of cloud governance are as follows:

- **Cost management** – Evaluates and monitors costs, including controlling IT expenditures to establish well-defined cost management. It also includes adjusting resources according to demand. It's crucial to exercise control over cloud expenditure to derive greater value from your investments.
- **Security baseline** – Ensures compliance with IT security requirements by applying a security baseline to all adoption efforts.
- **Resource consistency** – Ensures consistency in resource configuration and enforcing practices for onboarding, recovery, and discoverability.
- **Identity baseline** – Ensures that the baseline for identity and access is enforced by consistently applying role definitions and assignments.
- **Deployment acceleration** – Accelerates the deployment of policies through centralization, consistency, and standardization across deployment templates.

Cloud governance with Azure Policy

Azure's primary governance tool is [Azure Policy](#). Azure Policy facilitates the governance of all resources, including current and forthcoming resources. It helps to enforce organizational standards and to assess compliance at scale by establishing guardrails across various resources.

Azure Policy allows for centralized management, allowing you to track compliance status and investigate changes leading to noncompliance. You can consolidate all compliance data into a singular repository, which simplifies auditing processes for effective cloud compliance and resource governance. The compliance dashboard offered by Azure Policy presents an aggregated view of the environment's overall state, with the ability to examine details at each resource and each policy level.

Some useful governance actions that you can enforce with Azure Policy include:

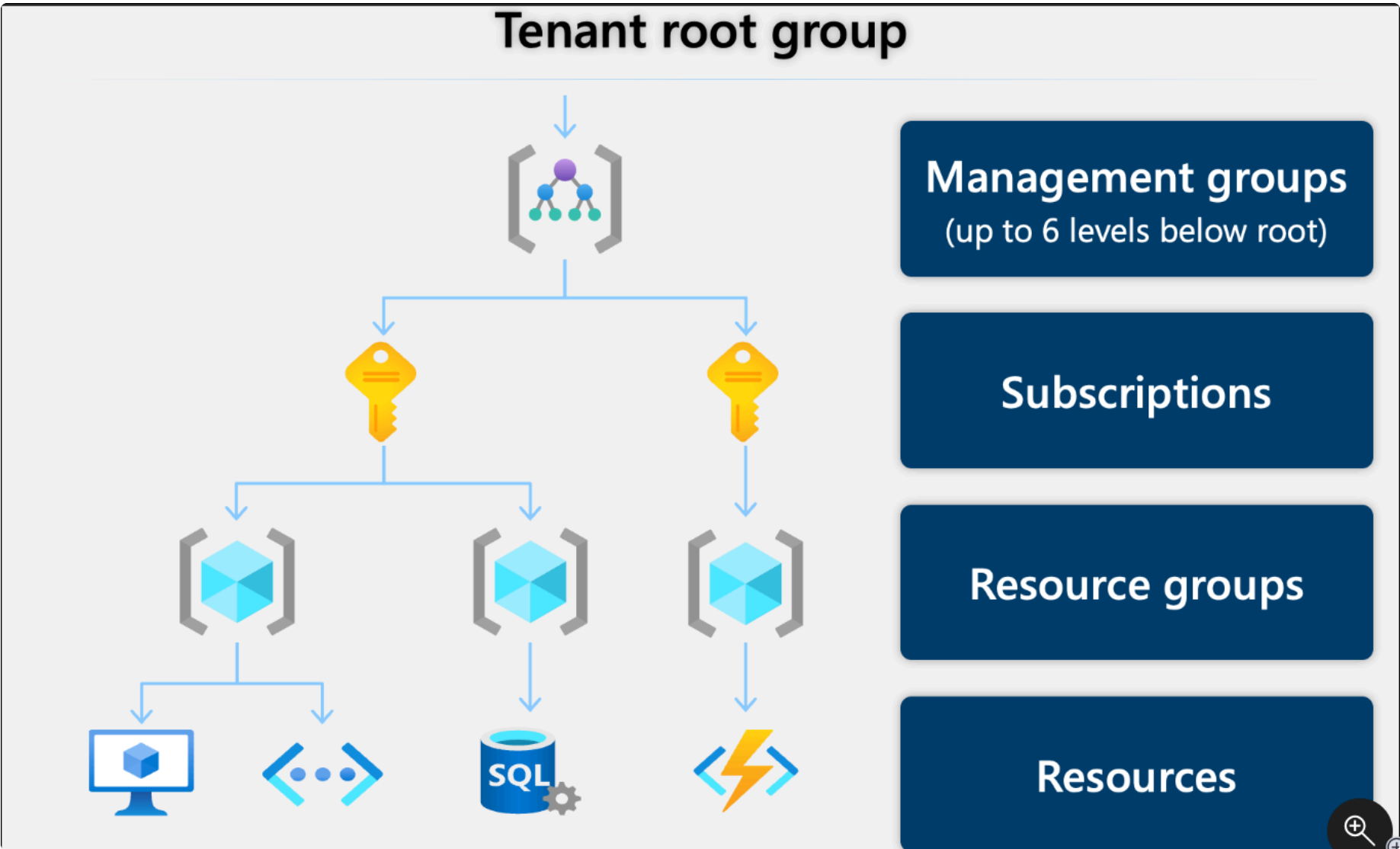
- Ensure that your team deploys Azure resources only to allowed regions.
- Enforce geo-replication rules to comply with your data residency requirements.
- Allow only certain virtual machine sizes for your cloud environment.
- Enforce the consistent application of taxonomic tags across resources.
- Recommend system updates on your servers.
- Allow multifactor authentication for all subscription accounts.
- Require resources to send diagnostic logs to an Azure Monitor Logs workspace.

Azure Policy evaluates your resources and highlights resources that aren't compliant with the policies that you create. Azure Policy can also prevent noncompliant resources from being created. Azure Policy comes with built-in policy and initiative definitions for Storage, Networking, Compute, Security Center, and Monitoring.

Azure Policy design principles

Hierarchy for governance

Azure provides four levels of management to establish proper governance: Management groups, Subscriptions, Resource groups, and Resources. You can build a flexible structure of management groups and subscriptions to organize your resources into a hierarchy for unified policy and access management.



The Azure structure starts with the tenant root group at the top, followed by a hierarchy of management groups, which can extend to six levels beneath the root.

Concept	Description
Resource	A resource is the basic building block of Azure, and it includes instances of services that you create, provision, deploy, and so on. Virtual machines (VMs), virtual networks, databases, AI services, and so on, are considered resources in Azure.
Resource groups	Resource groups are groupings of resources. When you create a resource, you must place it into a resource group. While a resource group can contain many resources, a single resource can only be in one resource group at a time. When you apply an action to a resource group, that action applies to all resources in the resource group. If you delete a resource group, all resources are deleted. If you grant or deny access to a resource group, all resources in the resource group are also granted or denied access.
Subscriptions	In Azure, subscriptions are a unit of management, billing, and scale. Similar to how resource groups are a way of logically organizing resources, subscriptions allow you to logically organize your resource groups and facilitate billing. Each subscription has limits or quotas on the number of resources that you can create and use. Organizations can use subscriptions to manage costs and the resources that users, teams, and projects create. Using Azure requires an Azure subscription. An Azure subscription provides you with authenticated and authorized access to Azure products and services. It also allows you to provision resources. An Azure subscription links to an Azure account, which is an identity in Microsoft Entra ID or in a directory that Microsoft Entra ID trusts.
Management groups	Azure management groups provide a level of scope above subscriptions. If you have many subscriptions, you might need a way to efficiently manage access, policies, and compliance for those subscriptions. You organize subscriptions into containers called management groups and then apply governance conditions to the management groups. Management groups give you enterprise-grade management at a large scale, no matter what type of subscriptions you might have. Management groups can be nested.

Introduction to Azure Resource Manager

Azure Resource Manager is the deployment and management service for Azure. It provides a management layer that allows you to create, update, and delete resources in your Azure account.

Azure operations are classified into two main types: control plane and data plane. The control plane helps you manage resources in your subscription, while the data plane allows you to access the capabilities provided by instances of specific resource types.

Control plane

Azure Policy operates in the control plane to enforce rules and compliance on your resources. Azure Resource Manager manages all control plane operations in Azure and includes the different components that are centralized between the different services. Azure Policy is integrated with Azure Resource Manager.

Data plane

The data plane is where the actual data operations occur, and Azure Policy ensures that the resources you interact with in the data plane are compliant with your policies. Data plane operations involve direct interaction with the data stored in a resource.

Service-specific access controls, such as RBAC or access control lists (ACLs), often manage data plane permissions. The service responds with the data or result of the data operation, ensuring that the requester has the correct permissions.

Azure Policy allows individual Azure services to implement an Azure Policy extension, enhancing policy behavior and integration with specific resource providers. Azure Policy currently supports data plane operations through the following resource provider modes:

- **Microsoft.Kubernetes.Data** - Used for managing Kubernetes clusters and components such as pods, containers, and ingresses.
- **Microsoft.KeyVault.Data** - Used for managing vaults and certificates in Azure Key Vault.
- **Microsoft.Network.Data** - Used for managing Microsoft Azure Virtual Network Manager custom membership policies by using Azure Policy.
- **Microsoft.ManagedHSM.Data** - Used for managing Azure Key Vault Managed HSM keys by using Azure Policy.
- **Microsoft.DataFactory.Data** - Used for using Azure Policy to deny Microsoft Azure Data Factory outbound traffic domain names.
- **Microsoft.MachineLearningServices.v2.Data** - Used for managing Microsoft Azure Machine Learning model deployments. This Resource Provider mode reports compliance for newly created and updated components.

Operation flows of Azure Resource Manager

Azure Resource Manager includes two scenarios for handling Azure requests: **Greenfield** and **Brownfield**. As you deploy resources, Azure Resource Manager understands when to create new resources and when to update existing resources.

Greenfield refers to a scenario where an Azure Policy (policy-first) exists, and when you're creating or updating an Azure resource.

Brownfield is the scenario where the resources exist already (resource-first), and you're assigning a new Azure Policy to those resources.

In this case, policy evaluation happens through a compliance scan, which runs automatically every 24 hours, or it can be manually triggered.

Azure Policy resources

Definitions

Azure Policy definitions describe resource compliance conditions and the effect to take if a condition is met. Several settings determine which resources are evaluated by any Azure Policy.

Scope in Azure Policy is the same as the levels of hierarchy for governance in Azure. Four levels of management scope are under the root tenant: the management groups, subscriptions, resource groups, and resources. The definition might be saved in a management group or a subscription. The definition location determines the scope to which the initiative or policy can be assigned. An assignment also has several properties that set a scope. The use of these properties determines which resource for Azure Policy to evaluate and which resources count toward compliance

Initiatives

Azure Policy initiatives, also known as a policy set, allow you to group several policy definitions to simplify assignments and management because you work with the initiatives as a single item. Initiatives offer a streamlined and automated approach to governance, allowing organizations to manage and monitor compliance at scale.

The initiative definition contains all policy definitions to help track your compliance state for a larger goal, such as organizational compliance goals or compliance with regulatory frameworks.

Built-in policy is a type of policy definition that's generated by Azure Resource Providers and is available by default. A group of such policy definitions is known as a **built-in initiative**. Azure built-in policy initiatives are a powerful tool set that enables centralized control across Azure resources and enforcement of specific configurations.

For a list of built-in policies and initiatives, see [Policies](#) and [Initiatives](#).

Custom policy is a type of policy definition that's written by a policy user when no built-in policy maps to your requirements. A group of such policy definitions is known as a **custom initiative**. Azure Policy custom initiatives help you to tailor a set of policies specifically to your organization's unique requirements, giving you control to enforce the standards and rules that best fit your environment.

Assignments

Policy assignments define which resources are evaluated by a policy definition or initiative. Policy assignments can be done in the portal, an API call, or through the command line interface.

Policy and initiative definitions are deployed to a definition location (management group or subscription). This location determines the scope to which the initiative or policy can be assigned. The location should be the resource container shared by all resources that you want to use the policy definition on.

Policies and initiatives are assigned to a specific scope (management group, subscription, or resource group). While doing so, you can define several optional aspects, including the resource scope and policy definition.

- Optional *resource selectors* to allow gradual rollout based on resource location or type.
- Optional *overrides* to change the effect of a policy definition without modifying the underlying definition.
- *enforcementMode* can be disabled to support "what-if" scenarios without changing the definition, which is equivalent to changing the definition to an audit effect mode, but a way to do it at assignment level. For example, if the policy has *Deny* effects, that denial isn't effective, but you can still view the result of the compliance evaluation of that policy.
- Optional *excluded scopes* to exclude inner containers or resources from the assignment scope.
- *Noncompliance messages* can be defined.
- *Parameters* can be assigned values.
- If you have a policy with the *deployIfNotExists* effect type, a *managed identity* can be assigned (system-assigned or user-assigned) to turn on remediation actions. An assignment has several properties that set a scope. The use of these properties determines which resource for Azure Policy to evaluate and which resources count toward compliance. These properties map to the following concepts:
 - **Inclusion** - For more information, see [Azure Policy assignment structure](#).
 - **Exclusion** - For more information, see [Azure Policy assignment structure excluded scopes](#).

Exemptions

Use the Policy exemptions feature to *exempt* a resource hierarchy or an individual resource from evaluation of initiatives or definitions. Resources that are *exempt* count toward overall compliance but can't be evaluated or have a temporary waiver. They're created as a child object on the resource hierarchy, or the individual resource granted the exemption.

Policy exemptions aren't created during assignment time, but after, and the effect is still the same as an excluded scope. Two exemption categories exist and are used to group exemptions:

- **Mitigated** - The exemption is granted because the policy intent is met through another method.
- **Waiver** - The exemption is granted because the noncompliance state of the resource is temporarily accepted.

Azure Policy definitions

Azure Policy definition describes resource compliance conditions and the action or effects that take place if those conditions are met. The policy consists of two parts:

- A **condition** that compares a resource property field or a value, accessed by using aliases, to a required value.
- The **effect** determines what happens when the policy rule is evaluated to match the condition. For each new resource, an updated resource, or an existing resource, the effects behave differently.

Anatomy of a policy definition

Element	Description	Properties or values
<i>displayName</i> (string, max 128 characters)	Used to identify the policy definition.	
<i>description</i> (string, max 512 characters)	Provides context for when the definition is used.	
<i>policyType</i> (read-only string)	Indicates the origin of the policy definition. This property can't be set, but SDK returns three values which are visible in the portal.	• Built in: Provided and maintained by Microsoft. • Custom: Custom definitions created by the customer. • Static: Regulatory Compliance policy with Microsoft ownership.

1.1 - Introduction to Azure Cloud Shell		
Element	Description	Properties or values
<i>mode (string)</i>	Configured depending on the target of the policy: an Azure Resource Manager property or a Resource Provider property.	• Resource Manager Modes: - o On All: Evaluates resource groups, subscriptions, and all resource types. - o Indexed: Evaluates resource groups, subscriptions, and all resource types. • Resource Provider Modes (limited to Built in policies and fully supported): - o Microsoft.Kubernetes.Data - o Microsoft.KeyVault.Data - o Microsoft.Network.Data • Resource Provider Modes (limited to Built in policies and in preview mode): - o Microsoft.ManagedHSM.Data - o Microsoft.DataFactory.Data
<i>version (string, optional)</i>	Built-in policy definitions can host multiple versions with the same definitionID. If no version number is specified, all experiences show the latest version of the definition.	
<i>metadata (object, optional, max 1,024 characters)</i>	Stores information about the policy definition.	Common properties (for Built-in policies): • <i>version (string)</i>: Tracks details about the version of the contents of a policy definition. • <i>category (string)</i>: Determines under which category in the Azure portal the policy definition is displayed. • <i>preview (Boolean)</i>: True or false flag that indicates if the policy definition is in preview. • <i>deprecated (Boolean)</i>: True or false flag that indicates if the policy definition is deprecated. • <i>portalReview (string)</i>: Determines if parameters require review in the portal.
<i>parameters (object, optional)</i>	Help simplify your policy management by reducing the number of policy definitions. By including parameters in a policy definition, you can reuse that policy for different scenarios by using different values.	Properties: • name • type (String, Array, Object, Boolean, Integer, Float, DateTime) • metadata (description, displayName, strongType, assignPermissions) • defaultValue • allowedValues • schema

Element	Description	Properties or values
<i>policyRule</i> (object)	The effect of a policy is defined in the <i>policyRule</i>. The policy rule consists of the <i>if and then</i> blocks. • In the <i>if</i> block, you define one or more conditions that specify when the policy applies. • In the <i>then</i> block, you define the effect that happens when the <i>if</i> conditions result true.	

```
{
  "displayName": "Allowed locations",
  "description": "This policy enables you to restrict the locations your organization can specify when deploying resources. Use to enforce your geo-compliance requirements. Excludes resource groups, Microsoft.AzureActiveDirectory/b2cDirectories, and resources that use the 'global' region.",
  "policyType": "BuiltIn",
  "mode": "Indexed",
  "metadata": {
    "version": "1.0.0",
    "category": "General"
  },
  "parameters": {
    "listOfAllowedLocations": {
      "type": "Array",
      "metadata": {
        "description": "The list of locations that can be specified when deploying resources.",
        "strongType": "location",
        "displayName": "Allowed locations"
      }
    }
  },
  "policyRule": {
    "if": {
      "allOf": [
        {
          "field": "location",
          "notIn": "[parameters('listOfAllowedLocations')]"
        },
        {
          "field": "location",
          "notEquals": "global"
        },
        {
          "field": "type",
          "notEquals": "Microsoft.AzureActiveDirectory/b2cDirectories"
        }
      ]
    },
  },
}
```

```
    "then": {
      "effect": "deny"
    }
  }
}
```

Logical operators supported in the *if* block

In the *if* condition, you can put different logical operators.

Operator	Type	Description
<i>not</i>	{condition or operator}	The <i>not</i> syntax inverts the result of the condition.
<i>allOf</i>	[{condition or operator}, {condition or operator}]	The <i>allOf</i> syntax (like the logical <i>and</i> operation) requires all conditions to be true.
<i>anyOf</i>	[{condition or operator}, {condition or operator}]	The <i>anyOf</i> syntax (like the logical <i>or</i> operation) requires one or more conditions to be true.

Conditions

Properties like fields, values, or counts can be evaluated within a condition.

Fields	Conditions that evaluate whether the values of properties in the resource request payload meet certain criteria can be formed by using a field expression.	Name, fullName, kind, type, location, ID, identity.type, tags, tags['tagName'], property aliases
Value	Conditions that evaluate whether a value meets certain criteria can be formed by using a value expression.	
Count	Conditions that count how many members of an array meet certain criteria can be formed by using a count expression.	- Field count, value count - The current () function returns the value of the array member that's being evaluated

Evaluation criteria	Value type
<i>equals</i>	stringValue
<i>notEquals</i>	stringValue
<i>like</i>	stringValue
<i>notLike</i>	stringValue
<i>match</i>	stringValue
<i>notMatch</i>	stringValue
<i>matchInsensitively</i>	stringValue
<i>notMatchInsensitively</i>	stringValue
<i>contains</i>	stringValue
<i>notContains</i>	stringValue
<i>In</i>	["stringValue1", "stringValue2"]
<i>notIn</i>	["stringValue1", "stringValue2"]
<i>containsKey</i>	keyName
<i>notContainsKey</i>	keyName
<i>less</i>	dateValue

Evaluation criteria	Value type
<i>less</i>	stringValue
<i>less</i>	intValue
<i>lessOrEquals</i>	dateValue
<i>lessOrEquals</i>	stringValue
<i>lessOrEquals</i>	intValue
<i>greater</i>	dateValue
<i>greater</i>	stringValue
<i>greater</i>	intValue
<i>greaterOrEquals</i>	dateValue
<i>greaterOrEquals</i>	stringValue
<i>greaterOrEquals</i>	intValue
<i>exists</i>	bool

Policy functions

Functions can be used to introduce extra logic into a policy rule. They're resolved in the policy rule of a policy definition and in the parameter values that are assigned to the policy definitions in an initiative.

The Resource Manager template functions are available to use in a policy rule except a [few policy functions and user-defined functions](#).

The following table describes the functions that are only available in policy rules.

Function	Description
<code>addDays(dateTime, numberOfDaysToAdd)</code>	<code>dateTime</code> : [Required] string - String in the Universal ISO 8601 DateTime format 'yyyy-MM-ddTHH:mm:ss.FFFFFFFZ'. <code>numberOfDaysToAdd</code> : [Required] integer - Number of days to add.
<code>Field(fieldName)</code>	<code>fieldName</code> : [Required] string - Name of the field to retrieve. • Returns the value of that field from the resource evaluated by the <i>If</i> condition. • <code>field</code> is primarily used with <code>auditIfNotExists</code> and <code>deployIfNotExists</code> to reference fields on the resource that are being evaluated.
<code>requestContext().apiVersion</code>	Returns the API version of the request that triggered policy evaluation. This value is the API version that was used in the PUT/PATCH request for evaluations on resource creation/update. The latest API version is always used during compliance evaluation on existing resources.
<code>policy()</code>	Returns the following information about the policy that's being evaluated. Properties can be accessed from the returned object. <pre>"assignmentId": "", "definitionId": "", "setDefinitionId": "", "definitionReferenceId": ""</pre>

Function	Description
<code>ipRangeContains(range, targetRange)</code>	<code>range</code> : [Required] string - String specifying a range of IP addresses to check if the <i>targetRange</i> is within range. <code>targetRange</code> : [Required] string - String specifying a range of IP addresses to validate as included within the <i>range</i>. Returns a <i>boolean</i> for whether the <i>range</i> IP address range contains the <i>targetRange</i> IP address range. Empty ranges or mixing between IP families isn't allowed and results in evaluation failure.
<code>current(indexName)</code>	Special function that can only be used inside count expressions.

Effect types (*then* blocks)

The second part of the *policyRule* in an Azure Policy definition is the *then* block. This block defines the effect that takes place when the policy rule is evaluated to match the condition resources. More than one effect can be valid for a given policy definition. Parameters are often used to specify allowed effect values (*allowedValues*) in such cases so that a single definition can be more versatile during assignment.

Effect	Description	Type
<i>disabled</i>	The <i>disabled</i> effect is a way to deactivate the policy. If a policy definition has <i>Disabled</i> as its effect, any assignments of that policy aren't active. This effect is checked first to determine if the policy rule should be evaluated. This flexibility makes it possible to deactivate a single assignment instead of deactivating all of that policy's assignments.	Synchronous evaluation
<i>append</i>	The <i>append</i> effect is used to add more fields to the requested resource during creation or update. It's mostly obsolete because <i>Modify</i> can also be used to add fields to the request.	Synchronous evaluation
<i>modify</i>	The <i>modify</i> effect is used to add, update, or remove properties or tags on a subscription or resource during creation or update. It allows Azure Policy to modify requests to Azure Resource Manager by altering fields to ensure compliance.	Synchronous evaluation
<i>deny</i>	The <i>deny</i> effect is used to prevent a resource request that doesn't match defined standards through a policy definition and fails the request.	Synchronous evaluation
<i>denyAction</i>	The <i>denyAction</i> effect is used to block requests based on intended action to resources at scale. Currently, the only supported action is DELETE.	Synchronous evaluation
<i>audit</i>	The <i>audit</i> effect is used to create a warning event in the activity log when you're evaluating a noncompliant resource, but it doesn't stop the request.	Asynchronous evaluation
<i>auditIfNotExists</i>	The <i>auditIfNotExists</i> effect allows the auditing of resources that are related to the resource that matches the <i>if</i> condition but doesn't have the properties specified in the details of the <i>then</i> condition.	Asynchronous evaluation
<i>deployIfNotExists</i>	The <i>deployIfNotExists</i> policy definition runs a template deployment when the condition is met. It can trigger deployment of a related resource based on the compliance state of the currently evaluated resource.	Asynchronous evaluation

Effect	Description	Type
<i>manual</i>	The <i>manual</i> effect allows you to self-attest the compliance of resources or scopes. When a policy definition with <i>Manual</i> effect is assigned, you can set the compliance states of targeted resources or scopes through custom attestations.	Manual attestation

The following list provides general guidance around interchangeable effects:

- *audit*, *deny*, and either *modify* or *append* are often interchangeable.
- *auditIfNotExists* and *deployIfNotExists* are often interchangeable.
- *manual* isn't interchangeable.
- *disabled* is interchangeable with any effect.

Evaluation of resources through Azure Policy

A significant benefit of Azure Policy is the insight and controls that it provides over resources in a subscription or management group of subscriptions. This control can be used to prevent resources from being created in the wrong location, enforce common and consistent tag usage, or audit existing resources for appropriate configurations and settings.

Evaluation triggers

Evaluations of assigned policies and initiatives happen as the result of various events:

- A policy or initiative is newly assigned to a scope
- A policy or initiative already assigned to a scope is updated
- A resource is deployed to or updated in a scope with an assignment through Azure Resource Manager, REST API, or a supported SDK
- A subscription (resource type Microsoft.Resources/subscriptions) is created or moved in a management group hierarchy with an assigned policy definition that targets the subscription resource type
- A policy exemption is created, updated, or deleted
- Standard compliance evaluation cycle
- The machine configuration resource provider is updated with compliance details by a managed resource
- On-demand scan

Evaluation timing

When you're working with policy assignments in Azure, you need to understand the behavior and timing of compliance scans, especially in Brownfield scenarios, where new policies are applied to existing resources. Compliance scans through Azure policies are triggered by various methods:

- **Automatic full scan** - A full compliance scan is triggered automatically every 24 hours.
- **Manual scan for Brownfield scenarios** - In cases where a new policy is applied to existing resources (Brownfield scenarios), you can manually trigger a compliance scan by running *az policy state trigger-scan*.

After the scan starts, several factors influence how long it takes for a compliance scan to complete:

- **Policy definitions** - The size and complexity of the policy definitions can increase scan time.
- **Number of policies** - The more policies applied, the longer the scan might take.

- **Scope size** - The size of the resource scope assigned to the policy also plays a role.
- **System load** - Compliance scans are a low-priority operation, meaning that if the system is busy with more critical tasks, the scan might take longer. The system prioritizes interactive and high-importance operations, so scans might take several minutes, or tens of minutes, even in smaller environments.
- **Synchronous scan (Low-Priority Execution)** - Because compliance scans are synchronous and assigned a low priority in Azure's system, they're delayed if the system is busy. This scan can significantly extend the time it takes for the scan to complete, even for smaller scopes or policies.

Resource compliance states

When initiative or policy definitions are assigned, Azure Policy determines which resources are applicable. Then, it evaluates those resources that aren't excluded or exempted. Evaluation provides one of the compliance states to each resource based on conditions in the policy rule and each resource's adherence to those requirements.

- **Non-compliant**
- **Compliant**
- **Error** (for template or evaluation error)
- **Conflicting** (two or more policy assignments in the same scope with contradicting rules, such as two policies appending the same tag with different values)
- **Protected** (resource covered under an assignment with a *denyAction* effect)
- **Exempted Unknown** (default state for definitions with a *manual* effect)

The compliance percentage is determined by dividing **Compliant**, **Exempt**, and **Unknown** resources by total resources. Total resources include resources with **Compliant**, **Non-compliant**, **Unknown**, **Exempt**, **Conflicting**, and **Error** states.

Enforcement Mode

enforcementMode is a property of a policy assignment that lets you deactivate the enforcement of certain policy effects. This mode allows you to test the policy's outcome on existing resources without initiating the policy effect or triggering entries in the [Azure Activity log](#). The *enforcementMode* can be changed to Enabled after the policy is thoroughly tested.

The enforcementMode is different from the disabled effect. The disabled effect prevents resource evaluation from happening at all while enforcementMode lets the evaluation happen without the effect taking place.

Mode	JSON value	Type	Remediate manually	Activity log entry	Description
Enabled	Default	string	Yes	Yes	The policy effect is enforced during resource creation or update.
Disabled	DoNotEnforce	string	Yes	No	The policy effect isn't enforced during resource creation or update.

Policy enforcement and safe deployment best practices

Without the appropriate knowledge of best practices and proper testing, applying a set of policy to an existing environment that's running production workloads can result in unintended behaviors of policy resources. Treating policy as code (keeping your policy definitions in source

control, and whenever a change is made, testing and validating that change) allows you to automate testing and make sure that no manual error factor happens. The best practices framework focuses on minimizing the impact of policy changes while ensuring compliance, and it includes two aspects:

- **First aspect** - Start from Assignments of new policies with *enforcementMode* Disabled. When assigning policies that include deny or modify actions, beginning with *enforcementMode* Disabled allows you to view the compliance state and evaluate policy outcomes without triggering actions or denying operations. This "what-if" scenario minimizes impact and helps identify issues in the new policies or changes without disrupting the environment.
- **Second aspect** - Deploy policies in deployment rings. To control potential negative impacts, policies should be deployed gradually in smaller subsets and then in bigger sets. You can start with test and development environments and then move to production by applying the policy to a small subset first. This strategy helps in testing the policy thoroughly. Gradually expanding the scope (through deployment rings) can cover the full production environment.

The following steps correspond with the outlined steps in the preceding screenshot:

1. **Create definition** - Begin by defining the policy definition with the scope as root (tenant).
2. **Create assignment** - Define deployment rings (1 to 5) by using resource selectors. Assign the policy to a specific scope (such as a resource group, subscription, or management group) in Ring 5. Assign with *enforcementMode* Disabled to evaluate compliance without enforcing changes.
3. a) **Compliance check** - Verify that the policy is being applied correctly and that the desired compliance state is achieved for the resources in Ring 5.

b) **Application health check** - Assess the impact of the policy for the resources in Ring 5. Ensure that no unexpected side effects exist.
4. **Repeat for each ring (Non-production)** - Repeat step 3 for all nonproduction environment rings.
5. **Update assignment (Optional)** - If necessary, adjust the policy definition or assignment based on the evaluation of resources of the nonproduction environment and then reassign it to the resources in Ring 5 with the *enforcementMode* Enabled.
6. a) **Compliance check** - Reevaluate compliance after making changes (same as step 3a).

b) **Application health check** - Again, verify that the policy isn't causing issues (same as step 3b).
7. **Repeat for each ring (Non-production)** - Repeat step 6 for all nonproduction environment rings.
8. **Repeat for production rings** - After the policy is validated in a nonproduction environment, gradually deploy it to production environments, starting with a smaller subset (ring) and expanding the scope over time.

Reacting to policy state changes

Azure Policy events allow applications to react to state changes. This integration is done without the need for complicated code or expensive and inefficient polling services. Events from Azure Policy (Event Source) are pushed through Microsoft Azure Event Grid to Event Handlers.

Azure Event Grid provides reliable delivery services to your applications through rich retry policies and dead-letter delivery. Event Grid takes care of the proper routing, filtering, and multicasting of the events to destinations through Event Grid subscriptions.

An Event Handler is the place where the event is sent. Several services, such as Microsoft Azure Functions, Microsoft Azure Logic Apps, or your own custom HTTP listener, can be configured to handle events. You can also use any webhook for handling events.

5 - Secure azure with RBAC

Introduction

Azure RBAC is an authorization system in Azure that helps you manage who has access to Azure resources, what they can do with those resources, and where they have access.

Azure subscriptions

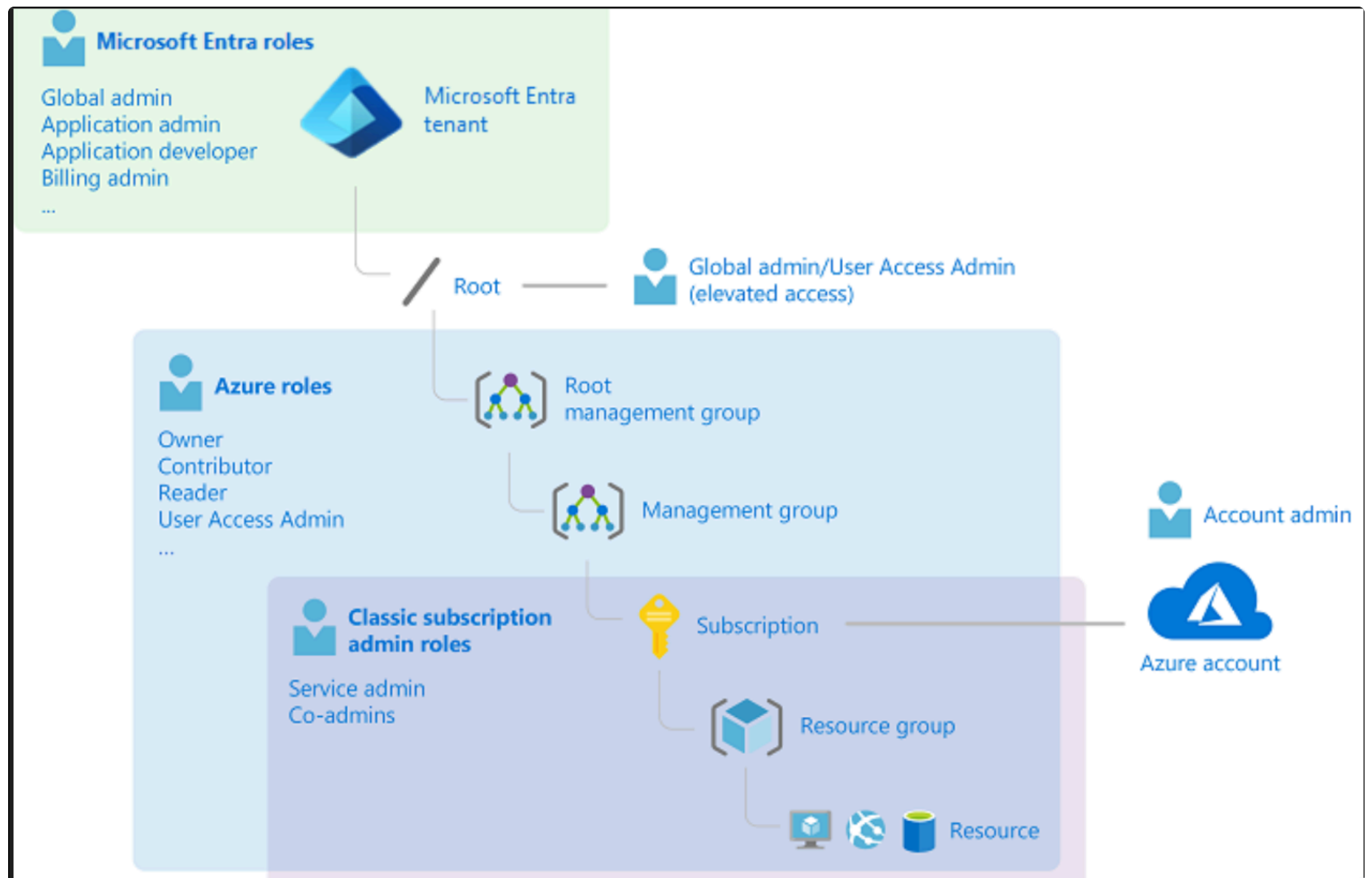
First, remember that each Azure subscription is associated with a single Microsoft Entra directory. Users, groups, and applications in that directory can manage resources in the Azure subscription. The subscriptions use Microsoft Entra ID for single sign-on (SSO) and access management. You can extend your on-premises Active Directory to the cloud by using **Microsoft Entra Connect**. This feature allows your employees to manage their Azure subscriptions by using their existing work identities.

What's Azure RBAC?

Azure RBAC is an authorization system built on Azure Resource Manager that provides fine-grained access management for resources in Azure. With Azure RBAC, you can grant the exact access that users need to do their jobs. For example, you can use Azure RBAC to let one employee manage virtual machines in a subscription, while another manages SQL databases within the same subscription.

You can grant access by assigning the appropriate Azure role to users, groups, and applications at a certain scope. The scope of a role assignment can be a management group, subscription, a resource group, or a single resource. A role assigned at a parent scope also grants access to the child scopes contained within it.

The Azure role that you assign dictates what resources the user, group, or application can manage within that scope.



What can I do with Azure RBAC?

Azure RBAC allows you to grant access to Azure resources that you control. Suppose you need to manage access to resources in Azure for the development, engineering, and marketing teams. You've started to receive access requests, and you need to quickly learn how access management works for Azure resources.

Here are some scenarios you can implement with Azure RBAC:

- Allow one user to manage virtual machines in a subscription and another user to manage virtual networks.
- Allow a database administrator group to manage SQL databases in a subscription.
- Allow a user to manage all resources in a resource group, such as virtual machines, websites, and subnets.
- Allow an application to access all resources in a resource group.

How does Azure RBAC work?

You can control access to resources using Azure RBAC by creating role assignments, which control how permissions are enforced. To create a role assignment, you need three elements: a security principal, a role definition, and a scope. You can think of these elements as *who*, *what*, and *where*.

1. Security principal (who)

A *security principal* is just a fancy name for a user, group, or application to which you want to grant access.

2. Role definition (what)

A *role definition* is a collection of permissions. It's sometimes just called a role. A role definition lists the permissions the role can perform such as read, write, and delete. Roles can be high-level, like Owner, or specific, like Virtual Machine Contributor.

Azure includes several built-in roles that you can use. The following lists four fundamental built-in roles:

- **Owner:** Has full access to all resources, including the right to delegate access to others.
- **Contributor:** Can create and manage all types of Azure resources, but can't grant access to others.
- **Reader:** Can view existing Azure resources.
- **User Access Administrator:** Lets you manage user access to Azure resources.

3. Scope (where)

Scope is the level where the access applies. This is helpful if you want to make someone a Website Contributor but only for one resource group.

In Azure, you can specify a scope at multiple levels: management group, subscription, resource group, or resource. Scopes are structured in a parent-child relationship. When you grant access at a parent scope, the child scopes automatically inherit those permissions.

Once you have determined the who, what, and where, you can combine those elements to grant access. A *role assignment* is the process of binding a role to a security principal at a particular scope for the purpose of granting access.

To grant access, you'll create a role assignment. To revoke access, you'll remove a role assignment.

Azure RBAC is an allow model

Azure RBAC is an *allow* model. This means that when you're assigned a role, Azure RBAC allows you to perform certain actions such as read, write, or delete. If one role assignment grants you read permissions to a resource group, and a different role assignment grants you write permissions to the same resource group, then you'll have read and write permissions on that resource group.

Azure RBAC has something called `NotActions` permissions. You can use `NotActions` to create a set of not allowed permissions. The access a role grants—the *effective permissions*—is computed by subtracting the `NotActions` operations from the `Actions` operations.

6 - allow users to reset their password (SSPR)

Why use SSPR?

In Microsoft Entra ID, any user can change their password if they're already signed in. But if they're not signed in, forgot their password, or it's expired, they'll need to reset their password. With SSPR, users can reset their passwords in a web browser or from a Windows sign-in screen to regain access to Azure, Microsoft 365, and any other application that uses Microsoft Entra ID for authentication.

How SSPR works

The user initiates a password reset either by going directly to the password-reset portal, or by selecting the **Can't access your account** link on a sign-in page. The reset portal takes these steps:

1. **Localization:** The portal checks the browser's locale setting and renders the SSPR page in the appropriate language.
2. **Verification:** The user enters their username and passes a CAPTCHA to ensure that it's a user and not a bot.
3. **Authentication:** The user enters the required data to authenticate their identity. They might enter a code or answer security questions.
4. **Password reset:** If the user passes the authentication tests, they can enter a new password and confirm it.

5. **Notification:** A message is sent to the user to confirm the reset.

Authenticate a password reset

It's critical to verify a user's identity before you allow a password reset. Malicious users might exploit any weakness in the system to impersonate that user. Azure supports six different ways to authenticate reset requests.

Enable two or more of these methods so that users can choose the ones they can easily use.

Authentication method	How to register	How to authenticate for a password reset
Mobile app notification	Install the Microsoft Authenticator app on your mobile device, then register it on the multifactor authentication setup page.	Azure sends a notification to the app, which you can either verify or deny.
Mobile app code	This method also uses the Authenticator app, and you install and register it in the same way.	Enter the code from the app.
Email	Provide an email address that's external to Azure and Microsoft 365.	Azure sends a code to the address, which you enter in the reset wizard.
Mobile phone	Provide a mobile phone number.	Azure sends a code to the phone in an SMS message, which you enter in the reset wizard. You can also choose to get an automated call.
Office phone	Provide a nonmobile phone number.	You receive an automated call to this number and press #.
Security questions	Select questions such as "In what city was your mother born?" and save their responses.	Answer the questions.

Require the minimum number of authentication methods

You can specify the minimum number of methods that the user must set up, either one or two. For example, you might enable the mobile app code, email, office phone, and security questions methods and specify a minimum of two methods. Users can then choose the two methods they prefer, like mobile app code and email.

Recommendations

- Enable two or more of the authentication reset request methods.
- Use the mobile app notification or code as the primary method. But also enable the email or office phone methods to support users without mobile devices.
- The mobile phone method isn't a recommended method, because it's possible to send fraudulent SMS messages.
- The security-question option is the least recommended method, because the answers to the security questions might be known to other people. Only use the security-question method in combination with at least one other method.

Accounts associated with administrator roles

- A strong, two-method authentication policy is always applied to accounts with an administrator role, regardless of your configuration for other users.

- The security-question method isn't available to accounts associated with an administrator role.

Configure notifications

Administrators can choose how users are notified of password changes. There are two options you can enable:

- **Notify users on password resets:** The user who resets their own password is notified to their primary and secondary email addresses. If the reset was done by a malicious user, this notification alerts the user, who can take mitigation steps.
- **Notify all admins when other admins reset their password:** All administrators are notified when another administrator resets their password.

SSPR deployment options

You can deploy SSPR with password writeback by using [Microsoft Entra Connect](#) or [cloud sync](#), depending on user needs. You can deploy each option side-by-side in different domains to target different sets of users. This helps existing users on-premises to write back password changes, while adding an option for users in disconnected domains because of a company merger or split. Users from an existing on-premises domain can use Microsoft Entra Connect, while new users from a merger can use cloud sync in another domain.

enable SSPR in Microsoft Entra ID.

Prerequisites

Before you start to configure SSPR, you need a:

- **Microsoft Entra organization:** This organization must have at least a P1 or P2 trial license enabled.
- **Microsoft Entra account with Authentication Policy Administrator role:** You'll use this account to set up SSPR.
- **Non-administrative user account:** You'll use this account to test SSPR. It's important that this account isn't an administrator, because Microsoft Entra imposes extra requirements on administrative accounts for SSPR. This user, and all user accounts, must have a valid license to use SSPR.
- **Security group with which to test the configuration:** The non-administrative user account must be a member of this group. You'll use this security group to limit who you roll SSPR out to.

Scope of SSPR rollout

There are three settings for the **Self-service password reset enabled** property:

- **None:** No users in the Microsoft Entra organization can use SSPR. This value is the default.
- **Selected:** Only the members of the specified security group can use SSPR. You can use this option to enable SSPR for a targeted group of users who can test it and verify that it works as expected. When you're ready to roll it out broadly, set the property to **Enabled** so that all users have access to SSPR.
- **All:** All users in the Microsoft Entra organization can use SSPR.

Configure SSPR

Here are the high-level steps to configure SSPR:

1. Go to the [Azure portal](#), then to **Microsoft Entra ID > Manage > Password reset**.

2. Properties:

- Enable SSPR.
- You can enable it for all users in the Microsoft Entra organization or for selected users.
- To enable for selected users, you must specify the security group. Members of this group can use SSPR.

Authentication methods:

- Choose whether to require one or two authentication methods.
- Choose the authentication methods that the users can use.

Registration:

- Specify whether users are required to register for SSPR when they next sign in.
- Specify how often users are asked to reconfirm their authentication information.

Notifications: Choose whether to notify users and administrators of password resets.

Customization: Provide an email address or web page URL where your users can get help.

Register for SSPR

Now that the SSPR configuration is complete, register an email for the user you created.

1. In a new browser window, go to <https://aka.ms/ssprsetup>.
2. Sign in with the user name [balas@organization-domain-name.onmicrosoft.com](#) and the password that you noted earlier. Remember to replace "organization-domain-name" with the domain name of the Microsoft Entra organization you created.
3. If you're asked to update your password, enter a new password of your choice. Make sure you note the new password.
4. Select the **Security info** tab, and then select **+ Add sign-in method**.
5. In the **Add a method** box, select **Email**.
6. Enter your email details.
7. When you receive the code in your email, enter the code in the text box and select **Next**.

Customize directory branding

Suppose you've been asked to display your retail organization's branding on the Azure sign-in page to reassure users that they're passing credentials to a legitimate system. Here, you'll learn how to configure this custom branding.

To complete this exercise, you must have two image files:

- A page background image. This must be a PNG or JPG file, 1920 x 1080 pixels, and smaller than 300 KB.
- A company logo image. This must be a PNG or JPG file, 32 x 32 pixels, and smaller than 5 KB.

Customize Microsoft Entra organization branding

Let's use Microsoft Entra ID to set up the custom branding.

1. Sign in to the [Azure portal](#).
2. Go to your Microsoft Entra organization by selecting **Microsoft Entra ID**. If you're not in the right Microsoft Entra organization, go to your Azure profile, and select **Switch directory** to find your organization.

- Under **Manage**, select **Company branding**. Then select the **Customize** button in the center of the screen.
- Next to **Favicon**, select **Browse**. Select your logo image.
- Next to **Background image**, select **Browse**. Select your page background image.
- Select a **Page background color** or accept the default.
- Select **Review + Create**, and then select **Create**.

1 - Configure virtual networks

Azure virtual networks are an essential component for creating private networks in Azure. They allow different Azure resources to securely communicate with each other, the internet, and on-premises networks.

Plan virtual networks

You can implement Azure Virtual Network to create a virtual representation of your network in the cloud. With some [planning](#), you can deploy virtual networks and connect the resources you need more effectively. Let's examine some characteristics of virtual networks in Azure.

- An Azure virtual network is a logical isolation of the Azure cloud resources.
- You can use virtual networks to provision and manage virtual private networks (VPNs) in Azure.
- Each virtual network has its own Classless Inter-Domain Routing (CIDR) block and can be linked to other virtual networks and on-premises networks.
- You can link virtual networks with an on-premises IT infrastructure to create hybrid or cross-premises solutions, when the CIDR blocks of the connecting networks don't overlap.
- You control the DNS server settings for virtual networks, and segmentation of the virtual network into subnets.

Things to consider when using virtual networks

Scenario	Description
<i>Create a dedicated private cloud-only virtual network</i>	Sometimes you don't require a cross-premises configuration for your solution. When you create a virtual network, your services and virtual machines within your virtual network can communicate directly and securely with each other in the cloud. You can still configure endpoint connections for the virtual machines and services that require internet communication, as part of your solution.
<i>Securely extend your data center with virtual networks</i>	You can build traditional site-to-site VPNs to securely scale your datacenter capacity. Site-to-site VPNs use IPSEC to provide a secure connection between your corporate VPN gateway and Azure.
<i>Enable hybrid cloud scenarios</i>	Virtual networks give you the flexibility to support a range of hybrid cloud scenarios. You can securely connect cloud-based applications to any type of on-premises system, such as mainframes and Unix systems.

Create subnets

Azure [Subnets](#) provide a way for you to implement logical divisions within your virtual network. Your network can be segmented into subnets to help improve security, increase performance, and make it easier to manage.

Things to know about subnets

There are certain conditions for the IP addresses in a virtual network when you apply segmentation with subnets.

- Each subnet contains a range of IP addresses that fall within the virtual network address space.
 - The address range for a subnet must be unique within the address space for the virtual network.
 - The range for one subnet can't overlap with other subnet IP address ranges in the same virtual network.
 - The IP address space for a subnet must be specified by using CIDR notation.
 - You can segment a virtual network into one or more subnets in the Azure portal.
- Characteristics about the IP addresses for the subnets are listed.

For each subnet, Azure reserves five IP addresses. The first four addresses and the last address are reserved.

Reserved address	Reason
192.168.1.0	This value identifies the virtual network address.
192.168.1.1	Azure configures this address as the default gateway.
192.168.1.2 and 192.168.1.3	Azure maps these Azure DNS IP addresses to the virtual network space.
192.168.1.255	This value supplies the virtual network broadcast address.

Things to consider when using subnets

When you plan for adding subnet segments within your virtual network, there are several factors to consider.

- **Consider service requirements.** Each service directly deployed into a virtual network has specific requirements for routing and the types of traffic that must be allowed into and out of associated subnets. A service might require or create their own subnet. There must be enough unallocated space to meet the service requirements. Suppose you connect a virtual network to an on-premises network by using Azure VPN Gateway. The virtual network must have a dedicated subnet for the gateway.
- **Consider network virtual appliances.** Azure routes network traffic between all subnets in a virtual network, by default. You can override Azure's default routing to prevent Azure routing between subnets. You can also override the default to route traffic between subnets through a network virtual appliance. If you require traffic between resources in the same virtual network to flow through a network virtual appliance, deploy the resources to different subnets.
- **Consider network security groups.** You can associate zero or one network security group to each subnet in a virtual network. You can associate the same or a different network security group to each subnet. Each network security group contains rules that allow or deny traffic to and from sources and destinations.
- **Consider private links.** Azure Private Link provides private connectivity from a virtual network to Azure platform as a service (PaaS), customer-owned, or Microsoft partner services. Private Link simplifies the network architecture and secures the connection between endpoints in Azure. The service eliminates data exposure to the public internet.

Create virtual networks

Things to know about creating virtual networks

Review these requirements for creating a virtual network.

- When you create a virtual network, you need to define the IP address space for the network.
- Plan to use an IP address space that's not already in use in your organization.
- The address space for the virtual network can be either on-premises or in the cloud, but not both.
- To create a virtual network, you need to define at least one subnet.
 - Each subnet contains a range of IP addresses that fall within the virtual network address space.
 - The address range for each subnet must be unique within the address space for the virtual network.
 - The range for one subnet can't overlap with other subnet IP address ranges in the same virtual network.

Plan IP addressing

You can assign IP addresses to Azure resources to communicate with other Azure resources, your on-premises network, and the internet. There are two types of Azure IP addresses: *private* and *public*.

Private IP addresses enable communication within an Azure virtual network and your on-premises network. You create a private IP address for your resource when you use a VPN gateway or Azure ExpressRoute circuit to extend your network to Azure.

Public IP addresses allow your resource to communicate with the internet. You can create a public IP address to connect with Azure public-facing services.

Things to know about IP addresses

Let's take a closer look at the characteristics of IP addresses.

- IP addresses can be statically assigned or dynamically assigned.
- You can separate dynamically and statically assigned IP resources into different subnets.
- Static IP addresses don't change and are best for certain situations, such as:
 - DNS name resolution, where a change in the IP address requires updating host records.
 - IP address-based security models that require apps or services to have a static IP address.
 - TLS/SSL certificates linked to an IP address.
 - Firewall rules that allow or deny traffic by using IP address ranges.
 - Role-based virtual machines such as Domain Controllers and DNS servers.

Create public IP addressing

Things to consider when creating a public IP address

To create a public IP address, configure these settings. Notice that public IP addresses are often used with load balancers.

- **IP Version:** Public IP addresses can be attached to a load balancer or to a network interface. IPv4 and IPv6 addresses are charged at the same rate.
- **SKU:** Select the SKU for the public IP address. A Public IP's SKU must match the SKU of the Load Balancer with which it is used.
- **Tier:** Must match the load balancer tier. A cross-region load balancer distributes traffic across regional backends. Regional distributes traffic within a virtual network.

- **IP address assignment:** Static addresses are assigned when a public IP address is created. Static addresses aren't released until a public IP address resource is deleted.

Associate public IP addresses

A [public IP address](#) resource can be associated with virtual machine network interfaces, internet-facing load balancers, VPN gateways, and application gateways. You can associate your resource with both dynamic and static public IP addresses.

Things to consider when associating public IP addresses

Top-level resource	IP address configuration
Virtual machine	Network interface configuration
Virtual Network Gateway (VPN), Virtual Network Gateway (ER), NAT Gateway	Gateway IP configuration
Public Load Balancer, Application Gateway, Azure Firewall, Route Server, API Management	Front-end configuration
Bastion host	Public IP configuration

Public IP address SKU features

Public IP address	Standard SKU
Allocation method	Static
Security	Secure by default model
Available zones	Supported. Standard IPs can be nonzonal, zonal, or zone-redundant.

Allocate or assign private IP addresses

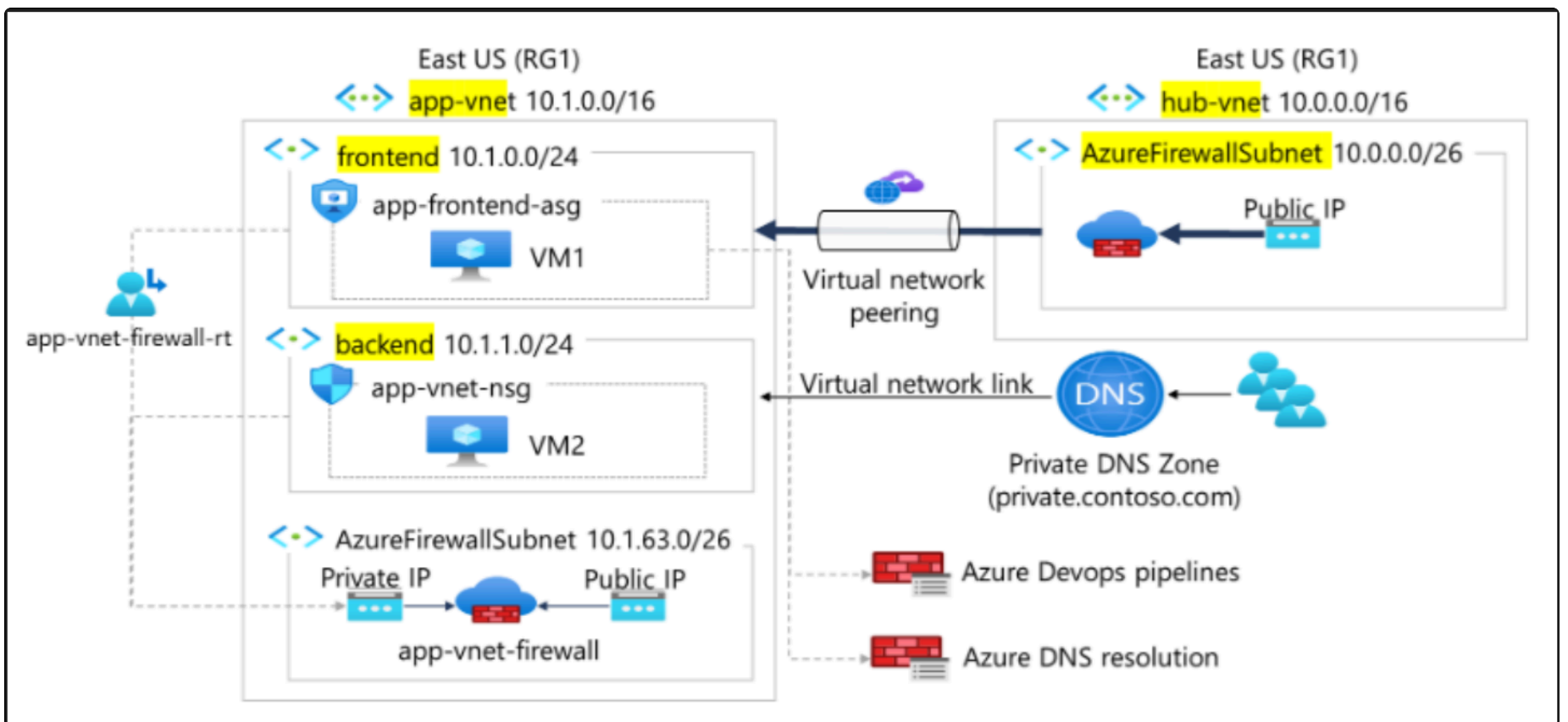
A [private IP address](#) resource can be associated with virtual machine network interfaces, internal load balancers, and application gateways. Azure can provide an IP address (dynamic assignment) or you can assign the IP address (static assignment).

Private IP address assignment

A private IP address is allocated from the address range of the virtual network subnet that a resource is deployed in. There are two options: dynamic and static.

- **Dynamic:** Azure assigns the next available unassigned or unreserved IP address in the subnet's address range. Dynamic assignment is the default allocation method. Suppose addresses 10.0.0.4 through 10.0.0.9 are already assigned to other resources. In this case, Azure assigns the address 10.0.0.10 to a new resource.
- **Static:** You select and assign any unassigned or unreserved IP address in the subnet's address range. Suppose a subnet's address range is 10.0.0.0/16, and addresses 10.0.0.4 through 10.0.0.9 are already assigned to other resources. In this scenario, you can assign any address between 10.0.0.10 and 10.0.255.254.

Create and configure virtual networks



2 - Configure network security groups

Network security groups are a way to limit network traffic to resources in your virtual network. Network security groups contain a list of security rules that allow or deny inbound or outbound network traffic.

Implement network security groups

You can limit network traffic to resources in your virtual network by using a [network security group](#). You can assign a network security group to a subnet or a network interface, and define security rules in the group to control network traffic.

Things to know about network security groups

Let's look at the characteristics of network security groups.

- A network security group contains a list of security rules that allow or deny inbound or outbound network traffic.
- A network security group can be associated to a subnet or a network interface.
- A network security group can be associated multiple times.
- You create a network security group and define security rules in the Azure portal.
- The Overview page for a virtual machine provides information about the associated network security groups. You can see details such as the assigned subnets, assigned network interfaces, and the defined security rules.

Network security groups and subnets

You can assign network security groups to a subnet and create a protected screened subnet (also referred to as a demilitarized zone or *DMZ*). A DMZ acts as a buffer between resources within your virtual network and the internet.

- Use the network security group to restrict traffic flow to all machines that reside within the subnet.
- Each subnet can have a maximum of one associated network security group.

Network security groups and network interfaces

You can assign network security groups to a network interface card (NIC).

- Define network security group rules to control all traffic that flows through a network interface.
- Each network interface that exists in a subnet can have zero, or one, associated network security groups.

Determine network security group rules

Security rules in network security groups enable you to filter network traffic. You can define rules to control the traffic flow in and out of virtual network subnets and network interfaces.

Things to know about security rules

Let's review the characteristics of security rules in network security groups.

- Azure creates several default security rules within each network security group, including inbound traffic and outbound traffic. Examples of default rules include `DenyAllInbound` traffic and `AllowInternetOutbound` traffic.
- Azure creates the default security rules in each network security group that you create.
- You can add more security rules to a network security group by [specifying conditions](#).

Here's a list of the most common conditions.

Setting	Value
Source	Any, IP Addresses, My IP address, Service Tag, or Application security group
Source port ranges	Specify the ports on which the rule allows or denies traffic
Destination	Any, IP Addresses, Service Tag, or Application security group
Protocol	Restrict the rule to Transmission Control Protocol (TCP), User Datagram Protocol (UDP), Internet Control Message Protocol (ICMP), Encapsulating Security Payload (ESP), or Authentication Header (AH). ESP and AH protocols are only available via JSON templates and PowerShell. The default is for the rule to apply to all protocols (Any).
Action	Allow or Deny
Priority	A value between 100 and 4,096 that's unique for all security rules within the NSG

- Each security rule is assigned a Priority value. All security rules for a network security group are processed in priority order. When a rule has a low Priority value, the rule has a higher priority or precedence in terms of order processing.
- You can't remove the default security rules.
- You can override a default security rule by creating another security rule that has a higher Priority setting for your network security group.

Determine network security group effective rules

Each network security group and its defined security rules are evaluated independently. Azure processes the conditions in each rule defined for each virtual machine in your configuration.

- For inbound traffic, Azure first processes network security group security rules for any associated subnets and then any associated network interfaces.
- For outbound traffic, the process is reversed. Azure first evaluates network security group security rules for any associated network interfaces followed by any associated subnets.
- For both the inbound and outbound evaluation process, Azure also checks how to apply the rules for intra-subnet traffic. Intra-subnet traffic refers to virtual machines in the same subnet.

Network security group evaluation

When you apply network security groups to both a subnet and a network interface, each network security group is evaluated independently. Both inbound and outbound rules are considered based on the priority and processing order.

Things to consider when creating effective rules

Review the following considerations regarding creating effective security rules for machines in your virtual network.

- **Consider allowing all traffic.** If you place your virtual machine within a subnet or utilize a network interface, you don't have to associate the subnet or NIC with a network security group. This approach allows all network traffic through the subnet or NIC according to the default Azure security rules. If you're not concerned about controlling traffic to your resource at a specific level, then don't associate your resource at that level to a network security group.
- **Consider importance of allow rules.** When you create a network security group, you must define an **allow** rule for both the subnet and network interface in the group to ensure traffic can get through. If you have a subnet or NIC in your network security group, you must define an allow rule at each level. Otherwise, the traffic is denied for any level that doesn't provide an allow rule definition.
- **Consider intra-subnet traffic.** The security rules for a network security group associated to a subnet can affect traffic between all virtual machines in the subnet. You can prohibit intra-subnet traffic by defining a rule in the network security group to deny all inbound and outbound traffic. This rule prevents all virtual machines in your subnet from communicating with each other.
- **Consider rule priority.** The security rules for a network security group are processed in priority order. To ensure a particular security rule is always processed, assign the lowest possible priority value to the rule. It's a good practice to leave gaps in your priority numbering, such as 100, 200, 300, and so. The gaps in the numbering allow you to add new rules without having to edit existing rules.

When to use augmented security rules

A single network security group rule can contain multiple values in the Source, Destination, and Service fields. This approach, called augmented security rules, reduces the total number of rules needed and simplifies NSG management.

Things to know about augmented security rules

- **Multiple IP addresses:** Combine multiple IP addresses into one rule.
- **Multiple port ranges:** Specify multiple ports and ranges in the Service field.
- **Service tags and ASGs:** Mix service tags, application security groups, and IP addresses within the same rule.
- **Reduced rule count:** Instead of creating separate rules for each IP range or port, combine them into fewer, more manageable rules.

Implement application security groups

You can implement [application security groups \(ASGs\)](#) in your Azure virtual network to logically group your virtual machines by workload. You can then define your network security group rules based on your application security groups.

Things to know about using application security groups

Application security groups provide an application-centric way of looking at your infrastructure. You join your virtual machines to an application security group. Then you use the application security group as a source or destination in the network security group rules.

Things to consider when using application security groups

There are several advantages to implementing application security groups in your virtual networks.

- **Consider IP address maintenance.** When you control network traffic by using application security groups, you don't need to configure inbound and outbound traffic for specific IP addresses. If you have many virtual machines in your configuration, it can be difficult to specify all of the affected IP addresses. As you maintain your configuration, the number of your servers can change. These changes can require you to modify how you support different IP addresses in your security rules.
- **Consider no subnets.** By organizing your virtual machines into application security groups, you don't need to also distribute your servers across specific subnets. You can arrange your servers by application and purpose to achieve logical groupings.
- **Consider simplified rules.** Application security groups help to eliminate the need for multiple rule sets. You don't need to create a separate rule for each virtual machine. You can dynamically apply new rules to designated application security groups. New security rules are automatically applied to all the virtual machines in the specified application security group.
- **Consider workload support.** A configuration that implements application security groups is easy to maintain and understand because the organization is based on workload usage. Application security groups provide logical arrangements for your applications, services, data storage, and workloads.
- **Consider service tags.** Service Tags represent a group of IP address prefixes from a specific Azure service. They help minimize the complexity of frequent updates on network security rules. While service tags are used to simplify the management of IP addresses for Azure services, ASGs are used to group VMs and manage network security policies based on those groups.

3 - Host your domain on Azure DNS

Azure DNS lets you host your Domain Name System (DNS) records for your domains on Azure infrastructure. With Azure DNS, you can use the same credentials, APIs, tools, and billing as your other Azure services.

What is DNS?

DNS, or the Domain Name System, is a protocol within the TCP/IP standard. DNS serves an essential role of translating the human-readable domain names—for example: `www.wideworldimports.com`—into a known IP address. IP addresses enable computers and network devices to identify and route requests among themselves.

How does DNS work?

A DNS server carries out one of two primary functions:

- Maintains a local cache of recently accessed or used domain names and their IP addresses. This cache provides a faster response to a local domain lookup request. If the DNS server can't find the requested domain, it passes the request to another DNS server. This process repeats at each DNS server until either a match is made or the search times out.
- Maintains the key-value pair database of IP addresses and any host or subdomain over which the DNS server has authority. This function is often associated with mail, web, and other

internet domain services.

DNS settings for your domain

Whether a third-party host your DNS server or you manage it in-house, you need to configure it for each host type you're using. Host types include web, email, or other services you're using.

As the administrator for your company, you want to set up a DNS server by using Azure DNS. In this instance, the DNS server acts as a start of authority (SOA) for your domain.

DNS record types

Configuration information for your DNS server is stored as a file within a zone on your DNS server. Each file is called a record. The following record types are the most commonly created and used:

- **A** is the host record, and is the most common type of DNS record. It maps the domain or host name to the IP address.
- **CNAME** is a Canonical Name record that's used to create an alias from one domain name to another domain name. If you had different domain names that all accessed the same website, you'd use CNAME.
- **MX** is the mail exchange record. It maps mail requests to your mail server, whether hosted on-premises or in the cloud.
- **TXT** is the text record. It's used to associate text strings with a domain name. Azure and Microsoft 365 use TXT records to verify domain ownership.

Additionally, there are the following record types:

- Wildcards
- CAA (certificate authority)
- NS (name server)
- SOA (start of authority)
- SPF (sender policy framework)
- SRV (server locations)

The SOA and NS records are created automatically when you create a DNS zone by using Azure DNS.

What is Azure DNS?

Azure DNS allows you to host and manage your domains by using a globally distributed name-server infrastructure. It allows you to manage all of your domains by using your existing Azure credentials.

Why use Azure DNS to host your domain?

Azure DNS is built on the Azure Resource Manager service, which offers the following benefits:

- Improved security
- Ease of use
- Private DNS domains
- Alias record sets

Security features

Azure DNS provides the following security features:

- Role-based access control, which gives you fine-grained control over users' access to Azure resources. You can monitor their usage and control the resources and services to which they have access.
- Activity logs, which let you track changes to a resource and pinpoint where faults occurred.
- Resource locking, which gives you a greater level of control to restrict or remove access to resource groups, subscriptions, or any Azure resources.

Ease of use

Azure DNS can manage DNS records for your Azure services and provide DNS for your external resources. Azure DNS uses the same Azure credentials, support contract, and billing as your other Azure services.

You can manage your domains and records by using the Azure portal, Azure PowerShell cmdlets, or the Azure CLI. Applications that require automated DNS management can integrate with the service by using the REST API and software development kit (SDKs).

Private domains

Azure DNS handles translating external domain names to IP addresses. Azure DNS lets you create private zones. These zones provide name resolution for virtual machines (VMs) within a virtual network and between virtual networks without having to create a custom DNS solution. Private zones allow you to use your own custom domain names rather than the Azure-provided names.

To publish a private DNS zone to your virtual network, you specify the list of virtual networks that are allowed to resolve records within the zone.

Private DNS zones have the following benefits:

- DNS zones are supported as part of the Azure infrastructure, so there's no need to invest in a DNS solution.
- All DNS record types are supported: A, CNAME, TXT, MX, SOA, AAAA, PTR, and SRV.
- Host names for VMs in your virtual network are automatically maintained.
- Split-horizon DNS support allows the same domain name to exist in both private and public zones. It resolves to the correct one based on the originating request location.

Configure Azure DNS to host your domain

Step 1: Create a DNS zone in Azure

You used a third-party domain-name registrar to register the wideworldimports.com domain. The domain doesn't point to your organization's website yet.

To host the domain name with Azure DNS, you first need to create a DNS zone for that domain. A DNS zone holds all the DNS entries for your domain.

When creating a DNS zone, you need to supply the following details:

- **Subscription:** The subscription to be used.
- **Resource group:** The name of the resource group to hold your domains. If one doesn't exist, create one to allow for better control and management.
- **Name:** Your domain name, which in this case is wideworldimports.com.
- **Resource group location:** The location defaults to the location of the resource group.

Step 2: Get your Azure DNS name servers

After you create a DNS zone for the domain, you need to get the name server details from the name servers (NS) record. You use these details to update your domain registrar's information and point to the Azure DNS zone.

Step 3: Update the domain registrar setting

As the domain owner, you need to sign in to the domain-management application provided by your domain registrar. In the management application, edit the NS record and change the NS details to match your Azure DNS name server details.

Changing the NS details is called *domain delegation*. When you delegate the domain, you must use all four name servers provided by Azure DNS.

Step 4: Verify delegation of domain name services

The next step is to verify that the delegated domain now points to the Azure DNS zone you created for the domain. This process can take as few as 10 minutes, but might take longer.

To verify the success of the domain delegation, query the start of authority (SOA) record. The SOA record is automatically created when the Azure DNS zone is set up. You can verify the SOA record using a tool like nslookup.

The SOA record represents your domain and becomes the reference point when other DNS servers are searching for your domain on the internet.

Step 5: Configure your custom DNS settings

The domain name is `wideworldimports.com`. When it's used in a browser, the domain resolves to your website. But what if you want to add in web servers or load balancers? These resources need to have their own custom settings in the DNS zone, either as an A record or a CNAME.

A record

Each A record requires the following details:

- **Name:** The name of the custom domain, for example *webserver1*.
- **Type:** In this instance, it's A.
- **TTL:** Represents the "time-to-live" as a whole unit, where 1 is one second. This value indicates how long the A record lives in a DNS cache before it expires.
- **IP address:** The IP address of the server to which this A record should resolve.

CNAME record

The CNAME is the canonical name, or the alias for an A record. Use CNAME when you have different domain names that all access the same website. For example, you might need a CNAME in the *wideworldimports* zone if you want both www.wideworldimports.com and `wideworldimports.com` to resolve to the same IP address.

You'd create the CNAME record in the *wideworldimports* zone with the following information:

- NAME: `www`
- TTL: 600 seconds
- Record type: CNAME

Configure a private DNS zone

Another type of DNS zone that you can configure and host in Azure is a private DNS zone. Private DNS zones aren't visible on the Internet, and don't require that you use a domain registrar. You

can use private DNS zones to assign DNS names to virtual machines (VMs) in your Azure virtual networks.

Step 1: Create a private DNS zone

In the Azure portal, search for *private DNS zones*. To create the private zone, you need enter a resource group and the name of the zone.

Step 2: Identify virtual networks

Let's assume that your organization already created your VMs and virtual networks in a production environment. Identify the virtual networks associated with VMs that need name-resolution support. To link the virtual networks to the private zone, you need the virtual network names.

Step 3: Link your virtual network to a private DNS zone

To link the private DNS zone to a virtual network, you create a virtual network link. In the Azure portal, go to the private zone, and select **Virtual network links**.

Select **Add** to pick the virtual network you want to link to the private zone.

You add a virtual network link record for each virtual network that needs private name-resolution support.

Create a DNS zone and an A record by using Azure DNS

Create a DNS zone in Azure DNS

Before you can host the wideworldimports.com domain on your servers, you need to create a DNS zone. The DNS zone holds all the configuration records associated with your domain.

To create your DNS zone:

1. Sign in to the [Azure portal](#) with the account you used to activate the sandbox.
2. On the Azure **home** page, under **Azure services**, select **Create a resource**. The **Create a resource** pane appears.
3. In the *Search services and marketplace* search box, search for and select **DNS zone** by Microsoft. The **DNS zone** pane appears.
4. Select **Create > DNS zone**.
5. On the **Basics** tab, enter the following values for each setting.
6. Select **Review + create**.
7. After validation passes, select **Create**. It takes a few minutes to create the DNS zone.
8. When deployment is complete, select **Go to resource**. The **Overview** pane for your **DNS zone** appears.
9. Select **Record Sets** from the top menu bar.

By default, the NS and SOA record sets are automatically created whenever a DNS zone is created, and automatically deleted whenever a DNS zone is deleted. The NS record set defines the Azure DNS namespaces and contains the four Azure DNS records. You use all four records when you update the registrar.

Create a DNS record

The primary record set to create is the A record. The A record set is used to point traffic from a logical domain name to the hosting server's IP address. An A record set can have multiple records. In a record set, the domain name remains constant, while the IP addresses differ.

Dynamically resolve resource name by using alias record

The next phase of the deployment is to improve resiliency by using a load balancer. Load balancers distribute inbound data requests and traffic across one or more servers. They reduce the load on any one server and improve performance. This technology is well established. You can use it throughout your on-premises network.

What is an apex domain?

The apex domain is your domain's highest level. In our case, that's `wideworldimports.com`. The apex domain is also sometimes referred to as the *zone apex* or *root apex*. The `@` symbol often represents the apex domain in your DNS zone records.

What are alias records?

Azure alias records enable a zone apex domain to reference other Azure resources from the DNS zone. You don't need to create complex redirection policies. You can also use an Azure alias to route all traffic through Traffic Manager.

The Azure alias record can point to the following Azure resources:

- A Traffic Manager profile
- Azure Content Delivery Network endpoints
- A public IP resource
- A front-door profile

Alias records provide lifecycle tracking of target resources, ensuring that changes to any target resource are automatically applied to the DNS zone. Alias records also provide support for load-balanced applications in the zone apex.

The alias record set supports the following DNS zone record types:

- **A:** The IPv4 domain name-mapping record.
- **AAAA:** The IPv6 domain name-mapping record.
- **CNAME:** The alias for your domain, which links to the A record.

Uses for alias records

The following are some of the advantages of using alias records:

- **Prevents dangling DNS records:** A dangling DNS record occurs when the DNS zone records aren't up to date with changes to IP addresses. Alias records prevent dangling references by tightly coupling the lifecycle of a DNS record with an Azure resource.
- **Updates DNS record set automatically when IP addresses change:** When the underlying IP address of a resource, service, or application is changed, the alias record ensures that any associated DNS records are automatically refreshed.
- **Hosts load-balanced applications at the zone apex:** Alias records allow for zone apex resource routing to Traffic Manager.
- **Points zone apex to Azure Content Delivery Network endpoints:** With alias records, you can now directly reference your Azure Content Delivery Network instance.

Create alias records for Azure DNS

Set up a virtual network, load balancer, and VMs in Azure

When you manually create a virtual network, load balancer, and two VMs, it takes some time. To reduce this time, you can use a Bash setup script that's available on GitHub. Follow these

instructions to create a test environment for your alias record.

```
# In Azure Cloud Shell, run the following setup script:
git clone https://github.com/MicrosoftDocs/mslearn-host-domain-azure-dns.git

# To run the setup script, run the following commands:
cd mslearn-host-domain-azure-dns
chmod +x setup.sh
./setup.sh
```

The script:

- Creates a network security group.
- Creates two network interface controllers (NICs) and two VMs.
- Creates a virtual network and assigns the VMs.
- Creates a public IP address and updates the configuration of the VMs.
- Creates a load balancer that references the VMs, including rules for the load balancer.
- Links the NICs to the load balancer.

4 - Configure Azure Virtual Network peering

Azure Virtual Network peering lets you connect virtual networks in the same or different regions. Azure Virtual Network peering provides secure communication between resources in the peered networks.

Determine Azure Virtual Network peering uses

Virtual Network peering enables you to seamlessly connect two Azure virtual networks. After the networks are peered, the two virtual networks operate as a single network, for connectivity purposes.

Things to know about Azure Virtual Network peering

There are two types of Azure Virtual Network peering: *regional* and *global*.

- **Regional virtual network peering** connects Azure virtual networks that exist in the same region.
- **Global virtual network peering** connects Azure virtual networks that exist in different regions.
- You can create a regional peering of virtual networks in the same Azure public cloud region, or in the same China cloud region, or in the same Microsoft Azure Government cloud region.
- You can create a global peering of virtual networks in any Azure public cloud region, or in any China cloud region.
- Global peering of virtual networks in different Azure Government cloud regions isn't permitted.
- After you create a peering between virtual networks, the individual virtual networks are still managed as separate resources.
- Virtual networks can be peered across subscriptions and tenants.

Things to consider when using Azure Virtual Network peering

Benefit	Description
Private network connections	When you implement Azure Virtual Network peering, network traffic between peered virtual networks is private. Traffic between the virtual networks is kept on the Microsoft Azure backbone network. No public internet, gateways, or encryption is required in the communication between the virtual networks.
Strong performance	Because Azure Virtual Network peering utilizes the Azure infrastructure, you gain a low-latency, high-bandwidth connection between resources in different virtual networks.
Simplified communication	Azure Virtual Network peering lets resources in one virtual network communicate with resources in a different virtual network, after the virtual networks are peered.
Seamless data transfer	You can create an Azure Virtual Network peering configuration to transfer data across Azure subscriptions, deployment models, and across Azure regions.
No resource disruptions	Azure Virtual Network peering doesn't require downtime for resources in either virtual network when creating the peering, or after the peering is created.

Things to know about peering requirements and limitations

Requirements/Limitations	Description
Nonoverlapping address spaces	Peered virtual networks must have non-overlapping IP address spaces. Peering creation fails if address ranges overlap.
Address space modification restrictions	If you want to change a VNet's address range, you need to delete the peering first, update the address space, and then set up the peering again.
Basic Load Balancer limitations	Resources in one VNet can't communicate with Basic Internal Load Balancer IPs in VNets peered across regions. Use the Standard Load Balancer for cross-region connections.
DNS resolution boundaries	Azure's built-in name resolution does not work across peered VNets. Configure Azure Private DNS zones or custom DNS servers for cross-VNet name resolution.

Determine gateway transit and connectivity

When virtual networks are peered, you can configure Azure VPN Gateway in the peered virtual network as a *transit point*. In this scenario, a peered virtual network uses the remote VPN gateway to gain access to other resources.

The Azure portal provides four key settings when configuring virtual network peering.

- **Traffic to remote virtual network.** Controls whether traffic flows from this VNet to the remote VNet.
- **Traffic forwarded from remote virtual network.** Controls whether forwarded (non-originating) traffic is accepted from the peered VNet.
- **Virtual network gateway or Route Server.** Enables gateway transit. Lets peered VNets use this VNet's VPN Gateway or Azure Route Server.
- **Remote virtual network gateway or Route Server.** Enables this VNet to use the remote VNet's VPN Gateway or Route Server.

Things to know about Azure VPN Gateway

Let's take a closer look at how Azure VPN Gateway is implemented with Azure Virtual Network peering.

- A virtual network can have only one VPN gateway.
- Gateway transit is supported for both regional and global virtual network peering.
- When you allow VPN gateway transit, the virtual network can communicate to resources outside the peering. In our sample illustration, the gateway subnet gateway within the hub virtual network can complete tasks such as:
 - Use a site-to-site VPN to connect to an on-premises network.
 - Use a vnet-to-vnet connection to another virtual network.
 - Use a point-to-site VPN to connect to a client.
- Gateway transit allows peered virtual networks to share the gateway and get access to external resources. With this implementation, you don't need to deploy a VPN gateway in the peered virtual network.
- You can apply network security groups in a virtual network to block or allow access to other virtual networks or subnets. When you configure virtual network peering, you can choose to open or close the network security group rules between the virtual networks.

Create virtual network peering

Azure Virtual Network peering can be configured for virtual networks by using PowerShell, the Azure CLI, and in the Azure portal.

Things to know about creating virtual network peering

There are a few points to review before we look at how to create the peering in the Azure portal.

- To implement virtual network peering, your Azure account must be assigned to the `Network Contributor` role. Alternatively, your Azure account can be assigned to a custom role that can complete the necessary peering actions. For details, see [Permissions](#).
- To create a peering, you need two virtual networks.
- The second virtual network in the peering is referred to as the *remote network*.
- Initially, the virtual machines in your virtual networks can't communicate with each other. After the peering is established, the machines can communicate within the peered network based on your configuration settings.

How to check your peering status

In the Azure portal, you can check the connectivity status of the virtual networks in your virtual network peering. The status conditions depend on how your virtual networks are deployed.

- The two peering status conditions are **Initiated** and **Connected**.
- When you create the initial peering to the second (remote) virtual network from the first virtual network, the peering status for the first virtual network is **Initiated**.

Extend peering with user-defined routes and service chaining

Virtual network peering is nontransitive. The communication capabilities in a peering are available to only the virtual networks and resources in the peering. Other mechanisms have to be used to enable traffic to and from resources and networks outside the private peering network.

Things to know about extending peering

Mechanism	Description
Hub and spoke network	When you deploy a hub-and-spoke network, the hub virtual network can host infrastructure components like a network virtual appliance (NVA) or Azure VPN gateway. All the spoke virtual networks can then peer with the hub virtual network. Traffic can flow through NVAs or VPN gateways in the hub virtual network.
User-defined route (UDR)	Virtual network peering enables the next hop in a user-defined route to be the IP address of a virtual machine in the peered virtual network, or a VPN gateway.
Service chaining	Service chaining is used to direct traffic from one virtual network to a virtual appliance or gateway. To enable service chaining, configure UDRs that point to virtual machines in peered virtual networks as the next hop IP address. UDRs could also point to virtual network gateways to enable service chaining.
Azure Virtual Network Manager	Centrally manages hub-and-spoke or mesh peering topologies at scale. Automates peering creation without manual per-VNet configuration.

exercice

https://microsoftlearning.github.io/AZ-104-MicrosoftAzureAdministrator/Instructions/Labs/LAB_05-Implement_Intersite_Connectivity.html

The main takeaways from this module are:

- Azure Virtual Network peering allows for the connection of virtual networks in a hub and spoke topology.
- There are two types of peering: regional and global. Regional peering connects virtual networks in the same region. Global peering connects virtual networks in different regions.
- Network traffic between peered virtual networks is private and kept on the Azure backbone network.
- You can configure Azure VPN Gateway in the peered virtual network as a transit point to access resources in another network.
- Network security groups can be applied to block or allow access between virtual networks when configuring virtual network peering.

5 - Manage and control traffic flow in your Azure deployment with routes

Identify routing capabilities of an Azure virtual network

Azure routing

Network traffic in Azure is automatically routed across Azure subnets, virtual networks, and on-premises networks. System routes control this routing. They're assigned by default to each subnet in a virtual network. With these system routes, any Azure virtual machine that is deployed into a virtual network can communicate with any other in the network. These virtual machines are also potentially accessible from on-premises through a hybrid network or the internet.

Every subnet has the following default system routes:

Address prefix	Next hop type
Unique to the virtual network	Virtual network

Address prefix	Next hop type
0.0.0.0/0	Internet
10.0.0.0/8	None
172.16.0.0/12	None
192.168.0.0/16	None
100.64.0.0/10	None

The **Next hop type** column shows the network path taken by traffic sent to each address prefix. The path can be one of the following hop types:

- **Virtual network:** A route is created in the address prefix. The prefix represents each address range created at the virtual-network level. If multiple address ranges are specified, multiple routes are created for each address range.
- **Internet:** The default system route 0.0.0.0/0 routes any address range to the internet, unless you override Azure's default route with a custom route.
- **None:** Any traffic routed to this hop type is dropped and doesn't get routed outside the subnet. By default, the following IPv4 private-address prefixes are created: 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16. The prefix 100.64.0.0/10 for a shared address space is also added. None of these address ranges are globally routable.

Within Azure, there are other system routes. Azure creates these routes if the following capabilities are enabled:

- Virtual network peering
- Service chaining
- Virtual network gateway
- Virtual network service endpoint

Virtual network peering and service chaining

Virtual network peering and service chaining let virtual networks within Azure connect to one another. With this connection, virtual machines can communicate with each other within the same region or across regions. This communication in turn creates more routes within the default route table. Service chaining lets you override these routes by creating user-defined routes between peered networks.

Virtual network gateway

Use a virtual network gateway to send encrypted traffic between Azure and on-premises over the internet and to send encrypted traffic between Azure networks. A virtual network gateway contains routing tables and gateway services.

Virtual network service endpoint

Virtual network endpoints extend your private address space in Azure by providing a direct connection to your Azure resources. This connection restricts the flow of traffic: your Azure virtual machines can access your storage account directly from the private address space and deny access from a public virtual machine. As you enable service endpoints, Azure creates routes in the route table to direct this traffic.

Custom routes

System routes might make it easy for you to quickly get your environment up and running. However, there are many scenarios in which you want to more closely control the traffic flow

within your network. For example, you might want to route traffic through an NVA or through a firewall. This control is possible with custom routes.

You have two options for implementing custom routes: create a user-defined route, or use Border Gateway Protocol (BGP) to exchange routes between Azure and on-premises networks.

User-defined routes

You can use a user-defined route to override the default system routes so traffic can be routed through firewalls or NVAs.

When creating user-defined routes, you can specify these next hop types:

- **Virtual appliance:** A virtual appliance is typically a firewall device used to analyze or filter traffic that is entering or leaving your network. You can specify the private IP address of a Network Interface Card (NIC) attached to a virtual machine so that IP forwarding can be enabled. Or you can provide the private IP address of an internal load balancer.
- **Virtual network gateway:** Use to indicate when you want routes for a specific address to be routed to a virtual network gateway. The virtual network gateway is specified as a VPN for the next hop type.
- **Virtual network:** Use to override the default system route within a virtual network.
- **Internet:** Use to route traffic to a specified address prefix that is routed to the internet.
- **None:** Use to drop traffic sent to a specified address prefix.

Service tags for user-defined routes

You can specify a service tag as the address prefix for a user-defined route instead of an explicit IP range. A service tag represents a group of IP address prefixes from a given Azure service. Microsoft manages the address prefixes encompassed by the service tag and automatically updates the service tag as addresses change, thus minimizing the complexity of frequent updates to user-defined routes and reducing the number of routes you need to create.

Border gateway protocol

A network gateway in your on-premises network can exchange routes with a virtual network gateway in Azure by using BGP. BGP is the standard routing protocol that's normally used to exchange routing information among two or more networks. BGP is used to transfer data and information between autonomous systems on the internet, such as different host gateways.

Typically, you use BGP to advertise on-premises routes to Azure when you're connected to an Azure datacenter through Azure ExpressRoute. You can also configure BGP if you connect to an Azure virtual network by using a VPN site-to-site connection.

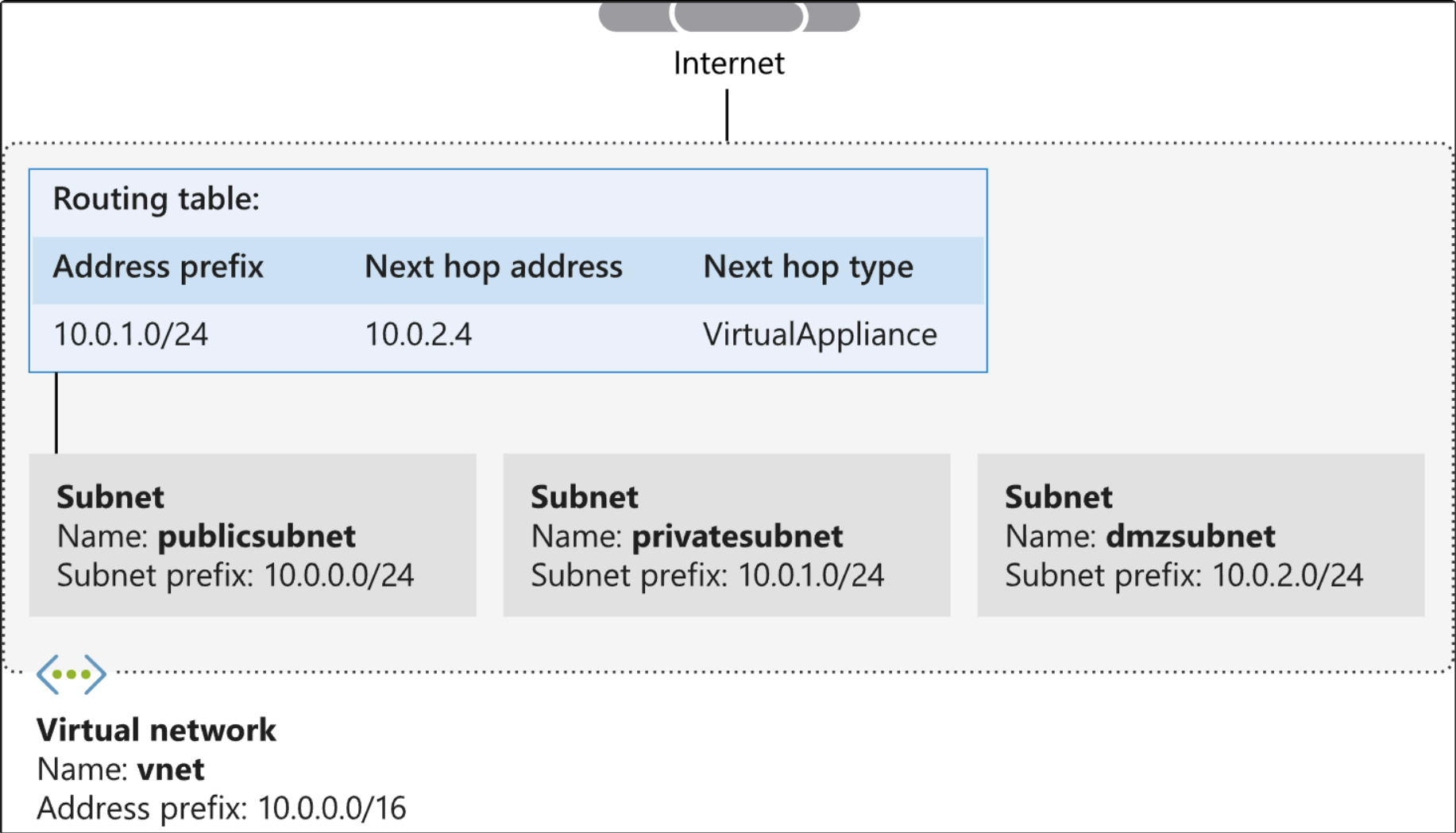
You can't configure multiple user-defined routes with the same address prefix.

If there are multiple routes with the same address prefix, Azure selects the route based on the type in the following order of priority:

1. User-defined routes
2. BGP routes
3. System routes

Create custom routes

In the following exercise, you use a network virtual appliance (NVA) to help secure and monitor traffic. You want to ensure communication between front-end public servers and internal private servers is always routed through the appliance.



Create a route table and custom route

```
# using azure CLI create a route table
az network route-table create --name publictable --resource-group
"myResourceGroupName" --disable-bgp-route-propagation false

# create a custom route:
az network route-table route create --route-table-name publictable --resource-group
"myResourceGroupName" --name productionsubnet --address-prefix 10.0.1.0/24 --next-
hop-type VirtualAppliance --next-hop-ip-address 10.0.2.4

# create the **vnet** virtual network and the **publicsubnet** subnet
az network vnet create --name vnet --resource-group "myResourceGroupName" --address-
prefixes 10.0.0.0/16 --subnet-name publicsubnet --subnet-prefixes 10.0.0.0/24

# create the **privatesubnet** subnet
az network vnet subnet create --name privatesubnet --vnet-name vnet --resource-group
"myResourceGroupName" --address-prefixes 10.0.1.0/24

# create the **dmzsubnet** subnet
az network vnet subnet create --name dmzsubnet --vnet-name vnet --resource-group
"myResourceGroupName" --address-prefixes 10.0.2.0/24

# show all of the subnets in the **vnet** virtual network
az network vnet subnet list --resource-group "myResourceGroupName" --vnet-name vnet
--output table

# Run the following command to associate the route table with the public subnet.
az network vnet subnet update --name publicsubnet --vnet-name vnet --resource-group
"myResourceGroupName" --route-table publictable
```

What is an NVA?

A network virtual appliance (NVA) is a virtual appliance that consists of various layers like:

- A firewall
- A WAN optimizer
- Application-delivery controllers
- Routers
- Load balancers
- IDS/IPS
- Proxies

You can deploy NVAs that you choose from providers in Azure Marketplace. Such providers include Cisco, Check Point, Barracuda, Sophos, WatchGuard, and SonicWall. You can use an NVA to filter traffic inbound to a virtual network, to block malicious requests, and to block requests made from unexpected resources.

Network virtual appliance

Network virtual appliances (NVAs) are virtual machines that control the flow of network traffic by controlling routing. You'll typically use them to manage traffic flowing from a perimeter-network environment to other networks or subnets

Some NVAs require multiple network interfaces. One network interface is dedicated to the management network for the appliance. Additional network interfaces manage and control the traffic processing. After you've deployed the NVA, you can then configure the appliance to route the traffic through the proper interface.

Create an NVA and virtual machines

Deploy the network virtual appliance

```
# deploy the appliance.
az vm create --resource-group "myResourceGroupName" --name nva --vnet-name vnet --
subnet dmzsubnet --image Ubuntu2204 --admin-username azureuser --admin-password
<password>

# get the NVA network interface's ID
NICID=$(az vm nic list --resource-group "myResourceGroupName" --vm-name nva --query
"[].{id:id}" --output tsv)

echo $NICID

# get the NVA network interface's name
NICNAME=$(az vm nic show --resource-group "myResourceGroupName" --vm-name nva --nic
$NICID --query "{name:name}" --output tsv)

echo $NICNAME

# enable IP forwarding for the network interface:
az network nic update --name $NICNAME --resource-group "myResourceGroupName" --ip-
forwarding true

# save the NVA virtual machine's public IP address to the variable `NVAIP`
NVAIP=$(az vm list-ip-addresses --resource-group "myResourceGroupName" --name nva -
```

```
-query "[].virtualMachine.network.publicIpAddresses[*].ipAddress" --output tsv)"

echo $NVAIP

# enable IP forwarding within the NVA
ssh -t -o StrictHostKeyChecking=no azureuser@$NVAIP 'sudo sysctl -w
net.ipv4.ip_forward=1; exit;'
```

Route traffic through the NVA

Create public and private virtual machines

```
# create a file named **cloud-init.txt**.
code cloud-init.txt

# Add the following configuration information to the file then save
#cloud-config
package_upgrade: true
packages:
  - inetutils-traceroute

# Create the **public** VM for the **azureuser** account.
az vm create --resource-group "myResourceGroupName" --name public --vnet-name vnet --
-subnet publicsubnet --image Ubuntu2204 --admin-username azureuser --no-wait --
custom-data cloud-init.txt --admin-password <password>

# create the **private** VM
az vm create --resource-group "myResourceGroupName" --name private --vnet-name vnet
--subnet privatesubnet --image Ubuntu2204 --admin-username azureuser --no-wait --
custom-data cloud-init.txt --admin-password <password>

# Run the following Linux `watch` command to check that the VMs are running.
watch -d -n 5 "az vm list \
  --resource-group "myResourceGroupName" \
  --show-details \
  --query '[*].{Name:name, ProvisioningState:provisioningState,
PowerState:powerState}' \
  --output table"

# save the public IP address of the **public** VM to a variable named `PUBLICIP`
PUBLICIP="$(az vm list-ip-addresses --resource-group "myResourceGroupName" --name
public --query "[].virtualMachine.network.publicIpAddresses[*].ipAddress" --output
tsv)"

echo $PUBLICIP

#save the public IP address of the **private** VM to a variable named `PRIVATEIP`
PRIVATEIP="$(az vm list-ip-addresses --resource-group "myResourceGroupName" --name
private --query "[].virtualMachine.network.publicIpAddresses[*].ipAddress" --output
tsv)"
```

```
echo $PRIVATEIP
```

```
# trace the route from **public** to **private**.
```

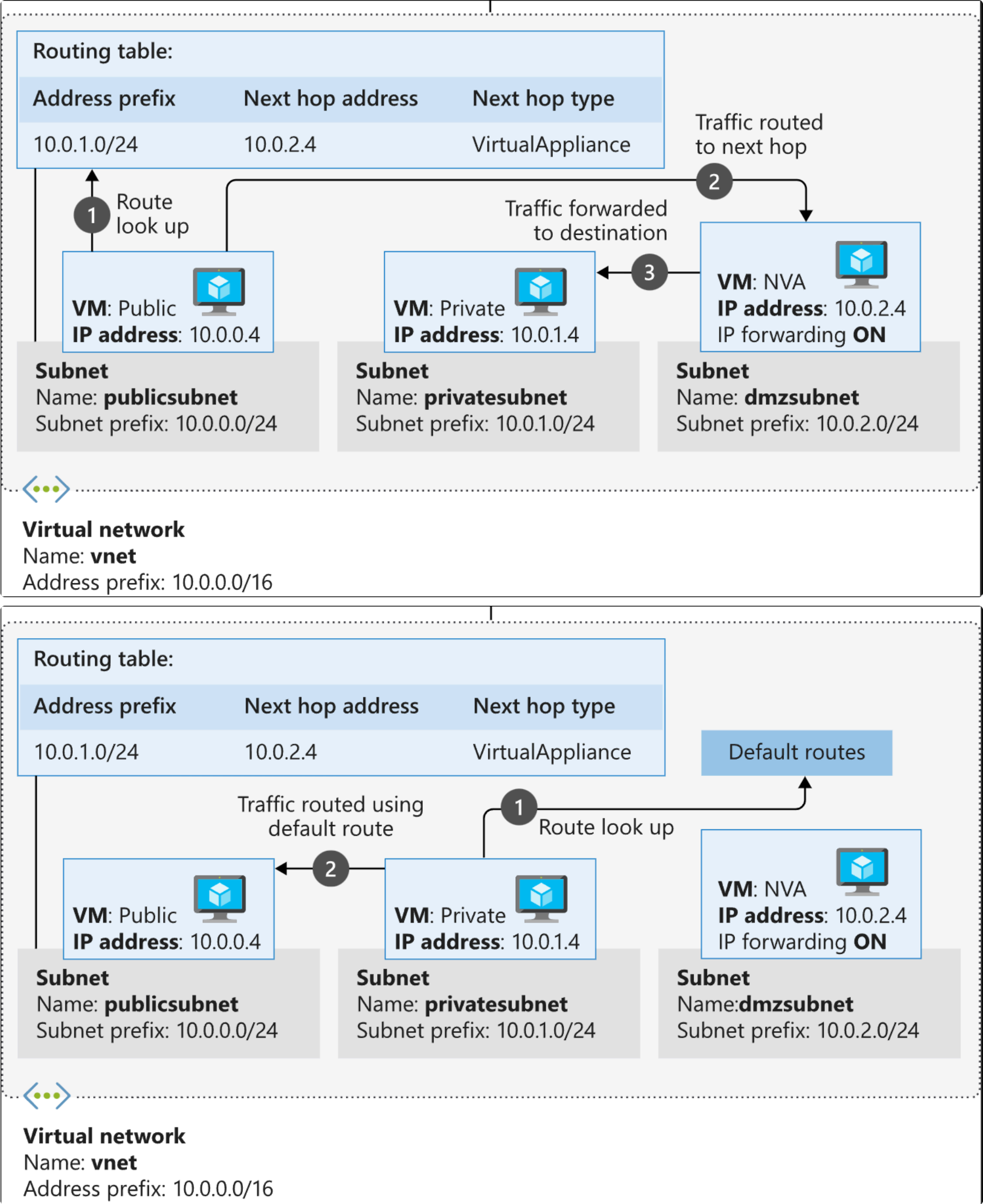
```
ssh -t -o StrictHostKeyChecking=no azureuser@$PUBLICIP 'traceroute private --  
type=icmp; exit'
```

```
# trace the route from **private** to **public**.
```

```
ssh -t -o StrictHostKeyChecking=no azureuser@$PRIVATEIP 'traceroute public --  
type=icmp; exit'
```

```
#
```

You've now configured routing between subnets to direct traffic from the public internet through the **dmzsubnet** subnet before it reaches the private subnet. In the **dmzsubnet** subnet, you added a VM that acts as an NVA. You can configure this NVA to detect potentially malicious requests and block them before they reach their intended targets.



6 - Network appliances in Azure

1 - Introduction to Azure Load Balancer

Azure Load Balancer distributes inbound traffic across a set of VMs in a back-end pool. The back-end pool can be made up of Azure infrastructure as a service (IaaS) VMs or instances in a Virtual Machine Scale Set. You can configure how incoming traffic is distributed across the back-end pool using load-balancing rules. You can ensure that traffic isn't directed to unresponsive nodes using health probes.

What is Azure Load Balancer?

Load balancing is a process in which you distribute incoming traffic equitably across multiple computers. A pool of computers that have lower levels of resources often responds to traffic

more effectively than a single server with higher performance.

Azure Load Balancer is an Azure service that allows you to evenly distribute incoming network traffic across a group of Azure VMs, or across instances in a Virtual Machine Scale Set. Load Balancer delivers high availability and network performance in the following ways:

- Load-balancing rules determine how traffic is distributed to instances that comprise the back end.
- Health probes ensure the resources in the back end are healthy and that traffic isn't directed to unhealthy back-end instances.

You can deploy **public** load balancers and **internal** (or *private*) load balancers in Azure:

- *Public load balancers* are used to load balance internet traffic to your virtual machines (VMs). A public load balancer maps the public IP address and port number of incoming traffic to the private IP address and port number of the back-end pool VMs. For example, you can spread the load of incoming web-request traffic from the internet across multiple web servers. A public load balancer can also provide outbound connections for VMs inside your virtual network.
- An *internal load balancer* directs traffic to resources that are inside a virtual network or that use a VPN to access Azure infrastructure. Internal load balancer front-end IP addresses and virtual networks are never directly exposed to an internet endpoint. Internal line-of-business (LOB) applications run in Azure and are accessed from within Azure or from on-premises resources. An internal load balancer is used where private IPs are needed at the front end only. Internal load balancers are often used to balance traffic from the front-end web tier infrastructure as a service (IaaS) VMs across a set of secondary VMs that perform tasks such as performing calculations or data processing.

An internal load balancer enables the following types of load balancing:

- **Within a virtual network:** Load balancing from VMs in the virtual network to a set of VMs that reside within the same virtual network.
- **For a cross-premises virtual network:** Load balancing from on-premises computers to a set of VMs that reside within the same virtual network.
- **For multi-tier applications:** Load balancing for internet-facing multi-tier applications where the back-end tiers aren't internet-facing. The back-end tiers require traffic load balancing from the internet-facing tier.
- **For LOB applications:** Load balancing for LOB applications that are hosted in Azure without added load balancer hardware or software. This scenario includes on-premises servers that are in the set of computers whose traffic is load balanced.

How Azure Load Balancer works

Azure Load Balancer operates at the transport layer of the Open Systems Interconnection (OSI) model. This Layer 4 functionality allows traffic management based on specific properties of the traffic. Properties including source and destination address, TCP or UDP protocol type, and port number.

Load Balancer has several elements that work together to ensure an application's high availability and performance:

- Front-end IP
- Load balancer rules
- Back-end pool
- Health probes

- Inbound NAT rules
- High availability ports
- Outbound rules

Front-end IP

The front-end IP address is the address clients use to connect to your web application. A front-end IP address can be either a public or a private IP address. Azure load balancers can have multiple front-end IPs. The selection of a public or a private IP address determines which type of load balancer to create:

- **Public IP address: A public load balancer:** A public load balancer maps the public IP and port of incoming traffic to the private IP and port of the virtual machine (VM). You can distribute specific types of traffic across multiple VMs or services by applying load-balancing rules.
- **Private IP address: An internal load balancer:** An internal load balancer distributes traffic to resources that are inside a virtual network. Azure restricts access to the front-end IP addresses of a virtual network that are load balanced. Front-end IP addresses and virtual networks are never directly exposed to an internet endpoint. Internal line-of-business applications run in Azure and are accessed from within Azure or from on-premises resources through a VPN or ExpressRoute connection.

Load Balancer rules

A load balancer rule defines how traffic is distributed to the back-end pool. The rule maps a given front-end IP and port combination to a set of back-end IP addresses and port combination.

Traffic is managed using a five-tuple hash made from the following elements:

- **Source IP:** The IP address of the requesting client.
- **Source port:** The port of the requesting client.
- **Destination IP:** The destination IP address of the request.
- **Destination port:** The destination port of the request.
- **Protocol type:** The specified protocol type, Transmission Control Protocol (TCP), or User Datagram Protocol (UDP).
- **Session affinity:** Ensures that the same pool node always handles traffic for a client.

Back-end pool

The back-end pool is a group of VMs or instances in a Virtual Machine Scale Set that responds to the incoming request. To scale cost-effectively to meet high volumes of incoming traffic, computing guidelines generally recommend adding more instances to the back-end pool.

Health probes

A health probe is used to determine the health status of the instances in the back-end pool. This health probe determines if an instance is healthy and can receive traffic. You can define the unhealthy threshold for your health probes. When a probe fails to respond, the load balancer stops sending new connections to the unhealthy instances. A probe failure doesn't affect existing connections. The connection continues until:

- The application ends the flow.
- Idle timeout occurs.
- The VM shuts down.

High availability ports

A load balancer rule configured with `protocol - all` and `port - 0` is known as a *high availability (HA) port rule*. This rule enables a single rule to load balance all TCP and UDP flows that arrive on all ports of an internal standard load balancer.

The load-balancing decision is made per flow. This action is based on the following five-tuple connection:

- Source IP address
- Source port
- Destination IP address
- Destination port
- Protocol

Inbound NAT rules

You can use load balancing rules in combination with Network Address Translation (NAT) rules. For example, you could use NAT from the load balancer's public address to TCP 3389 on a specific VM. This rule combination allows remote desktop access from outside of Azure.

When to use Azure Load Balancer

Azure Load Balancer is best suited for applications that require ultra-low latency and high performance. Load Balancer is suitable for your organization's needs because you're replacing existing network hardware devices that load balance traffic across applications.

In comparison to these solutions:

- **Azure Load Balancer** is a high-performance, ultra-low-latency Layer 4 load-balancing service (inbound and outbound) for all UDP and TCP protocols. Its built to handle millions of requests per second while ensuring your solution is highly available. Azure Load Balancer is zone-redundant, ensuring high availability across availability zones. If Adatum had applications that required web application firewall functionality, Azure Load Balancer wouldn't be an appropriate solution for the company.

2 - Introduction to Azure Application Gateway

Azure Application Gateway is an Azure service that processes traffic to web apps that are hosted on a pool of web servers. The processing performed by Azure Application Gateway includes load balancing HTTP traffic and inspecting traffic using web application firewall. It also includes encrypting traffic between users and an application gateway, and encrypting traffic between application servers and an application gateway.

What is Azure Application Gateway?

Azure Application Gateway manages the requests that client applications send to web apps that are hosted on a pool of web servers. The pool of web servers can be Azure virtual machines, Azure Virtual Machine Scale Sets, Azure App Service, and even on-premises servers.

Application Gateway provides features such as load balancing HTTP traffic and web application firewall. It provides support for TLS/SSL encryption of traffic between users and an application gateway and between application servers and an application gateway.

Azure Application Gateway includes the following features:

- Support for the HTTP, HTTPS, HTTP/2, and WebSocket protocols.
- A web application firewall to protect against web application vulnerabilities.

- End-to-end request encryption.
- Autoscaling to dynamically adjust capacity as your web traffic load change.
- Connection draining allowing graceful removal of back-end pool members during planned service updates.

How Azure Application Gateway works

Application Gateway includes the following components:

- **Front-end IP address:** Client requests are received through a front-end IP address. You can configure Application Gateway to have a public IP address, a private IP address, or both. Application Gateway can't have more than one public IP address and one private IP address.
- **Listeners:** Application Gateway uses one or more listeners to receive incoming requests. A listener accepts traffic arriving on a specified combination of protocol, port, host, and IP address. Each listener routes requests to a back-end pool of servers following routing rules that you specify. A listener can be Basic or Multi-site. A *Basic* listener only routes a request based on the path in the URL. A *Multi-site* listener can also route requests using the hostname element of the URL. Listeners also handle TLS/SSL certificates for securing your application between the user and Application Gateway.
- **Routing rules:** A routing rule binds a listener to the back-end pools. A rule specifies how to interpret the hostname and path elements in the URL of a request and direct the request to the appropriate back-end pool. A routing rule also has an associated set of HTTP settings. These HTTP settings indicate whether (and how) traffic is encrypted between Application Gateway and the back-end servers. Other configuration information includes Protocol, Session stickiness, Connection draining, Request timeout period, and Health probes.

Load balancing in Application Gateway

Application Gateway uses a round-robin mechanism to automatically load balance the requests sent to the servers in each back-end pool. Load-balancing works with the Open Systems Interconnection (OSI) Layer 7 routing implemented by Application Gateway routing, which means that it load balances requests based on the routing parameters (host names and paths) used by the Application Gateway rules.

Web application firewall

The web application firewall (WAF) is an optional component that handles incoming requests before they reach a listener. The web application firewall checks each request for many common threats based on the Open Web Application Security Project (OWASP). Common threats include: SQL-injection, Cross-site scripting, Command injection, HTTP request smuggling, HTTP response splitting, Remote file inclusion, Bots, crawlers, and scanners, and HTTP protocol violations and anomalies.

OWASP defines a set of generic rules for detecting attacks. These rules are referred to as the Core Rule Set (CRS). The rule sets are under continuous review as attacks evolve in sophistication. WAF supports four rule sets: CRS 3.2, 3.1, 3.0 and 2.2.9. CRS 3.1 is the default.

Back-end pools

A back-end pool is a collection of web servers that can be made up of: a fixed set of virtual machines, a virtual machine scale-set, an app hosted by Azure App Services, or a collection of on-premises servers.

Each back-end pool has an associated load balancer that distributes work across the pool. When configuring the pool, you provide the IP address or name of each web server. All the servers in the

back-end pool should be configured in the same way, including their security settings.

If the servers are using TLS/SSL, you must configure the rule to indicate:

- That your servers expect traffic through the HTTPS protocol.
- Which certificate to use to encrypt traffic and authenticate the connection to a server.

Application Gateway routing also includes these features:

- **Redirection.** Redirection can be used to another site, or from HTTP to HTTPS.
- **Rewrite HTTP headers.** HTTP headers allow the client and server to pass parameter information with the request or the response.
- **Custom error pages.** Application Gateway allows you to create custom error pages instead of displaying default error pages. You can use your own branding and layout using a custom error page.

WebSocket and HTTP/2 traffic

Application Gateway provides native support for the WebSocket and HTTP/2 protocols. The WebSocket and HTTP/2 protocols enable full duplex communication between a server and a client over a long-running Transmission Control Protocol (TCP) connection. This type of communication is more interactive between the web server and the client, and can be bidirectional without the need for polling as required in HTTP-based implementations.

When to use Azure Application Gateway

Azure Application Gateway can meet your organization's needs for the following reasons:

- Azure Application Gateway routing allows traffic to be directed from an endpoint in Azure to a back-end pool made up of servers running in Adatum's on-premises datacenter. The health-probe functionality of Azure Application Gateway ensures that traffic isn't being directed to any server that becomes unavailable.
- Azure Application Gateway TLS termination functionality reduces the amount of CPU capacity that servers in the back-end pool allocate to encryption and decryption operations.
- Azure Application Gateway allows Adatum to use a web application firewall to block cross-site scripting and SQL injection traffic before it reaches servers in the back-end pool.
- Azure Application Gateway supports session affinity. This support is required because the several web applications deployed by Adatum use user session state information stored locally on individual servers in the back-end pool.

When not to use Azure Application Gateway

Azure Application Gateway isn't appropriate if you have a web application that doesn't require load balancing. For example, if you have a web application that only receives a small amount of traffic and the existing infrastructure already competently deals with the existing load, there's no need to deploy a back-end pool of web apps or virtual machines and no need for Application Gateway.

Azure provides other load balancing solutions, including Azure Front Door, Azure Traffic Manager, and Azure Load Balancer. The following list describes the differences between these services:

- **Front Door** is an application delivery network that provides global load balancing and site acceleration service for web applications. It offers Layer 7 capabilities for your application like TLS/SSL offload, path-based routing, fast failover, web application firewall, and caching to improve performance and high-availability of your applications. Choose this option in scenarios such as load balancing a web app deployed across multiple Azure regions.

- **Traffic Manager** is a DNS-based traffic load balancer that enables you to distribute traffic optimally to services across global Azure regions while providing high availability and responsiveness. Because Traffic Manager is a DNS-based load-balancing service, it load-balances only at the domain level. For that reason, it can't fail over as quickly as Front Door because of common challenges around DNS caching and systems not honoring DNS TTLs.
- **Azure Load Balancer** is a high-performance, ultra low-latency Layer 4 load-balancing service (inbound and outbound) for all UDP and TCP protocols. Azure Load Balancer is built to handle millions of requests per second while ensuring that your solution is highly available. Azure Load Balancer is zone-redundant, ensuring high availability across availability zones. Azure Load Balancer works within a region rather than globally.

3 - Introduction to Azure Network Watcher

What is Azure Network Watcher?

Azure Network Watcher provides a suite of tools to monitor, diagnose, view metrics, and enable or disable logs for Azure IaaS (Infrastructure-as-a-Service) resources. Network Watcher enables you to monitor and repair the network health of IaaS products like virtual machines (VMs), virtual networks (VNETs), application gateways, load balancers, etc.

Network Watcher consists of three major sets of tools and capabilities:

- Monitoring
 - Topology
 - The **Topology** tool provides a visualization of the entire network for understanding network configuration. It provides an interactive interface to view resources and their relationships in Azure spanning across multiple subscriptions, resource groups, and locations.
 - Connection monitor
 - **Connection monitor** provides end-to-end connection monitoring for Azure and hybrid endpoints. It helps you understand network performance between various endpoints in your network infrastructure.
- Network diagnostic
 - IP flow verify
 - **IP flow verify** allows you to detect traffic filtering issues at a virtual machine level. It checks if a packet is allowed or denied to or from an IP address (IPv4 or IPv6 address). It also tells you which security rule allowed or denied the traffic.
 - NSG diagnostics
 - **NSG diagnostics** allows you to detect traffic filtering issues at a virtual machine, virtual machine scale set, or application gateway level. It checks if a packet is allowed or denied to or from an IP address, IP prefix, or a service tag.
 - Next hop
 - **Next hop** allows you to detect routing issues. It checks if traffic is routed correctly to the intended destination. It provides you with information about the Next hop type, IP address, and Route table ID for a specific destination IP address.
 - Effective security rules
 - **Effective security rules** allows you to view the effective security rules applied to a network interface. It shows you all security rules applied to the network interface, the subnet the network interface is in, and the aggregate of both.
 - Connection troubleshoot
 - **Connection troubleshoot** enables you to test a connection between a virtual machine, a virtual machine scale set, an application gateway, or a Bastion host and a

virtual machine, an FQDN, a URI, or an IPv4 address.

- Packet capture
 - **Packet capture** allows you to remotely create packet capture sessions to record all network traffic to and from a virtual machine (VM) or a virtual machine scale set.
- VPN troubleshoot
 - **VPN troubleshoot** enables you to troubleshoot virtual network gateways and their connections.
- Traffic
 - Flow logs
 - **Flow logs** allows you to log information about your Azure IP traffic and stores the data in Azure storage. You can log IP traffic flowing through a network security group or Azure virtual network.
 - Traffic analytics
 - **Traffic analytics** provides rich visualizations of flow logs data.

How Azure Network Watcher works

The topology capability of Azure Network Watcher allows you to view all of the following resources in a virtual network. Including, the resources associated to resources in a virtual network and the relationships between the resources.

- Subnets
- Network interfaces
- Network security groups
- Load balancer
- Load balancer health probes
- Public IP addresses
- Virtual network peering
- Virtual network gateways
- VPN gateway connections
- Virtual machines
- Virtual Machine Scale Sets

All resources returned in a topology have the following properties:

- **Name:** The name of the resource.
- **Id:** The URI of the resource.
- **Location:** The Azure region the resource is in.
- **Associations:** A list of associations to the referenced object. Each association has the following properties:
 - **AssociationType:** References the relationship between the child object and the parent. Valid values are `Contains` and `Associated`.
 - **Name:** The name of the referenced resource.
 - **ResourceId:** The URI of the resource referenced in the association.

Connection Monitor tool

Connection Monitor provides unified, end-to-end connection monitoring in Azure Network Watcher. Connection Monitor supports both hybrid and Azure cloud deployments. You can use the Connection Monitor tool to measure the latency between resources. Connection Monitor can detect changes that affect connectivity, such as network configuration changes or modifications

to NSG rules. You can configure Connection Monitor to probe VMs at regular intervals to look for failures or changes. Connection Monitor can diagnose problems and provide explanations about why the issue occurred and the steps that you can take to fix an issue.

When to use Azure Network Watcher

Azure Network Watcher is useful when you're attempting to troubleshoot networking issues related to Azure IaaS products. For example, you can use the tools included in Azure Network Watcher in the following scenarios:

- Resolve connectivity issues related to IaaS VMs.
- Troubleshoot VPN connections.
- Determine cross region network latencies.

When not to use Network Watcher

Network Watcher tools provide intermediate levels of network diagnostic functionality. These tools don't provide some of the advanced features available in some third-party tools. If your organization needs access to this advanced functionality, you might need to deploy a third-party tool that includes this advanced functionality to accomplish your diagnostic goals.

1 - Configure storage accounts

In this module, you learn how to:

- Identify features and usage cases for Azure storage accounts.
- Select between different types of Azure Storage and create storage accounts.
- Select a storage replication strategy.
- Configure secure network access to storage endpoints.

Implement Azure Storage

[Azure Storage](#) is Microsoft's cloud storage solution for modern data storage scenarios. Azure Storage offers a massively scalable object store for data objects. It provides a file system service for the cloud, a messaging store for reliable messaging, and a NoSQL store.

Azure Storage is an AI-ready service that you can use to store files, messages, tables, and other types of information. You use Azure Storage for applications like file shares. Developers use Azure Storage for working data. Working data includes websites, mobile apps, and desktop applications. Azure Storage is also used by IaaS virtual machines, and PaaS cloud services.

Things to know about Azure Storage

You can think of Azure Storage as supporting three categories of data: structured data, unstructured data, and virtual machine data.

Category	Description	Storage examples
Virtual machine data	Virtual machine data storage includes disks and files. Disks are persistent block storage for Azure IaaS virtual machines. Files are fully managed file shares in the cloud.	Storage for virtual machine data is provided through Azure managed disks. Data disks are used by virtual machines to store data like database files, website static content, or custom application code. The number of data disks you can add depends on the virtual machine size.

Category	Description	Storage examples
Unstructured data	Unstructured data is the least organized. The format of unstructured data is referred to as <i>nonrelational</i> .	Unstructured data can be stored by using Azure Blob Storage and Azure Data Lake Storage. Blob Storage is a highly scalable, REST-based cloud object store. Azure Data Lake Storage is the Hadoop Distributed File System (HDFS) as a service.
Structured data	Structured data is stored in a relational format that has a shared schema. Structured data is often contained in a database table with rows, columns, and keys. Tables are an autoscaling NoSQL store.	Structured data can be stored by using Azure Table Storage, Azure Cosmos DB, and Azure SQL Database. Azure Cosmos DB is a globally distributed database service. Azure SQL Database is a fully managed database-as-a-service built on SQL.

Things to consider when using Azure Storage

As you think about your configuration plan for Azure Storage, consider these prominent features.

- **Consider durability and availability.** Azure Storage is durable and highly available. Redundancy ensures your data is safe during transient hardware failures. You replicate data across datacenters or geographical regions for protection from local catastrophe or natural disaster. Replicated data remains highly available during an unexpected outage.
- **Consider secure access.** Azure Storage encrypts all data. Azure Storage provides you with fine-grained control over who has access to your data.
- **Consider scalability.** Azure Storage is designed to be massively scalable to meet the data storage and performance needs of modern applications.
- **Consider manageability.** Microsoft Azure handles hardware maintenance, updates, and critical issues for you.
- **Consider data accessibility.** Data in Azure Storage is accessible from anywhere in the world over HTTP or HTTPS. Microsoft provides SDKs for Azure Storage in various languages. You can use .NET, Java, Node.js, Python, PHP, Ruby, Go, and the REST API. Azure Storage supports scripting in Azure PowerShell or the Azure CLI. The Azure portal and Azure Storage Explorer offer easy visual solutions for working with your data.
- **Consider SFTP support.** Blob Storage can use SFTP (SSH File Transfer Protocol), so you can keep using existing SFTP tools to move files directly to and from blobs. To use SFTP, enable hierarchical namespace (HNS). You can turn it on when you create the storage account (Advanced tab) or later under Settings → Configuration.
- **Consider NFSv3 protocol support.** Blob Storage can also be accessed using NFSv3, which lets Linux clients mount a container like an NFS share. NFSv3 can simplify migrations from Linux file workloads to Azure.
- **Consider default authorization preferences.** In the Azure portal, you can enable **Default to Microsoft Entra authorization**. This authentication makes role-based access control (RBAC) the default instead of shared access keys, which can improve security.

Explore Azure Storage services

Azure Blob Storage

[Azure Blob Storage](#) is Microsoft's object storage solution for the cloud. Blob Storage is optimized for storing massive amounts of unstructured or *nonrelational* data, such as text or binary data. Blob Storage is ideal for:

- Serving images or documents directly to a browser.
- Storing files for distributed access.
- Streaming video and audio.
- Storing data for backup and restore, disaster recovery, and archiving.
- Storing data for analysis by an on-premises or Azure-hosted service.

Azure Files

[Azure Files](#) enables you to set up highly available network file shares. Shares can be accessed by using the Server Message Block (SMB) protocol and the Network File System (NFS) protocol. Multiple virtual machines can share the same files with both read and write access. You can also read the files by using the REST interface or the storage client libraries.

The storage account credentials are used to provide authentication for access to the file share. All users who have the share mounted should have full read/write access to the share.

Azure Queue Storage

[Azure Queue Storage](#) is used to store and retrieve messages. Queue messages can be up to 64 KB in size, and a queue can contain millions of messages. Queues are used to store lists of messages to be processed asynchronously.

Azure Table Storage

[Azure Table storage](#) is a service that stores nonrelational structured data (also known as structured NoSQL data) in the cloud, providing a key/attribute store with a schemaless design. Because Table storage is schemaless, it's easy to adapt your data as the needs of your application evolve. Access to Table storage data is fast and cost-effective for many types of applications, and is typically lower in cost than traditional SQL for similar volumes of data. In addition to the existing Azure Table Storage service, there's a new Azure Cosmos DB Table API offering that provides throughput-optimized tables, global distribution, and automatic secondary indexes.

Things to consider when choosing Azure Storage services

As you think about your configuration plan for Azure Storage, consider the prominent features of the types of Azure Storage and which options support your application needs.

- **Consider storage optimization for massive data.** Azure Blob Storage is optimized for storing massive amounts of unstructured data. Objects in Blob Storage can be accessed from anywhere in the world via HTTP or HTTPS. Blob Storage is ideal for serving data directly to a browser, streaming data, and storing data for backup and restore.
- **Consider storage with high availability.** Azure Files supports highly available network file shares. On-premises apps use file shares for easy migration. By using Azure Files, all users can access shared data and tools. Storage account credentials provide file share authentication to ensure all users who have the file share mounted have the correct read/write access.
- **Consider storage for messages.** Use Azure Queue Storage to store large numbers of messages. Queue Storage is commonly used to create a backlog of work to process asynchronously.
- **Consider storage for structured data.** Azure Table Storage is ideal for storing structured, nonrelational data. It provides throughput-optimized tables, global distribution, and automatic secondary indexes.

Determine storage account types

Things to know about storage account types

Standard storage accounts are backed by magnetic hard disk drives (HDD). A standard storage account provides the lowest cost per GB. You can use Standard storage for applications that require bulk storage or where data is infrequently accessed.

Premium storage accounts are backed by solid-state drives (SSD) and offer consistent low-latency performance. You can use Premium storage for Azure virtual machine disks with I/O-intensive applications like databases.

Storage account	Supported services	Redundancy options	Recommended usage
Standard general-purpose v2	Blob Storage (including Data Lake Storage), Queue Storage, Table Storage, and Azure Files	LRS, GRS, RA-GRS, ZRS, GZRS, RA-GZRS	Standard storage account for most scenarios, including blobs, file shares, queues, tables, and disks (page blobs).
Premium block blobs	Blob Storage (including Data Lake Storage)	LRS, ZRS	Premium storage account for block blobs and append blobs. Recommended for applications with high transaction rates. Use Premium block blobs if you work with smaller objects or require consistently low storage latency. This storage is designed to scale with your applications.
Premium file shares	Azure Files	LRS, ZRS	Premium storage account for file shares only. Recommended for enterprise or high-performance scale applications. Use Premium file shares if you require support for both Server Message Block (SMB) and NFS file shares.
Premium page blobs	Page blobs only	LRS only	Premium high-performance storage account for page blobs only. Page blobs are ideal for storing index-based and sparse data structures, such as operating systems, data disks for virtual machines, and databases.

Determine replication strategies

The data in your Azure storage account is always replicated to ensure durability and high availability. [Azure Storage replication](#) copies your data to protect from planned and unplanned events. These events range from transient hardware failures, network or power outages, massive natural disasters, and so on. You can choose to replicate your data within the same data center, across zonal data centers within the same region, and even across regions. Replication ensures your storage account meets the Service-Level Agreement (SLA) for Azure Storage even if there are failures.

Locally redundant storage

Locally redundant storage is the lowest-cost replication option and offers the least durability compared to other strategies. If a data center-level disaster occurs, such as fire or flooding, all

replicas might be lost or unrecoverable. Despite its limitations, LRS can be appropriate in several scenarios:

- Your application stores data that can be easily reconstructed if data loss occurs.
- Your data is constantly changing like in a live feed, and storing the data isn't essential.
- Your application is restricted to replicating data only within a location due to data governance requirements.

Zone redundant storage

Zone redundant storage synchronously replicates your data across three storage clusters in a single region. Each storage cluster is physically separated from the others and resides in its own availability zone. Each availability zone, and the ZRS cluster within it, is autonomous, and has separate utilities and networking capabilities. Storing your data in a ZRS account ensures you can access and manage your data if a zone becomes unavailable. ZRS provides excellent performance and low latency.

- ZRS isn't currently available in all regions.
- Changing to ZRS from another data replication option requires the physical data movement from a single storage stamp to multiple stamps within a region.

Geo-redundant storage

Geo-redundant storage replicates your data to a secondary region (hundreds of miles away from the primary location of the source data). GRS provides a higher level of durability even during a regional outage. GRS is designed to provide at least 99.99999999999999% **(16 9's) durability**.

If you implement GRS, you have two related options to choose from:

- **GRS** replicates your data to another data center in a secondary region. The data is available to be read only if Microsoft initiates a failover from the primary to secondary region.
- **Read-access geo-redundant storage (RA-GRS)** is based on GRS. RA-GRS replicates your data to another data center in a secondary region, and also provides you with the option to read from the secondary region. With RA-GRS, you can read from the secondary region regardless of whether Microsoft initiates a failover from the primary to the secondary.

For a storage account with GRS or RA-GRS enabled, all data is first replicated with locally redundant storage.

Geo-zone redundant storage

Geo-zone-redundant storage combines the high availability of zone-redundant storage with protection from regional outages as provided by geo-redundant storage. Data in a GZRS storage account is replicated across three Azure availability zones in the primary region, and also replicated to a secondary geographic region for protection from regional disasters.

Microsoft recommends using GZRS for applications that require consistency, durability, high availability, excellent performance, and resilience for disaster recovery. Enable RA-GZRS for read access to a secondary region when there's a regional disaster.

Things to consider when choosing replication strategies

Node in data center unavailable	Entire data center unavailable	Region-wide outage	Read access during region-wide outage
- LRS - ZRS - GRS - RA-GRS - GZRS - RA-GZRS	- ZRS - GRS - RA-GRS - GZRS - RA-GZRS	- GRS - RA-GRS - GZRS - RA-GZRS	- RA-GRS - RA-GZRS

Access storage

Every object you store in Azure Storage has a unique URL address. Your storage account name forms the *subdomain* portion of the URL address. The combination of the subdomain and the domain name, which is specific to each service, forms an endpoint for your storage account.

Service	Default endpoint
Container service	// mystorageaccount .blob.core.windows.net
Table service	// mystorageaccount .table.core.windows.net
Queue service	// mystorageaccount .queue.core.windows.net
File service	// mystorageaccount .file.core.windows.net

We create the URL to access an object in your storage account by appending the object's location in the storage account to the endpoint.

For example, to access the *myblob* data in the *mycontainer* location in your storage account, we use the following URL address:

```
// mystorageaccount .blob.core.windows.net/ mycontainer / myblob .
```

Configure custom domains

You can configure a [custom domain](#) to access blob data in your Azure storage account. As we reviewed, the default endpoint for Azure Blob Storage is `<storage-account-name>.blob.core.windows.net` . If you map a custom domain and subdomain, such as `www.contoso.com` , to the blob or web endpoint for your storage account, your users can use that domain to access blob data in your storage account.

Direct mapping lets you enable a custom domain for a subdomain to an Azure storage account. For this approach, you create a `CNAME` record that points from the subdomain to the Azure storage account.

The following example shows how a subdomain is mapped to an Azure storage account to create a `CNAME` record in the domain name system (DNS):

- Subdomain: `blobs.contoso.com`
- Azure storage account: `<storage account>.blob.core.windows.net`
- Direct `CNAME` record: `contosoblobs.blob.core.windows.net`

Secure storage endpoints

To access these settings for your storage account, you use the **Firewalls and virtual networks** settings. You add the virtual networks that should have access to the service for the account. - This setting restricts access to your storage account from specific subnets on virtual networks or public IPs.

Things to know about configuring service endpoints

- You can configure the service to allow access to one or more public IP ranges.
- Subnets and virtual networks must exist in the same Azure region or region pair as your storage account.

Things to know about configuring private endpoints

A private endpoint uses a private IP address from your virtual network to bring the Azure Storage service into your VNet. All traffic between your VNet and the storage service goes over the Microsoft backbone network, eliminating exposure to the public internet.

Key differences from service endpoints

- Private endpoints assign a private IP from your VNet to the storage account, keeping all traffic within the Microsoft backbone. Use private endpoints for production workloads requiring complete network isolation and compliance requirements
- Service endpoints keep the storage account on its public endpoint but restrict access to specific VNets and subnets. Use service endpoints for development scenarios or when you need simpler configuration with some public internet access
-

2 - Configure Azure Blob Storage

Azure Blob Storage is an AI-ready service for storing large amounts of unstructured object data. Unstructured data is data that doesn't adhere to a particular data model or definition, such as text or binary data.

Implement Azure Blob Storage

[Azure Blob Storage](#) is a service that stores unstructured data in the cloud as objects or blobs. Blob stands for Binary Large Object. Blob Storage is also referred to as *object storage* or *container storage*.

Things to know about Azure Blob Storage

- Blob Storage can store any type of text or binary data. Some examples are text documents, images, video files, and application installers.
- Blob Storage uses three resources to store and manage your data:
 - An Azure storage account
 - Containers in an Azure storage account
 - Blobs in a container
- To implement Blob Storage, you configure several settings:
 - Blob container options.
 - Blob types and upload options.
 - Blob Storage access tiers.
 - Blob lifecycle rules.
 - Blob object replication options

Things to consider when implementing Azure Blob Storage

Consider the following scenarios and think about your own data needs:

- **Consider browser uploads.** Use Blob Storage to serve images or documents directly to a browser.

- **Consider distributed access.** Blob Storage can store files for distributed access, such as during an installation process.
- **Consider streaming data.** Stream video and audio by using Blob Storage.
- **Consider archiving and recovery.** Blob Storage is a great solution for storing data for backup and restore, disaster recovery, and archiving.
- **Consider application access.** You can store data in Blob Storage for analysis by an on-premises or Azure-hosted service.

Create blob containers

Azure Blob Storage uses a container resource to group a set of blobs. A blob can't exist by itself in Blob Storage. *A blob must be stored in a container resource.*

Things to know about containers and blobs

Let's look at the configuration characteristics of containers and blobs.

- All blobs must be in a container.
- Containers organize your blob storage.
- A container can store an unlimited number of blobs.
- An Azure storage account can contain an unlimited number of containers.
- You must create a storage container before you can begin to upload data.

Configure a container

In the Azure portal, you configure settings to create a container for an Azure storage account.

- **Name:** Enter a name for your container. The name must be unique within the Azure storage account.
 - The name can contain only lowercase letters, numbers, and hyphens.
 - The name must begin with a letter or a number.
 - The minimum length for the name is three characters.
 - The maximum length for the name is 63 characters.
- **Public access level:** The access level specifies whether the container and its blobs can be accessed publicly. By default, container data is private and visible only to the account owner. There are three access level choices:
 - **Private:** (Default) Prohibit anonymous access to the container and blobs.
 - **Blob:** Allow anonymous public read access for the blobs only.
 - **Container:** Allow anonymous public read and list access to the entire container, including the blobs.

Assign blob access tiers

Azure Storage supports several [access tiers](#) for blob data. These tiers include Hot, Cool, Cold, and Archive. Each access tier is optimized to support a particular pattern of data usage.

Things to know about blob access tiers

Hot tier

The Hot tier is optimized for frequent reads and writes of objects in the Azure storage account. A good usage case is data that is actively being processed. *The hot tier has the highest storage costs, but the lowest access costs.*

Cool tier

The Cool tier is optimized for storing large amounts of infrequently accessed data. This tier is intended for data that remains in the Cool tier for at least 30 days. A usage case for the Cool tier is short-term backup and disaster recovery datasets and older media content.

Storing data in the Cool tier is more cost-effective. The cool tier has lower storage costs and higher access costs compared to the hot tier.

Cold tier

The Cold tier is also optimized for storing large amounts of infrequently accessed data. This tier is intended for data that can remain in the tier for at least 90 days. The cold tier has lower storage costs and higher access costs compared to the cool tier.

Archive tier

The Archive tier is an offline tier that's optimized for data that can tolerate several hours of retrieval latency. Data must remain in the Archive tier for at least 180 days or be subject to an early deletion charge. Data for the Archive tier includes secondary backups, original raw data, and legally required compliance information. *This tier is the most cost-effective option for storing data.*

Accessing data is more expensive in the Archive tier than accessing data in the other tiers.

Compare access tiers

Comparison	Hot access tier	Cool access tier	Cold access tier	Archive access tier
Availability	99.9%	99%	99%	99%
Availability (RA-GRS reads)	99.99%	99.9%	99.9%	99.9%
Latency (time to first byte)	milliseconds	milliseconds	milliseconds	hours
Minimum storage duration	N/A	30 days	90 days	180 days

Add blob lifecycle management rules

Azure Blob Storage supports [lifecycle management](#) for data sets. It offers a rich rule-based policy for GPv2 accounts and Premium block blob accounts. Legacy Blob Storage accounts are also supported, but GPv2 is recommended for new deployments. You can use lifecycle policy rules to transition your data to the appropriate access tiers, and set expiration times for the end of a data set's lifecycle.

Things to know about lifecycle management

You can use Azure Blob Storage lifecycle management policy rules to accomplish several tasks.

- Transition blobs to a cooler storage tier (Hot to Cool, Hot to Cold, Hot to Archive, Cool to Cold, Cool to Archive, Cold to Archive) to optimize for performance and cost.
- Delete current versions of a blob, previous versions of a blob, or blob snapshots at the end of their lifecycles.
- Automatically transition blobs from Cool back to Hot when accessed. This setting optimizes for unpredictable access patterns without early deletion charges.
- Apply rules to an entire storage account, to select containers, or to a subset of blobs using name prefixes or blob index tags as filters.

Configure lifecycle management policy rules

In the Azure portal, you create lifecycle management policy rules for your Azure storage account by specifying several settings. For each rule, you create **If - Then** block conditions to transition or expire data based on your specifications.

- **If:** The **If** clause sets the evaluation clause for the policy rule. When the **If** clause evaluates to true, the **Then** clause is executed. Use the **If** clause to set the time period to apply to the blob data. The lifecycle management feature checks if the data is accessed or modified according to the specified time.
 - **More than (days ago):** The number of days to use in the evaluation condition.
- **Then:** The **Then** clause sets the action clause for the policy rule. When the **If** clause evaluates to true, the **Then** clause is executed. Use the **Then** clause to set the transition action for the blob data. The lifecycle management feature transitions the data based on the setting.
 - **Move to cool storage:** The blob data is transitioned to Cool tier storage.
 - **Move to cold storage:** The blob data is transitioned to Cold tier storage.
 - **Move to archive storage:** The blob data is transitioned to Archive tier storage.
 - **Delete the blob:** The blob data is deleted.

Determine blob object replication

[Object replication](#) copies blobs in a container asynchronously according to policy rules that you configure.

Replication includes the blob content, metadata properties, and versions.

Things to know about blob object replication

- Object replication requires that [Blob versioning](#) is enabled on both the source and destination accounts. When blob versioning is enabled, you can access earlier versions of a blob. This access lets you recover your modified or deleted data.
- Object replication doesn't support blob snapshots. Any snapshots on a blob in the source account aren't replicated to the destination account.
- Object replication is supported when the source and destination accounts are in the Hot, Cool, or Cold tier. The source and destination accounts can be in different tiers.
- When you configure object replication, you create a replication policy that specifies the source Azure storage account and the destination storage account.
- A replication policy includes one or more rules that specify a source container and a destination container. The policy identifies the blobs in the source container to replicate.

Things to consider when configuring blob object replication

Consider the following scenarios and think about how replication can be a part of your Blob Storage strategy.

- **Consider latency reductions.** Minimize latency with blob object replication. You can reduce latency for read requests by enabling clients to consume data from a region that's in closer physical proximity.
- **Consider efficiency for compute workloads.** Improve efficiency for compute workloads by using blob object replication. With object replication, compute workloads can process the same sets of blobs in different regions.
- **Consider data distribution.** Optimize your configuration for data distribution. You can process or analyze data in a single location and then replicate only the results to other regions.
- **Consider costs benefits.** Manage your configuration and optimize your storage policies. After your data is replicated, you can reduce costs by moving the data to the Archive tier by

using lifecycle management policies.

Manage blobs

A blob can be any type of data and any size file. Azure Storage offers three types of blobs: *block blob*, *page blob*, and *append blob*.

Things to know about blob types

Block blobs. A block blob consists of blocks of data that are assembled to make a blob. Most Blob Storage scenarios use block blobs. Block blobs are ideal for storing text and binary data in the cloud, like files, images, and videos. The block blob type is the default type for a new blob.

Append blobs. An append blob is similar to a block blob because the append blob also consists of blocks of data. The blocks of data in an append blob are optimized for *append* operations. Append blobs are useful for logging scenarios, where the amount of data can increase as the logging operation continues

Page blobs. A page blob can be up to 8 TB in size. Page blobs are more efficient for frequent read/write operations. Azure Virtual Machines uses page blobs for operating system disks and data disks

After you create a blob, you can't change its type.

Things to consider when managing blob storage

You can use the portal to upload and manage blobs. This option is good for a few files. After you identify the files to upload, you choose the blob type and block size, and the container folder. You also set the access tier and the encryption scope.

For larger numbers of files, it's best to use a tool.

- [Azure Storage Explorer](#). Upload, download, and manage blobs, files, queues, and tables, as well as Azure Data Lake Storage entities and managed disks. You can also view, edit, and manage resources, preview data, and configure storage permissions and access controls.
- [AzCopy](#). An easy-to-use command-line tool for Windows and Linux. You can copy data to and from Blob Storage, across containers, and across storage accounts.
- [Azure Data Box Disk](#). A service for transferring on-premises data to Blob Storage when large datasets or network constraints make uploading data over the wire unrealistic. You can use Azure Data Box Disk to request solid-state disks (SSDs) from Microsoft. You can copy your data to those disks and ship them back to Microsoft to be uploaded into Blob Storage.

Determine Blob Storage pricing

The primary tool for estimating these costs is the Azure pricing calculator. The pricing tool can calculate migration, monthly estimates, and future pricing estimates based on the workload-driven input that you specify. In general, the cost of block blob storage depends on:

- Volume of data stored per month.
- Quantity and types of operations performed, along with any data transfer costs.
- Data redundancy option selected.

Things to know about pricing for Blob Storage

- **Performance tiers.** The Blob Storage tier determines the amount of data stored and the cost for storing that data. As the performance tier gets cooler, the per-gigabyte cost decreases.

- **Data access costs.** Data access charges increase as the tier gets cooler. For data in the Cool, Cold, and Archive tiers, you're billed a per-gigabyte data access charge for read actions.
- **Transaction costs.** There's a per-transaction charge for all tiers. The charge increases as the tier gets cooler.
- **Geo-replication data transfer costs.** This charge only applies to accounts that have geo-replication configured. Geo-replication data transfer incurs a per-gigabyte charge.
- **Outbound data transfer costs.** Outbound data transfers incur billing for bandwidth usage on a per-gigabyte basis. This billing is consistent with general-purpose Azure storage accounts.
- **Changes to the storage tier.** If you change the account storage tier from Cool to Hot, you incur a charge equal to reading all the data existing in the storage account. Changing the account storage tier from Hot to Cool incurs a charge equal to writing all the data into the Cool tier (GPv2 accounts only).

Provide storage for the public website

3 - Configure Azure Storage security

Administrators use different strategies to ensure their data is secure. Common approaches include encryption, authentication, authorization, and user access control with credentials, file permissions, and private signatures. Azure Storage offers a suite of security capabilities based on common strategies to help you secure your data.

Things to know about Azure Storage security strategies

- **Encryption at rest.** Storage Service Encryption (SSE) with a 256-bit Advanced Encryption Standard (AES) cipher encrypts all data written to Azure Storage. When you read data from Azure Storage, Azure Storage decrypts the data before returning it. This process incurs no extra charges and doesn't degrade performance. Encryption at rest includes encrypting virtual hard disks (VHDs) with Azure Disk Encryption. This encryption uses BitLocker for Windows images, and uses dm-crypt for Linux.
- **Encryption in transit.** You can configure your storage account to only accept requests from secure connections by setting the **Secure transfer required** property for the storage account. Existing accounts should explicitly disallow TLS 1.0 and 1.1, which are deprecated.
- **Encryption models.** Azure supports various encryption models, including server-side encryption that uses service-managed keys, customer-managed keys in Key Vault, or customer-managed keys on customer-controlled hardware. With client-side encryption, you can manage and store keys on-premises or in another secure location.
- **Authorize requests.** For optimal security, Microsoft recommends using Microsoft Entra ID with managed identities to authorize requests against blob, queue, and table data, whenever possible. Authorization with Microsoft Entra ID and managed identities provides superior security and ease of use over Shared Key authorization.
- **RBAC.** RBAC ensures that resources in your storage account are accessible only when you want them to be, and to only those users or applications whom you grant access. Assign RBAC roles scoped to an Azure storage account.
- **Storage analytics.** Azure Storage Analytics performs logging for a storage account. You can use this data to trace requests, analyze usage trends, and diagnose issues with your storage account.

Things to consider when using authorization security

Think about what security strategies would work for your Azure Storage.

Authorization strategy	Description
Microsoft Entra ID	Microsoft Entra ID is Microsoft's cloud-based identity and access management service. With Microsoft Entra ID, you can assign fine-grained access to users, groups, or applications by using role-based access control.
Shared Key	Access is authorized with an account access key. The key can be the primary or secondary access key. To enforce Entra ID authorization, disable the Shared Key at the storage account level.
Shared access signatures	A SAS delegates access to a particular resource in your Azure storage account with specified permissions and for a specified time interval.
Anonymous access to containers and blobs	Anonymous public access is disabled by default on new storage accounts. Microsoft recommends keeping anonymous access disabled for accounts containing sensitive data.

Create shared access signatures

A [shared access signature \(SAS\)](#) is a uniform resource identifier (URI) that grants restricted access rights to Azure Storage resources. SAS is a secure way to share your storage resources without compromising your account keys.

You can provide a SAS to clients who shouldn't have access to your storage account key. By distributing a SAS URI to these clients, you grant them access to a resource for a specified period of time. You'd typically use a SAS for a service where users read and write their data to your storage account.

- A *user delegation SAS* is secured with Microsoft Entra credentials and also by the permissions specified for the SAS. A user delegation SAS is supported for Blob Storage and Data Lake Storage,
- An *account-level SAS* to allow access to anything that a service-level SAS can allow, plus other resources and abilities. For example, you can use an account-level SAS to allow the ability to create file systems.
- A *service-level SAS* to allow access to specific resources in a storage account. You'd use this type of SAS, for example, to allow an app to retrieve a list of files in a file system, or to download a file.
- A *stored access policy* can provide another level of control when you use a service-level SAS on the server side. You can group SASs and provide other restrictions by using a stored access policy.

Create shared access signatures

A [shared access signature \(SAS\)](#) is a uniform resource identifier (URI) that grants restricted access rights to Azure Storage resources. SAS is a secure way to share your storage resources without compromising your account keys.

You can provide a SAS to clients who shouldn't have access to your storage account key. By distributing a SAS URI to these clients, you grant them access to a resource for a specified period of time. You'd typically use a SAS for a service where users read and write their data to your storage account.

- A *user delegation SAS* is secured with Microsoft Entra credentials and also by the permissions specified for the SAS. A user delegation SAS is supported for Blob Storage and Data Lake Storage,

- An *account-level SAS* to allow access to anything that a service-level SAS can allow, plus other resources and abilities. For example, you can use an account-level SAS to allow the ability to create file systems.
- A *service-level SAS* to allow access to specific resources in a storage account. You'd use this type of SAS, for example, to allow an app to retrieve a list of files in a file system, or to download a file.
- A *stored access policy* can provide another level of control when you use a service-level SAS on the server side. You can group SASs and provide other restrictions by using a stored access policy.

Recommendations for managing risks

Recommendation	Description
Always use HTTPS for creation and distribution	If a SAS is passed over HTTP and intercepted, an attacker can intercept and use the SAS. These <i>man-in-the-middle</i> attacks can compromise sensitive data or allow for data corruption by the malicious user.
Reference stored access policies where possible	Stored access policies give you the option to revoke permissions without having to regenerate the Azure storage account keys. Set the storage account key expiration date far in the future.
Set near-term expiry times for an unplanned SAS	If a SAS is compromised, you can mitigate attacks by limiting the SAS validity to a short time. This practice is important if you can't reference a stored access policy. Near-term expiration times also limit the amount of data that can be written to a blob by limiting the time available to upload to it.
Require clients automatically renew the SAS	Require your clients to renew the SAS well before the expiration date. By renewing early, you allow time for retries if the service providing the SAS is unavailable.
Plan carefully for the SAS start time	If you set the start time for a SAS to now, then due to clock skew (differences in current time according to different machines), failures might be observed intermittently for the first few minutes. In general, set the start time to at least 15 minutes in the past. Or, don't set a specific start time, which causes the SAS to be valid immediately in all cases. The same conditions generally apply to the expiry time. You might observe up to 15 minutes of clock skew in either direction on any request. For clients that use a REST API version earlier than 2012-02-12, the maximum duration for a SAS that doesn't reference a stored access policy is 1 hour. Any policies that specify a longer term fail.
Define minimum access permissions for resources	A security best practice is to provide a user with the minimum required privileges. If a user only needs read access to a single entity, then grant them read access to that single entity, and not read/write/delete access to all entities. This practice also helps lessen the damage if a SAS is compromised because the SAS has less power in the hands of an attacker.
Validate data written by using a SAS	When a client application writes data to your Azure storage account, keep in mind there can be problems with the data. If your application requires validated or authorized data, validate the data after written, but before used. This practice also protects against corrupt or malicious data being written to your account, either by a user who properly acquired the SAS, or by a user exploiting a leaked SAS.

Recommendation	Description
Don't assume a SAS is always the correct choice	In some scenarios, the risks associated with a particular operation against your Azure storage account outweigh the benefits of using a SAS. For such operations, create a middle-tier service that writes to your storage account after performing business rule validation, authentication, and auditing. Also, sometimes it's easier to manage access in other ways. If you want to make all blobs in a container publicly readable, you can make the container Public, rather than providing a SAS to every client for access.

Identify URI and SAS parameters

When you create your shared access signature (SAS), a uniform resource identifier (URI) is created by using parameters and tokens. The URI consists of your Azure Storage resource URI and the SAS token.

Things to know about URI definitions

```
# URI example
https://myaccount.blob.core.windows.net/?restype=service&comp=properties&sv=2015-04-05&ss=bf&st=2015-04-29T22%3A18%3A26Z&se=2015-04-30T02%3A23%3A26Z&sr=b&sp=rw&sip=168.1.5.60-168.1.5.70&spr=https&sig=F%6GRVAZ5Cdj2Pw4tgU7IłSTkWgn7bUkkAg8P6HESXwmf%4B
```

Parameter	Example	Description
Resource URI	https://myaccount. blob .core.windows.net/ ? restype= service &comp=properties	Defines the Azure Storage endpoint and other parameters. This example defines an endpoint for Blob Storage and indicates that the SAS applies to service-level operations. When the URI is used with <code>GET</code> , the Storage properties are retrieved. When the URI is used with <code>SET</code> , the Storage properties are configured.

1.1 - Introduction to Azure Cloud Shell		
Parameter	Example	Description
Storage version	<code>sv =2015-04-05</code>	For Azure Storage version 2012-02-12 and later, this parameter indicates the version to use. This example indicates that version 2015-04-05 (April 5, 2015) should be used.
Storage service	<code>ss =bf</code>	Specifies the Azure Storage to which the SAS applies. This example indicates that the SAS applies to Blob Storage and Azure Files.
Start time	<code>st =2015-04-29T22%3A18%3A26Z</code>	(Optional) Specifies the start time for the SAS in UTC time. This example sets the start time as April 29, 2015 22:18:26 UTC. If you want the SAS to be valid immediately, omit the start time.
Expiry time	<code>se =2015-04-30T02%3A23%3A26Z</code>	Specifies the expiration time for the SAS in UTC time. This example sets the expiry time as April 30, 2015 02:23:26 UTC.
Resource	<code>sr =b</code>	Specifies which resources are accessible via the SAS. This example specifies that the accessible resource is in Blob Storage.

Parameter	Example	Description
Permissions	<code>sp =rw</code>	Lists the permissions to grant. This example grants access to read and write operations.
IP range	<code>sip =168.1.5.60-168.1.5.70</code>	Specifies a range of IP addresses from which a request is accepted. This example defines the IP address range 168.1.5.60 through 168.1.5.70.
Protocol	<code>spr =https</code>	Specifies the protocols from which Azure Storage accepts the SAS. This example indicates that only requests by using HTTPS are accepted.
Signature	<code>sig =F%6GRVAZ5Cdj2Pw4tgU7I1 STkWgn7bUkkAg8P6HESXwmf%4B</code>	Specifies that access to the resource is authenticated by using a Hash-Based Message Authentication Code (HMAC) signature. The signature is computed with a key using the SHA256 algorithm, and encoded by using Base64 encoding.

Determine Azure Storage encryption

Azure Storage encryption for data at rest protects your data by ensuring your organizational security and compliance commitments are met. The encryption and decryption processes happen automatically. Because your data is secured by default, you don't need to modify your code or applications.

Microsoft recommends that you use Azure Key Vault to manage your access keys, and that you regularly rotate and regenerate your keys. Azure Key Vault supports automatic key rotation

policies, allowing you to define rotation schedules (for example, every 90 days) that rotate keys automatically. You can also manually rotate your keys when needed.

Things to know about Azure Storage encryption

- Data is automatically encrypted before written to Azure storage.
- Data is automatically decrypted when retrieved.
- Azure Storage encryption, encryption at rest, decryption, and key management are transparent to users.
- All data written to Azure Storage is encrypted through 256-bit advanced encryption standard (AES) encryption. AES is one of the strongest block ciphers available.
- Azure Storage encryption is enabled for all new and existing storage accounts and can't be disabled.

Configure Azure Storage encryption

In the Azure portal, you configure Azure Storage encryption by specifying the encryption type. You can manage the keys yourself, or you can have the keys managed by Microsoft.

- **Infrastructure encryption.** [Infrastructure encryption](#) can be enabled for the entire storage account, or for an encryption scope within an account. When infrastructure encryption is enabled for a storage account or an encryption scope, data is encrypted twice—once at the service level and once at the infrastructure level—with two different encryption algorithms and two different keys.
- **Platform-managed keys.** Platform-managed keys (PMKs) are encryption keys generated, stored, and managed entirely by Azure. Customers don't interact with PMKs. The keys used for Azure Data Encryption-at-Rest, for instance, are PMKs by default.
- **Customer-managed keys.** Customer managed keys (CMK), on the other hand, are keys read, created, deleted, updated, and/or administered by one or more customers. Keys stored in a customer-owned key vault or hardware security module (HSM) are CMKs. Bring Your Own Key (BYOK) is a CMK scenario in which a customer imports (brings) keys from an outside storage location. This topic is discussed in more detail on the next page.

Create customer-managed keys

For your Azure Storage security solution, you can use Azure Key Vault to manage your encryption keys. The Azure Key Vault APIs can be used to generate encryption keys. You can also create your own encryption keys and store them in a key vault.

Things to know about customer-managed keys

- By creating your own keys (referred to as *customer-managed* keys), you have more flexibility and greater control.
- You can create, disable, audit, rotate, and define access controls for your encryption keys.
- Customer-managed keys can be used with Azure Storage encryption. You can use a new key or an existing key vault and key. The Azure storage account and the key vault must be in the same region, but they can be in different subscriptions.
- Customer-managed keys are stored in a customer-owned Azure Key Vault or Azure Key Vault Managed HSM. Managed HSM provides FIPS 140-2 Level 3 validation for organizations with the highest compliance requirements.

Configure customer-managed keys

In the Azure portal, you can configure customer-managed encryption keys. You can create your own keys, or you can have the keys managed by Microsoft. Consider how you might use Azure Key Vault to create your own customer-managed encryption keys.

- **Encryption type:** Choose how the encryption key is managed: by Microsoft or by yourself (customer).
- **Encryption key:** Specify an encryption key by entering a URI, or select a key from an existing key vault.

Apply Azure Storage security best practices

[Storage insights](#) provides comprehensive monitoring of your Azure Storage accounts. Storage Insights delivers a unified view of your Azure Storage services performance, capacity, and availability.

What are the benefits of Storage insights?

- **Detailed Metrics and Logs.** Azure Storage Insights offers detailed metrics, logs, and diagnostic information that enhance visibility into storage operations. Insights helps in monitoring key performance indicators (KPIs) such as latency, throughput, capacity utilization, and transactions.
- **Enhanced Security and Compliance.** By using Azure Storage Insights, you can ensure enhanced security and compliance. It provides actionable insights and alerts that help in swiftly identifying and resolving security issues.
- **Role-Based Access Control (RBAC).** Azure Storage Insights integrates with Azure's security features, including role-based access control (RBAC), Microsoft Entra ID, connection strings, and access control list (ACL) permissions. RBAC ensures secure access to your data and resources.
- **Unified View.** It delivers a unified view of your Azure Storage services' performance, capacity, and availability, which is crucial for maintaining the security and efficiency of your storage accounts.

When to use Microsoft Defender for Storage

Microsoft Defender for Storage offers proactive threat detection for active security threats.

Key capabilities

- **Malware scanning.** Automatically scans blob uploads for malware and viruses.
- **Sensitive data threat detection.** Identifies when personally identifiable information (PII) or credentials are stored inappropriately.
- **Activity-based threat detection.** Monitors for unusual access patterns, suspicious download volumes, and hash reputation analysis.

4 - Configure Azure Files

Compare storage for file shares and blob data

[Azure Files](#) offers fully managed file shares in the cloud. You can access Azure file shares by using the Server Message Block (SMB), Network File System (NFS), and HTTP protocols. Clients can connect to Azure file shares from Windows, Linux, and macOS devices.

Things to know about Azure Files

- **Serverless deployment.** An Azure file share is a PaaS offering of a fully managed file share that doesn't require any infrastructure. You don't need to take care of any VMs, operating

systems, or updates.

- **Almost unlimited storage.** A single Azure file share can store up to 100 tebibytes (TiB) of files, and a file can be up to 4 TiB in size. The files are organized in a hierarchical folder structure in the same way as on on-premises file servers.
- **Data encryption.** The data on an Azure file share is encrypted at rest in an Azure datacenter and in transit on a network.
- **Access from anywhere.** By default, clients can access Azure file shares from anywhere if they have internet connectivity.
- **Integration into an existing environment.** You can control access to Azure file shares by using Microsoft Entra identities or AD DS identities that are synced to Microsoft Entra ID. This helps ensure that users can have the same experience accessing an Azure file share as when they access an on-premises file server.
- **Previous versions and backups.** You can create Azure file share snapshots that integrate with the Previous Versions feature in File Explorer. You can also use Azure Backup to back up Azure file shares.
- **Data redundancy.** Azure file share data replicates to multiple locations in the same Azure datacenter or across many Azure datacenters. The replication setting of the Azure storage account that includes the file share controls the data redundancy.

Things to consider when using Azure Files

- **Consider replacement and supplement options.** Replace or supplement traditional on-premises file servers or NAS devices by using Azure Files.
- **Consider global access.** Directly access Azure file shares by using most operating systems, such as Windows, macOS, and Linux, from anywhere in the world.
- **Consider lift and shift support.** *Lift and shift* applications to the cloud with Azure Files for apps that expect a file share to store file application or user data.
- **Consider using Azure File Sync.** Replicate Azure file shares to Windows Servers by using Azure File Sync. You can replicate on-premises or in the cloud for performance and distributed caching of the data where it's being used. We take a closer look at Azure File Sync in a later unit.
- **Consider shared applications.** Store shared application settings such as configuration files in Azure Files.
- **Consider diagnostic data.** Use Azure Files to store diagnostic data such as logs, metrics, and crash dumps in a shared location.
- **Consider tools and utilities.** Azure Files is a good option for storing tools and utilities that are needed for developing or administering Azure VMs or cloud services.

Compare Azure Files to Azure Blob Storage

It's important to understand when to use Azure Files to store data in file shares rather than using Azure Blob Storage to store data as blobs.

Azure Files (file shares)	Azure Blob Storage (blobs)
Azure Files provides the SMB and NFS protocols, client libraries, and a REST interface that allows access from anywhere to stored files.	Azure Blob Storage provides client libraries and a REST interface that allows unstructured data to be stored and accessed at a massive scale in block blobs.
- Files in an Azure Files share are true directory objects. - Data in Azure Files is accessed through file shares across multiple virtual machines.	- Blobs in Azure Blob Storage are a flat namespace. - Blob data in Azure Blob Storage is accessed through a container.

Azure Files (file shares)	Azure Blob Storage (blobs)
<i>Azure Files is ideal to lift and shift an application to the cloud that already uses the native file system APIs. Share data between the app and other applications running in Azure.</i>	<i>Azure Blob Storage is ideal for applications that need to support streaming and random-access scenarios.</i>
<i>Azure Files is a good option when you want to store development and debugging tools that need to be accessed from many virtual machines.</i>	<i>Azure Blob Storage is a good option when you want to be able to access application data from anywhere.</i>

Manage Azure file shares

Azure file shares don't support both the SMB and NFS protocols on the same file share, although you can create SMB and NFS Azure file shares within the same storage account.

Types of Azure file shares

Storage tier	Performance	Storage account type	Redundancy options	Billing model	Use cases
Premium	SSD-backed, consistent low latency	FileStorage	LRS, ZRS	Provisioned (pay for capacity reserved)	High-performance workloads requiring low latency
Transaction Optimized	HDD-backed, standard performance	General-purpose v2 (GPv2)	LRS, GRS, RA-GRS, ZRS, GZRS, RA-GZRS	Pay-as-you-go	High-transaction workloads, frequently accessed data
Hot	HDD-backed, standard performance	General-purpose v2 (GPv2)	LRS, GRS, RA-GRS, ZRS, GZRS, RA-GZRS	Pay-as-you-go	General-purpose team shares and collaborative workloads
Cool	HDD-backed, standard performance	General-purpose v2 (GPv2)	LRS, GRS, RA-GRS, ZRS, GZRS, RA-GZRS	Pay-as-you-go	Cost-efficient online archive and backup scenarios

Manage Azure file shares

Completed100 XP

- 5 minutes

Azure Files offers two industry-standard file system protocols for mounting Azure file shares: the Server Message Block (SMB) protocol and the Network File System (NFS) protocol. Azure file shares don't support both the SMB and NFS protocols on the same file share, although you can create SMB and NFS Azure file shares within the same storage account.

Types of Azure file shares

Azure Files supports two storage tiers: premium and standard. Standard file shares are created in general purpose (GPv2) storage accounts, while premium file shares are created in FileStorage storage accounts. The two storage tiers have the attributes described in the following table.

Storage tier	Performance	Storage account type	Redundancy options	Billing model	Use cases
Premium	SSD-backed, consistent low latency	FileStorage	LRS, ZRS	Provisioned (pay for capacity reserved)	High-performance workloads requiring low latency
Transaction Optimized	HDD-backed, standard performance	General-purpose v2 (GPv2)	LRS, GRS, RA-GRS, ZRS, GZRS, RA-GZRS	Pay-as-you-go	High-transaction workloads, frequently accessed data
Hot	HDD-backed, standard performance	General-purpose v2 (GPv2)	LRS, GRS, RA-GRS, ZRS, GZRS, RA-GZRS	Pay-as-you-go	General-purpose team shares and collaborative workloads
Cool	HDD-backed, standard performance	General-purpose v2 (GPv2)	LRS, GRS, RA-GRS, ZRS, GZRS, RA-GZRS	Pay-as-you-go	Cost-efficient online archive and backup scenarios

Types of authentication

Authentication method	Description
Identity-based authentication over SMB	SMB identity-based authentication supports three Active Directory sources: On-premises AD DS, Microsoft Entra Domain Services, and Microsoft Entra Kerberos. Once your Active Directory source is selected, assign Azure RBAC roles to users who need access to the file share.
Access key	An access key is an older and less flexible option. An Azure storage account has two access keys that can be used when making a request to the storage account, including to Azure Files. Access keys are static and provide full control access to Azure Files. Access keys should be secured and not shared with users, because they bypass all access control restrictions. A best practice is to avoid sharing storage account keys and use identity-based authentication whenever possible.
A Shared Access Signature (SAS) token	SAS is a dynamically generated Uniform Resource Identifier (URI) that's based on the storage access key. SAS provides restricted access rights to an Azure storage account. Restrictions include allowed permissions, start and expiry time, allowed IP addresses from where requests can be sent, and allowed protocols. With Azure Files, a SAS token is only used to provide REST API access from code.

Create file share snapshots

Azure Files provides the capability to take share [snapshots of file shares](#). Share snapshots provide point-in-time copies of your Azure file shares that protect against accidental deletion and enable recovery from application errors.

Things to know about file share snapshots

- Snapshots are incremental, read-only point-in-time copies at the share level.
- To reduce time and cost only captures from the last snapshot.
- Same experience for SMB and NFS shares in all Azure public regions.
- Snapshot adds a unique timestamp to the share URI.
- Uses the shares redundancy settings.
- Up to 200 snapshots per file share for low-RPO recovery points.
- Snapshots persist until deleted. Deleting the share deletes all snapshots.
- Azure Backup can lease snapshots to help prevent accidental deletion.
- Restore a file, folder, or full share; full restore requires only the latest snapshot.

Things to consider when using file share snapshots

Benefit	Description
Protect against application error and data corruption	File-share workloads constantly read and write data. If a misconfiguration, bad deployment, or software bug overwrites or corrupts data, a snapshot lets you roll the share back to a known-good point in time. Take a snapshot before releasing new code so you have a clean restore point if something goes wrong.
Protect against accidental deletions or unintended changes	If a file is changed, snapshots give you a quick way to restore an earlier version. Use snapshots to roll back to the last good copy when something unexpected happens.
Support backup and recovery	Create snapshots on a schedule to build a backup history for your file share. Keeping prior versions makes it easier to meet audit needs and recover data after mistakes or a broader outage.

Implement soft delete for Azure Files

Azure Files offers [soft delete for file shares](#). Soft delete lets you recover deleted files and file shares.

Things to know about soft delete for Azure Files

Let's take a look at the characteristics of soft delete for Azure Files.

- Soft delete for file shares is enabled at the storage account level.
- Soft delete transitions content to a soft deleted state instead of being permanently erased.
- Soft delete lets you configure the retention period. The retention period is the amount of time that soft deleted file shares are stored and available for recovery.
- Soft delete provides a retention period between 1 and 365 days.
- Soft delete can be enabled on either new or existing file shares.

Things to consider when using soft delete for Azure Files

- **Recover from accidental data loss.** You can recover deleted or corrupted data with soft delete.
- **Upgrade scenarios.** Use soft delete to restore to a known good state after a failed upgrade attempt.
- **Ransomware protection.** Use soft delete to recover data without paying ransom to cybercriminals.
- **Long-term retention.** Use soft delete to comply with data retention requirements.

- **Business continuity.** Use soft delete to prepare your infrastructure to be highly available for critical workloads.

Use Azure Storage Explorer

[Azure Storage Explorer](#) is a standalone application that makes it easy to work with Azure Storage data on Windows, macOS, and Linux. With Azure Storage Explorer, you can access multiple accounts and subscriptions, and manage all your Storage content.

Things to know about Azure Storage Explorer

Azure Storage Explorer has the following characteristics.

- Azure Storage Explorer requires both management (Azure Resource Manager) and data layer permissions to allow full access to your resources. You need Microsoft Entra ID permissions to access your storage account, the containers in your account, and the data in the containers.
- Azure Storage Explorer lets you connect to different storage accounts.
 - Connect to storage accounts associated with your Azure subscriptions.
 - Connect to storage accounts and services that are shared from other Azure subscriptions.
 - Connect to and manage local storage by using the Azure Storage Emulator.

Things to consider when using Azure Storage Explorer

Scenario	Description
Connect to an Azure subscription	Manage storage resources that belong to your Azure subscription.
Work with local development storage	Manage local storage by using the Azure Storage Emulator.
Attach to external storage	Manage storage resources that belong to another Azure subscription or that are under national Azure clouds by using the storage account name, key, and endpoints. This scenario is described in more detail in the next section.
Attach a storage account with a SAS	Manage storage resources that belong to another Azure subscription by using a shared access signature (SAS).
Attach a service with a SAS	Manage a specific Azure Storage service (blob container, queue, or table) that belongs to another Azure subscription by using a SAS.

Consider Azure File Sync

[Azure File Sync](#) enables you to cache several Azure Files shares on an on-premises Windows Server or cloud virtual machine. You can use Azure File Sync to centralize your organization's file shares in Azure Files, while keeping the flexibility, performance, and compatibility of an on-premises file server.

Azure File Sync consists of five main components that work together to synchronize files between on-premises Windows Servers and Azure file shares.

- The **Storage Sync Service** is the primary Azure resource responsible for managing file synchronization. It can support up to 100 sync groups, operates within a single Azure region, and allows for up to 99 registered Windows Servers.
- The **sync group** establishes the synchronization setup, containing one cloud endpoint (Azure file share) and up to 50 server endpoints. Server endpoints are specific NTFS paths on

registered Windows Servers, but cannot be on the system volume, and cloud tiering is not supported there.

- The **cloud endpoint** is an Azure file share that participates in the sync group. Only one cloud endpoint is allowed per sync group.
- The **server endpoint** is a path on a registered Windows Server that syncs with the cloud endpoint. The server endpoint must be an NTFS-formatted volume, and can't be a system volume.
- The **Azure File Sync Agent** is installed on each Windows Server. The agent is a background Windows service for sync operations and management tasks.

Things to know about Azure File Sync

- Azure File Sync transforms Windows Server into a quick cache of your Azure Files shares.
- You can use any protocol that's available on Windows Server to access your data locally with Azure File Sync, including SMB, NFS, and FTPS.
- Azure File Sync supports as many caches as you need around the world.
- There is a maximum of 100 sync groups per Storage Sync Service, and 50 server endpoints per sync group.

Things to consider when using Azure File Sync

- **Consider application lift and shift.** Use Azure File Sync to move applications that require access between Azure and on-premises systems. Provide write access to the same data across Windows Servers and Azure Files.
- **Consider support for branch offices.** Support your branch offices that need to back up files by using Azure File Sync. Use the service to set up a new server that connects to Azure storage.
- **Consider backup and disaster recovery.** After you implement Azure File Sync, Azure Backup backs up your on-premises data. Restore file metadata immediately and recall data as needed for rapid disaster recovery.
- **Consider file archiving with cloud tiering.** Azure File Sync stores only recently accessed data on local servers. Implement cloud tiering so older data moves to Azure Files.

The main takeaways for this module are:

- Azure Files provides the SMB and NFS protocols, client libraries, and a REST interface that allows access from anywhere to stored files.
- Azure Files is ideal to lift and shift an application to the cloud that already uses the native file system APIs. Share data between the app and other applications running in Azure.
- Azure Files offers two industry-standard file system protocols for mounting Azure file shares: the Server Message Block (SMB) protocol and the Network File System (NFS) protocol.
- Azure Files offers two types of file shares: standard and premium. The premium tier stores data on modern solid-state drives (SSDs), while the standard tier uses hard disk drives (HDDs).
- File share snapshots capture a point-in-time, read-only copy of your data.
- Soft delete allows you to recover your deleted file share.
- Azure Storage Explorer is a standalone application that makes it easy to work with stored data on Windows, macOS, and Linux.
- Azure File Sync enables you to cache file shares on an on-premises Windows Server or cloud virtual machine.

1 - Introduction to Azure VMs

Performing a migration of on-premises servers to Azure requires planning and care. You can move them all at once, or more likely, in small batches or even individually. Before you create a single VM, you should sit down and sketch out your current infrastructure model and see how it might map to the cloud.

What is an Azure resource?

An **Azure resource** is a manageable item in Azure. Just like a physical computer in your datacenter, VMs have several elements that are needed to do their job:

- The VM itself
- Disks for storage
- Virtual network
- Network interface to communicate on the network
- Network Security Group (NSG) to secure the network traffic
- An IP address (public, private, or both)

A good convention for VMs names is to include the following information in the name:

Element	Example	Notes
Environment	dev, prod, QA	Identifies the environment for the resource
Location	eus for East US, jw for Japan West	Identifies the region into which the resource is deployed
Instance	01, 02	For resources that have more than one named instance (web servers, etc.)
Product or Service	service	Identifies the product, application, or service that the resource supports
Role	sql, web, messaging	Identifies the role of the associated resource

The best way to determine the appropriate VM size is to consider the type of workload your VM needs to run. Based on the workload, you're able to choose from a subset of available VM sizes. Workload options are classified as follows on Azure:

Option	Description
General purpose	General-purpose VMs are designed to have a balanced CPU-to-memory ratio. Ideal for testing and development, small to medium databases, and low to medium traffic web servers.
Compute optimized	Compute optimized VMs are designed to have a high CPU-to-memory ratio. Suitable for medium traffic web servers, network appliances, batch processes, and application servers.
Memory optimized	Memory optimized VMs are designed to have a high memory-to-CPU ratio. Great for relational database servers, medium to large caches, and in-memory analytics.
Storage optimized	Storage optimized VMs are designed to have high disk throughput and IO. Ideal for VMs running databases.
GPU	GPU VMs are specialized virtual machines targeted for heavy graphics rendering and video editing. These VMs are ideal options for model training and inferencing with deep learning.
High performance compute	High performance compute is the fastest and most powerful CPU virtual machines with optional high-throughput network interfaces.

The default resources supporting a virtual machine and how they're billed are detailed in the following table:

Resource	Description	Cost
Virtual network	For giving your virtual machine the ability to communicate with other resources	Virtual Network pricing
A virtual Network Interface Card (NIC)	For connecting to the virtual network	There is no separate cost for NICs. However, there is a limit to how many NICs you can use based on your VM's size . Size your VM accordingly and reference Virtual Machine pricing .
A private IP address and sometimes a public IP address.	For communication and data exchange on your network and with external networks	IP Addresses pricing
Network security group (NSG)	For managing the network traffic too and from your VM. For example, you might need to open port 22 for SSH access, but you might want to block traffic to port 80. Blocking and allowing port access is done through the NSG.	There are no additional charges for network security groups in Azure.
OS Disk and possibly separate disks for data.	It's a best practice to keep your data on a separate disk from your operating system, in case you ever have a VM fail, you can simply detach the data disk, and attach it to a new VM.	All new virtual machines have an operating system disk and a local disk. Azure doesn't charge for local disk storage. The operating system disk, which is usually 127GiB but is smaller for some images, is charged at the regular rate for disks . You can see the cost for attach Premium (SSD based) and Standard (HDD) based disks to your virtual machines on the Managed Disks pricing page .
In some cases, a license for the OS	For providing your virtual machine runs to run the OS	The cost varies based on the number of cores on your VM, so size your VM accordingly . The cost can be reduced through the Azure Hybrid Benefit .

You're able to choose from two payment options for compute costs.

Option	Description
Pay as you go	With the pay-as-you-go option, you pay for compute capacity by the second, with no long-term commitment or upfront payments. You're able to increase or decrease compute capacity on demand and start or stop at any time. Select this option if you run applications with short-term or unpredictable workloads that can't be interrupted. For example, if you're doing a quick test, or developing an app in a VM, pay-as-you-go is the appropriate option.

Option	Description
Reserved Virtual Machine Instances	The Reserved Virtual Machine Instances (RI) option is an advance purchase of a virtual machine for one or three years in a specified region. The commitment is made up front, and in return, you get up to 72% price savings compared to pay-as-you-go pricing. RI s are flexible and can easily be exchanged or returned for an early termination fee. Select this option if the VM has to run continuously, or you need budget predictability, and you can commit to using the VM for at least a year.

Storage for the VM

All Azure virtual machines have at least two virtual hard disks (VHDs). The first disk stores the operating system, and the second is used as temporary storage. You should add more data disks to store application data. Separating out the data to different disks allows you to manage the disks independently. The VM size determines the maximum number of data disks you can attach to your VM, typically two per vCPU.

There are five disk types, each intended to address a specific customer scenario:

- [Ultra disks](#)
- [Premium SSD v2 \(preview\)](#)
- [Premium SSDs \(solid-state drives\)](#)
- [Standard SSDs](#)
- [Standard HDDs \(hard disk drives\)](#)

Ultra disk	Premium SSD v2	Premium SSD	Standard SSD	Standard HDD
Disk type	SSD	SSD	SSD	SSD
Scenario	IO-intensive workloads such as SAP HANA , top tier databases (for example, SQL, Oracle), and other transaction-heavy workloads.	Production and performance-sensitive workloads that consistently require low latency and high IOPS and throughput	Production and performance sensitive workloads	Web servers, lightly used enterprise applications and dev/test
Max disk size	65,536 gibibytes (GiB)	65,536 GiB	32,767 GiB	32,767 GiB
Max throughput	4,000 MB/s	1,200 MB/s	900 MB/s	750 MB/s
Max IOPS	160,000	80,000	20,000	6,000
Usable as OS Disk?	No	No	Yes	Yes

Describe the options available to create and manage an Azure Virtual Machine

The Azure portal is the easiest way to create resources such as VMs when you're getting started. However, it's not necessarily the most efficient or quickest way to work with Azure, particularly if you need to create several resources together.

other ways to create and administer resources in Azure:

- Azure Resource Manager templates

- Azure PowerShell
- Azure CLI
- Azure REST API
- Azure Client SDK
- Azure VM Extensions
- Azure Automation Services

Resource Manager templates

Let's assume you want to create a copy of a VM with the same settings. You could create a VM image, upload it to Azure, and reference it as the basis for your new VM. This process is inefficient and time-consuming. Azure provides you with the option to create a template from which to create an exact copy of a VM.

***Resource Manager templates** are JSON files that define the resources you need to deploy for your solution.*

Azure CLI

The Azure CLI is Microsoft's cross-platform command-line tool for managing Azure resources such as virtual machines and disks from the command line. It's available for Linux, macOS, Windows, or in a browser using the Cloud Shell.

```
# create a VM
az vm create --resource-group TestResourceGroup --name test-wp1-eus-vm --image
Ubuntu2204 --admin-username azureuser --generate-ssh-keys

# powershell
New-AzVm -ResourceGroupName "TestResourceGroup" -Name "test-wp1-eus-vm" -Location
"East US" -Image Debian11 -VirtualNetworkName "test-wp1-eus-network" -SubnetName
"default" -SecurityGroupName "test-wp1-eus-nsg" -PublicIpAddressName "test-wp1-eus-
pubip" -GenerateSshKey -SshKeyName myPSKey
    -OpenPorts 22
```

Terraform

Azure also has a Terraform provider, so you can easily use Terraform to create and manage your VMs. Terraform enables the definition, preview, and deployment of cloud infrastructure. Using Terraform, you create configuration files using HCL syntax. The HCL syntax allows you to specify the cloud provider - such as Azure - and the elements that make up your cloud infrastructure.

Azure REST API

The Azure REST API provides developers with operations categorized by resource and the ability to create and manage VMs. Operations are exposed as URIs with corresponding HTTP methods (GET , PUT , POST , DELETE , and PATCH) and a corresponding response.

The Azure Compute APIs give you programmatic access to virtual machines and their supporting resources.

Here's an example snippet of C# code to create an Azure VM using the `Microsoft.Azure.Management.Fluent` NuGet package.

```
var azure = Azure
    .Configure()
```

```

        .WithLogLevel(HttpLoggingDelegatingHandler.Level.Basic)
        .Authenticate(credentials)
        .WithDefaultSubscription();
// ...
var vmName = "test-wp1-eus-vm";

azure.VirtualMachines.Define(vmName)
    .WithRegion(Region.USEast)
    .WithExistingResourceGroup("TestResourceGroup")
    .WithExistingPrimaryNetworkInterface(networkInterface)
    .WithLatestWindowsImage("MicrosoftWindowsServer", "WindowsServer", "2012-R2-
Datacenter")
    .WithAdminUsername("jonc")
    .WithAdminPassword("aReallyGoodPasswordHere")
    .WithComputerName(vmName)
    .WithSize(VirtualMachineSizeTypes.StandardDS1)
    .Create();

// the same snippet in Java using the **Azure Java SDK**.
String vmName = "test-wp1-eus-vm";
// ...
VirtualMachine virtualMachine = azure.virtualMachines()
    .define(vmName)
    .withRegion(Region.US_EAST)
    .withExistingResourceGroup("TestResourceGroup")
    .withExistingPrimaryNetworkInterface(networkInterface)
    .withLatestWindowsImage("MicrosoftWindowsServer", "WindowsServer", "2012-R2-
Datacenter")
    .withAdminUsername("jonc")
    .withAdminPassword("aReallyGoodPasswordHere")
    .withComputerName(vmName)
    .withSize("Standard_DS1")
    .create();

```

Azure Automation services

Azure Automation enables you to integrate services that allow you to automate frequent, time-consuming, and error-prone management tasks with ease. These services include **process automation**, **configuration management**, and **update management**.

- **Process Automation.** Let's assume you have a VM that is monitored for a specific error event. You want to take action, and fix the problem as soon as it's reported. Process automation enables you to set up watcher tasks that can respond to events that may occur in your datacenter.
- **Configuration Management.** Perhaps you want to track software updates that become available for the operating system that runs on your VM. There are specific updates you may want to include or exclude. Configuration management enables you to track these updates, and take action as required. You use **Microsoft Endpoint Configuration Manager** to manage your company's PC, servers, and mobile devices. You can extend this support to your Azure VMs with Configuration Manager.
- **Update Management.** Use this service to manage updates and patches for your VMs. With this service, you're able to assess the status of available updates, schedule installation, and

review deployment results to verify updates applied successfully. Update management incorporates services that provide process and configuration management. You enable update management for a VM directly from your **Azure Automation** account. You can also enable update management for a single virtual machine from the virtual machine pane in the portal.

Auto-shutdown

Auto-shutdown is a feature in Azure that allows you to automatically shut down your VMs on a schedule. Use Auto-shutdown to save costs by ensuring that your VMs are not running when they aren't needed. You can set the schedule for auto-shutdown to occur daily or weekly, and you can also specify the time zone for the schedule.

Manage the availability of your Azure VMs

Azure provides tools you can use to manage availability, data security, and monitoring, so you know your services are always available for your customers.

Administration of an Azure VM isn't limited to managing the operating system, or software that runs on the VM. It helps to know which services Azure provides that ensure service availability and support automation. These services help you to plan your organization's business continuity and disaster recovery strategy.

Back up your virtual machines

Data backup and recovery is a necessary piece of the planning for any good infrastructure. Assume a bug erases some company data, or maybe you need to retrieve some archived data for auditing purposes.

Azure Backup is a *backup as a service* offering that protects physical or virtual machines no matter where they reside: on-premises or in the cloud

- Files and folders on Windows OS machines (physical or virtual, local or cloud)
- Application-aware snapshots (Volume Shadow Copy Service)
- Popular Microsoft server workloads such as Microsoft SQL Server, Microsoft SharePoint, and Microsoft Exchange
- Native support for Azure Virtual Machines, both Windows, and Linux
- Linux and Windows 10 client machines

Advantages of using Azure Backup

Azure Backup was designed to work in tandem with other Azure services and provides several distinct benefits.

- **Automatic storage management.** Azure Backup automatically allocates and manages backup storage and uses a pay-as-you-use model. You only pay for what you use.
- **Unlimited scaling.** Azure Backup uses the power and scalability of Azure to deliver high availability.
- **Multiple storage options.** Azure Backup offers locally redundant storage where all copies of the data exist within the same region and geo-redundant storage where your data is replicated to a secondary region.
- **Unlimited data transfer.** Azure Backup doesn't limit the amount of inbound or outbound data you transfer. Azure Backup also doesn't charge for the data that is transferred.
- **Data encryption.** Data encryption allows for secure transmission and storage of your data in Azure.

- **Application-consistent backup.** An application-consistent backup means that a recovery point has all required data to restore the backup copy. Azure Backup provides application-consistent backups.
- **Long-term retention.** Azure doesn't limit the length of time you keep the backup data.

Use Azure Backup

Azure Backup uses several components that you download and deploy to each computer you want to back up. The component that you deploy depends on what you want to protect.

- Azure Backup agent
- System Center Data Protection Manager
- Azure Backup Server
- Azure Backup VM extension

Azure Backup uses a Recovery Services vault for storing the backup data. Azure Storage blobs back up a vault, making it an efficient and economical long-term storage medium.

2 - Configure virtual machine availability

Things to know about maintenance planning

An availability plan for Azure virtual machines needs to include strategies for unplanned hardware maintenance, unexpected downtime, and planned maintenance.

- An **unplanned hardware maintenance** event occurs when the Azure platform predicts that the hardware or any platform component associated to a physical machine is about to fail. When the platform predicts a failure, it issues an unplanned hardware maintenance event. Azure uses Live Migration technology to migrate your virtual machines from the failing hardware to a healthy physical machine. Live Migration is a virtual machine preserving operation that only pauses the virtual machine for a short time, but performance might be reduced before or after the event.
- **Unexpected downtime** occurs when the hardware or the physical infrastructure for your virtual machine fails unexpectedly. Unexpected downtime can include local network failures, local disk failures, or other rack level failures. When detected, the Azure platform automatically migrates (heals) your virtual machine to a healthy physical machine in the same datacenter. During the healing procedure, virtual machines experience downtime (reboot) and in some cases loss of the temporary drive.
- **Planned maintenance** events are periodic updates made by Microsoft to the underlying Azure platform to improve overall reliability, performance, and security of the platform infrastructure that your virtual machines run on.

Create availability sets

An availability set is a logical feature you can use to ensure a group of related virtual machines are deployed together. The grouping helps to prevent a single point of failure from affecting all of your machines.

Things to know about availability sets

Let's review some characteristics of availability sets.

- All virtual machines in an availability set should perform the identical set of functionalities.
- All virtual machines in an availability set should have the same software installed.

- Azure ensures that virtual machines in an availability set run across multiple physical servers, compute racks, storage units, and network switches. If a hardware or Azure software failure occurs, only a subset of the virtual machines in the availability set are affected. Your application stays up and continues to be available to your customers.
- You can create a virtual machine and an availability set at the same time. A virtual machine can only be added to an availability set when the virtual machine is created. To change the availability set for a virtual machine, you need to delete and then recreate the virtual machine.
- You can build availability sets by using the Azure portal, Azure Resource Manager (ARM) templates, scripting, or API tools.

Things to consider when using availability sets

Availability sets are an essential capability when you want to build reliable cloud solutions.

- **Consider redundancy.** To achieve redundancy in your configuration, place multiple virtual machines in an availability set.
- **Consider separation of application tiers.** Each application tier exercised in your configuration should be located in a separate availability set. The separation helps to mitigate single point of failure on all machines.
- **Consider load balancing.** For high availability and network performance, create a load-balanced availability set by using Azure Load Balancer. Load Balancer distributes incoming traffic across working instances of services that are defined in your load-balanced availability set.
- **Consider managed disks.** You can use Azure managed disks with your Azure virtual machines in availability sets for block-level storage.

Review update domains and fault domains

Azure Virtual Machine Availability Sets implements two node concepts to help Azure maintain high availability and fault tolerance when deploying and upgrading applications: *update domains* and *fault domains*. Each virtual machine in an availability set is placed in one update domain and one fault domain.

Things to know about update domains

An update domain is a group of nodes that are upgraded together during the process of a service upgrade (or *roll out*). An update domain allows Azure to perform incremental or rolling upgrades across a deployment. Here are some other characteristics of update domains.

- Each update domain contains a set of virtual machines and associated physical hardware that can be updated and rebooted at the same time.
- During planned maintenance, only one update domain is rebooted at a time.
- You can specify between 1 and 20 update domains when creating an availability set. If you don't specify a value, Azure defaults to five update domains.
- The update domain count is immutable after creation; to change it, you must delete and recreate the availability set.

Things to know about fault domains

A fault domain is a group of nodes that represent a physical unit of failure. Think of a fault domain as nodes that belong to the same physical rack.

- A fault domain defines a group of virtual machines that share a common set of hardware (or *switches*) that share a single point of failure. An example is a server rack serviced by a set of power or networking switches.

- Two fault domains work together to mitigate against hardware failures, network outages, power interruptions, or software updates.

Review availability zones

Availability zones are a high-availability offering that protects your applications and data from datacenter failures. You can use availability zones to build high-availability into your application architecture by colocating your compute, storage, networking, and data resources within a zone and replicating in other zones.

Things to know about availability zones

Review the following characteristics of availability zones.

- Availability zones are unique physical locations within an Azure region.
- Each zone is made up of one or more datacenters that are equipped with independent power, cooling, and networking.
- To ensure resiliency, there's a minimum of three separate zones in all enabled regions.
- The physical separation of availability zones within a region protects applications and data from datacenter failures.
- Zone-redundant services replicate your applications and data across availability zones to protect against single-points-of-failure.

Things to consider when using availability zones

Category	Description	Examples
Zonal services	Azure <i>zonal</i> services pin each resource to a specific zone.	- Azure virtual machines - Azure managed disks
Zone-redundant services	For Azure services that are zone-redundant, the platform replicates automatically across all zones.	- Azure Storage that's zone-redundant - Azure SQL Database

Compare vertical and horizontal scaling

A robust virtual machine configuration includes support for scalability. Scalability allows throughput for a virtual machine in proportion to the availability of the associated hardware resources. A scalable virtual machine can handle increases in requests without adversely affecting response time and throughput. For most scaling operations, there are two implementation options: *vertical* and *horizontal*.

Things to know about vertical scaling

Vertical scaling, also known as *scale up and scale down*, involves increasing or decreasing the virtual machine **size** in response to a workload. Vertical scaling makes a virtual machine more (scale up) or less (scale down) powerful.

Things to know about horizontal scaling

Horizontal scaling, also referred to as *scale out and scale in*, is used to adjust the **number** of virtual machines in your configuration to support the changing workload. When you implement horizontal scaling, there's an increase (scale out) or decrease (scale in) in the number of virtual machine instances.

Things to consider when using vertical and horizontal scaling

- **Consider limitations.** Generally speaking, horizontal scaling has fewer limitations than vertical scaling. A vertical scaling implementation depends on the availability of larger hardware, which quickly hits an upper limit and can vary by region. Vertical scaling also usually requires a virtual machine to stop and restart, which can temporarily limit access to applications or data.
- **Consider flexibility.** When operating in the cloud, horizontal scaling is more flexible. A horizontal scaling implementation allows you to run potentially thousands of virtual machines to manage changes in workload and throughput.
- **Consider reprovisioning.** *Reprovisioning* is the process of removing an existing virtual machine and replacing it with a new machine. A robust availability plan considers where reprovisioning might be required and plans for interruptions to service. If reprovisioning might be required, determine if any data needs to be maintained and migrated to the new machine.

Implement Azure Virtual Machine Scale Sets

Azure Virtual Machine Scale Sets are an Azure Compute resource that you can use to deploy and manage a set of **identical** virtual machines. When you implement Virtual Machine Scale Sets and configure all your virtual machines in the same way, you gain true *autoscaling*. Virtual Machine Scale Sets automatically increases the number of your virtual machine instances as application demand increases, and reduces the number of machine instances as demand decreases.

The process of adding and removing machines can be manual or automated, or a combination of both.

Things to know about Azure Virtual Machine Scale Sets

- Virtual Machine Scale Sets support the use of Azure Load Balancer for basic layer-4 traffic distribution, and Azure Application Gateway for more advanced layer-7 traffic distribution and TLS/SSL termination.
- You can use Virtual Machine Scale Sets to run multiple instances of your application. If one of the virtual machine instances has a problem, customers continue to access your application through another virtual machine instance with minimal interruption.
- There are two types of orchestration modes available for Azure virtual machine scale sets: Uniform and Flexible. In **Uniform orchestration mode**, all virtual machine instances are created from the same base operating system image and configuration. In **Flexible orchestration mode**, VMs can use different images, sizes, or configurations within the same scale set. *The orchestration mode must be chosen when the scale set is created.*
- Customer demand for your application might change throughout the day or week. To meet customer demand, Virtual Machine Scale Sets implements autoscaling to automatically increase and decrease the number of virtual machines.

Create Virtual Machine Scale Sets

- **Orchestration mode:** Choose how the scale set manages the virtual machines. Flexible orchestration is the default and recommended mode for new deployments. For most new workloads, accept the Flexible default unless you have a specific requirement for identical instances.
- **Image:** Choose the base operating system or application for the VM.
- **VM Architecture:** Azure provides a choice of x64 or Arm64-based virtual machines to run your applications. x64-based VMs provide the most software compatibility. Arm64-based VMs provide up to 50% better price-performance than comparable x64 VMs.

- **Size:** Select a VM size to support the workload that you want to run. The size that you choose then determines factors such as processing power, memory, and storage capacity. Azure offers a wide variety of sizes to support many types of uses. Azure charges an hourly price based on the VM's size and operating system.

Under the **Advanced** tab, you can also select the following:

- **Spreading algorithm:** The spreading algorithm determines how VMs in the scale set are balanced across fault domains. With max spreading, VMs are spread across as many fault domains as possible in each zone. With fixed spreading, VMs are always spread across exactly five fault domains. In the case where fewer than five fault domains are available, a scale set using "Max spreading" completes, while a scale set using "Fixed spreading" fails. For this reason, Microsoft recommends using **Max spreading** for your implementation.

Implement autoscale

An Azure Virtual Machine Scale Sets implementation can automatically increase or decrease the number of virtual machine instances that run your application. This process is known as *autoscaling*.

Things to consider when using autoscaling

- **Consider automatic adjusted capacity.** You can create autoscaling rules to define the acceptable performance for a positive customer experience. When the defined thresholds are met, the autoscale rules act to adjust the capacity of your Virtual Machine Scale Sets implementation.
- **Consider scale out.** If your application demand increases, the load on the virtual machine instances in your implementation increases. If the increased load is consistent, rather than a brief demand, you can configure autoscale rules to increase the number of virtual machine instances in your implementation.
- **Consider scale in.** On an evening or weekend, your application demand might decrease. If the decreased load is consistent over a period of time, you can configure autoscale rules to decrease the number of virtual machine instances in your implementation. The scale-in action reduces the cost to run your Virtual Machine Scale Sets implementation as you only run the number of instances required to meet the current demand.
- **Consider scheduled events.** You can implement autoscaling and schedule events to automatically increase or decrease the capacity of your implementation at fixed times.
- **Consider overhead.** Using Azure Virtual Machine Scale Sets with autoscaling reduces your management overhead to monitor and optimize the performance of your application.

Configure autoscale

Scaling mode

- **Manually update the capacity:** Maintain a fixed instances count. Set the **Instance count** to the number of virtual machines in the scale set (0 - 1000). Configure the [Scale-in policy](#) which is the order virtual machines are selected for deletion. For example, you could balance across zones and then delete the virtual machine with the highest instance ID.
- **Autoscaling:** Scaling based on a CPU metric, or any schedule.
- **Default instance count.** The initial number of virtual machines deployed in this scale set (0-1000).
- **Instance limit.** The minimum instance count you want this condition to scale down to. The maximum instance count you want this condition to scale up to.

- **Scale out.** The CPU usage percentage threshold for triggering the scale-out autoscale rule. The number of instances to scale out by.
- **Scale in.** The CPU usage percentage threshold for triggering the scale-in autoscale rule. The number of instances to scale in by.
- **Query duration:** This duration is the time the Autoscale engine looks back for the metric usage average. This look back is to allow your metric to stabilize.
- **Schedule:** Specify the start and end dates. You can also repeat the schedule on specific days.

The main takeaways from this module are:

- Azure Virtual Machine Scale Sets allow for the deployment and management of a group of identical virtual machines, making it easier to build large-scale services.
- Autoscaling with Virtual Machine Scale Sets helps optimize performance by automatically adjusting the number of instances based on workload demands.
- Availability sets and availability zones are important features in Azure for achieving high availability and fault tolerance for virtual machines.

3 - Configure Azure App Service plans

Implement Azure App Service plans

An App Service plan defines a set of compute resources for a web application to run. The compute resources are analogous to a server farm in conventional web hosting. One or more applications can be configured to run on the same computing resources (or in the same App Service plan).

Things to know about App Service plans

- When you create an App Service plan in a region, a set of compute resources is created for the plan in the specified region. Any applications that you place into the plan run on the compute resources defined by the plan.
- Each App Service plan defines these settings:
 - **Operating system:** Linux or Windows.
 - **Region:** The region for the App Service plan, such as West US, Central India, North Europe, and so on.
 - **Pricing tier:** Determines what App Service features you get and how much you pay for the plan. The pricing tiers available to your App Service plan depend on the operating system selected at creation time.
 - **Number of VM instances:** Determined by your plan.
 - **Size of VM instances:** Defined by CPU, memory, and remote storage.
- You can continue to add new applications to an existing plan as long as the plan has enough resources to handle the increasing load.

Things to consider when using App Service plans

- **Consider cost savings.** Because you pay for the computing resources that your App Service plan allocates, you can potentially save money by placing multiple applications into the same App Service plan.
- **Consider multiple applications in one plan.** Create a single plan to support multiple applications, to make it easier to configure and maintain shared virtual machine instances. Because the applications share the same virtual machine instances, you need to carefully manage your plan resources and capacity.

- **Consider plan capacity.** Before you add a new application to an existing plan, determine the resource requirements for the new application and identify the remaining capacity of your plan.
- **Consider application isolation.** Isolate your application into a new App Service plan when:
 - The application is resource-intensive.
 - You want to scale the application independently from the other applications in the existing plan.
 - The application needs resource in a different geographical region.

Determine Azure App Service plan pricing

The pricing tier of an Azure App Service plan determines what App Service features you get and how much you pay for the plan. Pricing tier examples are: Free, Shared, Basic, Standard, Premium, PremiumV2, PremiumV3, Isolated, and IsolatedV2.

How applications run and scale in App Service plans

The Azure App Service plan is the scale unit of App Service applications. Depending on the pricing tier for your Azure App Service plan, your applications run and scale in a different manner. If your plan is configured to run five virtual machine instances, then all applications in the plan run on all five instances. If your plan is configured for autoscaling, then all applications in the plan are scaled out together based on the autoscale settings.

The pricing tiers are grouped into three categories:

- **Shared compute:**
 - Free and Shared, the two base tiers, run an app on the same Azure VM as other App Service apps, including apps of other customers.
 - These tiers allocate CPU quotas to each app that runs on the shared resources, and the resources can't scale out.
 - These tiers are intended to be used only for development and testing purposes.
- **Dedicated compute:**
 - The Basic, Standard, Premium, PremiumV2, and PremiumV3 tiers run apps on dedicated Azure VMs.
 - Only apps in the same App Service plan have the same compute resources. The higher the tier, the more VM instances that are available to you for scale-out.
- **Isolated:**
 - The Isolated and IsolatedV2 tiers run dedicated Azure VMs on dedicated Azure virtual networks.
 - This tier provides network isolation on top of compute isolation to your apps.
 - This tier provides the maximum scale-out capabilities.

Free and Shared

The Free and Shared service plans are base tiers that run on the same Azure virtual machines as other applications. Some applications might belong to other customers. These tiers are intended to be used for development and testing purposes only. No SLA is provided for the Free and Shared service plans. Free and Shared plans are metered on a per application basis.

Basic

The Basic service plan is designed for applications that have lower traffic requirements, and don't need advanced auto scale and traffic management features. Pricing is based on the size and number of instances you run. Built-in network load-balancing support automatically distributes

traffic across instances. The Basic service plan with Linux runtime environments supports Web App for Containers.

Standard

The Standard service plan is designed for running production workloads. Pricing is based on the size and number of instances you run. Built-in network load-balancing support automatically distributes traffic across instances. The Standard plan includes auto scale that can automatically adjust the number of virtual machine instances running to match your traffic needs. The Standard service plan with Linux runtime environments supports Web App for Containers.

Premium

The Premium service plan is designed for production apps that need higher performance and scale. PremiumV3 is the current Premium tier, offering Dav4 and Ddv4-series virtual machines and SSD storage. PremiumV3 supports standard compute SKUs and memory-optimized SKUs for high-memory workloads. PremiumV3 supports both rule-based autoscaling and automatic scaling. PremiumV3 is recommended for new deployments.

Isolated

The Isolated service plan supports mission-critical workloads needing network isolation. IsolatedV2 is the preferred tier offering newer hardware, up to 200 instances, private environments, and enhanced security. IsolatedV2 is recommended for new workloads due to better performance and simpler pricing.

Scale up and scale out Azure App Service

There are two methods for scaling your Azure App Service plan and applications: *scale up* and *scale out*. You can scale your applications manually or automatically, which is referred to as *autoscale*.

Things to know about Azure App Service scaling

- The scale up method increases the amount of CPU, memory, and disk space. Scaling up gives you extra features like dedicated virtual machines, custom domains and certificates, staging slots, autoscaling, and more. You scale up by changing the pricing tier of the Azure App Service plan where your application is placed.
- The scale-out method increases the number of virtual machine instances that run your application. You can scale out to the maximum number of instances for your pricing tier. Take advantage of App Service Environments in the Isolated tier to further increase your scale-out count to 100 instances. The scale instance count can be configured manually or automatically (autoscale).
- With autoscale, you can automatically increase the scale instance count for the scale-out method. Autoscale is based on predefined rules and schedules.
- Your App Service plan can be scaled up and down at any time by changing the pricing tier of the plan.

Things to consider when using Azure App Service scaling

Consider manually adjusting plan tiers. Start your plan at a lower pricing tier and scale up as needed to acquire more App Service features. Scale down when features are no longer needed, and control your overall costs.

The same scaling process works in reverse. If you decide you no longer need capabilities or features of a higher tier, scale your plan down to a lower tier and save money.

Consider autoscale to support users and reduce costs. Keep serving your users when your application is experiencing high throughput. Implement autoscale to control how many features and support are offered at a given time based on your preference settings and rule conditions.

- **Consider no redeployment.** When you change your scale settings, you don't need to change your code or redeploy your applications. Changing your plan scale settings takes only seconds to apply. Your changes affect all applications in your App Service plan.
- **Consider scaling for other Azure services.** If your App Service application depends on other Azure services, such as Azure SQL Database or Azure Storage, you can scale these resources separately.

Configure Azure App Service autoscale

The autoscale process allows you to have the right amount of resources running to handle the load on your application.

Things to know about autoscale

- To use autoscale, you specify the minimum, and maximum number of instances to run by using a set of rules and conditions.
- When your application runs under autoscale conditions, the number of virtual machine instances are automatically adjusted based on your rules. When rule conditions are met, one or more autoscale actions are triggered.
- An autoscale setting is used by the autoscale engine to determine whether to scale out or in. Autoscale settings are grouped into profiles.
- Autoscale rules include a trigger and a scale action (in or out). The trigger can be metric-based or time-based.
- **Metric-based** rules measure application load and add or remove virtual machines based on the load, such as "do this action when CPU usage is above 50%." Example metrics include CPU time, Average response time, and Requests.
 - **Time-based** rules (or, schedule-based) allow you to scale when you see time patterns in your load and want to scale before a possible load increase or decrease occurs. An example is "trigger a webhook every 8:00 AM on Saturday in a given time zone."
- The autoscale engine uses notification settings.

Things to consider when configuring autoscale

- **Minimum instance count.** Set a minimum instance count to make sure your application is always running even when there's no load.
- **Maximum instance count.** Set a maximum instance count to limit your total possible hourly cost.
- **Adequate scale margin.** Make sure your maximum and minimum instance count values are different, and set an adequate margin between the two values. You can automatically scale between the minimum and maximum by using rules you create.
- **Scale rule combinations.** Always use a scale-out and scale-in rule combination that performs an increase and decrease. If you don't set a scale-out rule, your application might fail, or performance might degrade under increased loads. If you don't set a scale-in rule, you can experience unnecessary and extensive costs when the load decreases.
- **Metric statistics.** Carefully choose the appropriate statistic for your diagnostic metrics, including Average, Minimum, Maximum, and Total.
- **Default instance count.** Always select a safe default instance count. The default instance count is important because autoscale scales your service to the count you specify when metrics aren't available.

- **Notifications.** Always configure autoscale notifications. It's important to maintain awareness of how your application is performing as the load changes.

Things to consider when configuring automatic scaling

In addition to rule-based autoscale, Azure App Service offers Automatic scaling (also called Elastic scaling) for PremiumV2 and PremiumV3 tiers.

This is a separate scaling feature that works differently from the autoscale rules.

- **HTTP traffic-based.** Automatic scaling responds directly to incoming HTTP requests without requiring you to configure scaling rules.
- **Platform-managed.** Azure automatically manages the scaling decisions based on traffic patterns, eliminating the need for rule configuration.
- **Always-ready instances.** Maintains warmed instances to handle traffic spikes immediately.
- **Tier availability.** Available only on PremiumV2 and PremiumV3 tiers.

Choose between Autoscale and Automatic scaling

- **Use rule-based Autoscale.** You need custom scaling logic, want to scale based on multiple metrics, or need schedule-based scaling.
- **Use Automatic scaling.** You want less management, can't predict load patterns, or need fast response to traffic changes without rule configuration.

The main takeaways from this module are:

- Azure App Service plans are used to define the compute resources for running web applications in Azure App Service.
- The pricing tier of the App Service plan determines the features and cost, with options ranging from Free and Shared plans to Isolated plans.
- Scaling in Azure App Service can be done through scale up (changing the pricing tier) or scale out (increasing the number of virtual machine instances).
- Autoscaling allows for automatic adjustment of resources based on application load, with metric-based and time-based rules.

4 - Configure Azure App Service

Implement Azure App Service

[Azure App Service](#) brings together everything you need to create websites, mobile backends, and web APIs for any platform or device. Applications run and scale with ease in both Windows and Linux-based environments.

App Service benefits

Benefit	Description
Multiple languages and frameworks	App Service has first-class support for ASP.NET, Java, Node.js, PHP, and Python. You can also run PowerShell and other scripts or executables as background services.
DevOps optimization	App Service supports continuous integration and deployment with Azure DevOps, GitHub, BitBucket, Docker Hub, and Azure Container Registry. You can promote updates through test and staging environments. Manage your apps in App Service by using Azure PowerShell or the cross-platform command-line interface (CLI).

Benefit	Description
Global scale with high availability	App Service helps you scale up or out manually or automatically. You can host your apps anywhere within the Microsoft global datacenter infrastructure, and the App Service SLA offers high availability.
Security and compliance	App Service is ISO, SOC, and PCI compliant. You can authenticate users with Microsoft Entra ID or with social logins via Google, Facebook, X, or Microsoft. Create IP address restrictions and manage service identities.
Application templates	Choose from an extensive list of application templates in Azure Marketplace, such as WordPress, Joomla, and Drupal.
Visual Studio integration	App Service offers dedicated tools in Visual Studio to help streamline the work of creating, deploying, and debugging.
API and mobile features	App Service provides turn-key CORS support for RESTful API scenarios. You can simplify your mobile app scenarios by enabling authentication, offline data sync, push notifications, and more.

Create an app with App Service

Things to know about configuration settings

Let's examine some of the basic configuration settings you need to create an app with App Service.

- **Name:** The name for your app must be unique. The name identifies and locates your app in Azure. An example name is `webappces1.azurewebsites.net`. You can map a custom domain name, if you prefer to use that option instead.
- **Publish:** App Service hosts (publishes) your app as code or as a Docker Container.
- **Runtime stack:** App Service uses a software stack to run your app, including the language and SDK versions. For Linux apps and custom container apps, you can set an optional start-up command or file. Your choices for the stack include .NET Core, .NET Framework, Node.js, PHP, and Python. Various versions of each product are available for Linux and Windows.
- **Operating system:** The operating system for your app runtime stack can be Linux or Windows.
- **Region:** The region location that you choose for your app affects the App Service plans that are available.
- **Pricing plans:** Your app needs to be associated with an Azure App Service plan to establish available resources, features, and capacity. You can choose from pricing tiers that are available for the region location you selected.

Post-creation settings

After your app is created, other **Configuration** settings become available in the Azure portal, including app deployment options and path mapping.

Here are a few of the extra application settings.

- **Always On:** You can keep your app loaded even when there's no traffic. This setting is required for continuous WebJobs or for WebJobs that are triggered by using a CRON expression.
- **Session affinity:** In a multi-instance deployment, you can ensure your app client is routed to the same instance for the life of the session.
- **HTTPS Only:** When enabled, all HTTP traffic is redirected to HTTPS.

Explore continuous integration and deployment

The Azure portal provides out-of-the-box continuous integration and deployment with Azure DevOps services, GitHub, Bitbucket, FTP, or a local Git repository on your development machine. You can connect your web app with any of the above sources and App Service handles the rest for you. App Service autosynchronizes your code and any future changes to the code into your web app.

Things to know about continuous and manual deployment

When you create your web app with App Service, you can choose continuous or manual deployment.

Continuous deployment (CI/CD) is a process used to push out new features and bug fixes in a fast and repetitive pattern with minimal impact on end users. Azure supports automated deployment directly from several sources:

- **GitHub:** Azure supports automated deployment directly from GitHub. Azure supports automated deployment directly from GitHub using two build providers. When you connect your GitHub repository to Azure, you can choose between [GitHub Actions](#) (default) and [App Service Build Service](#).
- **Bitbucket:** With its similarities to GitHub, you can configure an automated deployment with Bitbucket.
- **Local Git:** The App Service Web Apps feature offers a local URL that you can add as a repository.
- **Azure Repos:** Azure Repos is a set of version control tools that you can use to manage your code. Whether your software project is large or small, using version control as soon as possible is a good idea.

Manual deployment enables you to manually push your code to Azure.

- **Remote Git:** The App Service Web Apps feature offers a Git URL that you can add as a remote repository. Pushing to the remote repository deploys your app.

Create deployment slots

When you deploy your web app, web app on Linux, mobile backend, or API app to Azure App Service, you can use a separate deployment slot instead of the default production slot.

Things to know about deployment slots

- Deployment slots are live apps that have their own hostnames.
- Deployment slots are available in the Standard, Premium, and Isolated v2 App Service pricing tiers. Your app needs to be running in one of these tiers to use deployment slots.
- The Standard, Premium, and Isolated tiers offer different numbers of deployment slots.
- App content and configuration elements can be swapped between two deployment slots, including the production slot.

Things to consider when using deployment slots

- **Consider validation.** You can validate changes to your app in a staging deployment slot before swapping the app changes with the content in the production slot.
- **Consider reductions in downtime.** Deploying an app to a slot first and swapping it into production ensures that all instances are ready. This option eliminates downtime when you deploy your app. The traffic redirection is seamless, and no requests are dropped because of swap operations. The entire workflow can be automated by configuring **Auto swap** when preswap validation isn't needed.

- **Consider restoring to last known good site.** After a swap, the slot with the previously staged app now has the previous production app. If the changes swapped into the production slot aren't as you expected, you can perform the same swap immediately to return to your "last known good site."
- **Consider Auto swap.** Auto swap streamlines Azure Pipeline scenarios where you want to deploy your app continuously with zero cold starts and zero downtime for app customers. When Auto swap is enabled from a slot into production, every time you push your code changes to that slot, App Service automatically swaps the app into production after it's warmed up in the source slot.

Add deployment slots

Deployment slots are configured in the Azure portal. You can swap your app content and configuration elements between deployment slots, including the production slot.

Things to know about creating deployment slots

- New deployment slots can be empty or cloned.
- Deployment slot settings fall into three categories:
 - Slot-specific app settings and connection strings (if applicable).
 - Continuous deployment settings (when enabled).
 - Azure App Service authentication settings (when enabled).
- When you clone a configuration from another deployment slot, the cloned configuration is editable. Some configuration elements follow the content across the swap. Other slot-specific configuration elements stay in the source slot after the swap.

Swapped settings versus slot-specific settings

The following table lists settings that are swapped between deployment slots. The table also lists settings that remain in the source slot (slot-specific).

Swapped settings	Slot-specific settings
Language stack and version, 32/64-bit App settings * Connection strings * Mounted storage accounts* Public certificates WebJobs content Hybrid connections ** Service endpoints ** Azure Content Delivery Network ** Path mapping	Custom domain names Nonpublic certificates and TLS/SSL settings Scale settings Always On IP restrictions WebJobs schedulers Diagnostic settings Cross-origin resource sharing (CORS) Virtual network integration Managed identities

Secure your App Service app

Azure App Service provides built-in [authentication and authorization](#) support. You can sign in users and access data by writing minimal or no code in your web app, API, and mobile backend, and also your Azure Functions apps.

Things to know about app security with App Service

- The authentication and authorization security module in Azure App Service runs in the same environment as your application code, yet separately.

- The security module is configured by using app settings. No SDKs, specific languages, or changes to your application code are required.
- The security module handles several tasks for your app:
 - Authenticate users with the specified provider
 - Validate, store, and refresh tokens
 - Manage the authenticated session
 - Inject identity information into request headers

Things to consider when using App Service for app security

You configure authentication and authorization security in App Service by selecting features in the Azure portal.

- **Allow Anonymous requests (no action).** Defer authorization of unauthenticated traffic to your application code. For authenticated requests, App Service also passes along authentication information in the HTTP headers. This feature provides more flexibility for handling anonymous requests. With this feature, you can present multiple sign-in providers to your users.
- **Allow only authenticated requests.** Redirect all anonymous requests to `/.auth/login/<provider>` for the provider you choose. The feature is equivalent to **Log in with provider****. If the anonymous request comes from a native mobile app, the returned response is an `HTTP 401 Unauthorized` message. With this feature, you don't need to write any authentication code in your app.
- **Logging and tracing.** View authentication and authorization traces directly in your log files. If you see an authentication error that you didn't expect, you can conveniently find all the details by looking in your existing application logs. If you enable failed request tracing, you can see exactly how the security module participated in a failed request. In the trace logs, look for references to a module named `EasyAuthModule_32/64`.

Create custom domain names

When you create a web app, Azure assigns the app to a subdomain of `azurewebsites.net`. Suppose your web app is named `contoso`. Azure creates a URL for your web app as `contoso.azurewebsites.net`. Azure also assigns a virtual IP address for your app.

Steps to configure a custom domain name for your app

Creating a custom domain name requires providers, security, and naming information.

There are three steps to create a custom domain name.

1. **Reserve your domain name.** The easiest way to set up a custom domain is to buy one directly in the Azure portal. (This name isn't the Azure assigned name of `*.azurewebsites.net`.) The registration process enables you to manage your web app's domain name directly in the Azure portal instead of going to a third-party site. Configuring the domain name in your web app is also a simple process in the Azure portal.
2. **Create DNS records to map the domain to your Azure web app.** The Domain Name System (DNS) uses data records to map domain names to IP addresses. There are several types of DNS records.
 - For web apps, you create either an `A` (Address) record or a `CNAME` (Canonical Name) record.
 - An `A` record maps a domain name to an IP address.
 - A `CNAME` record maps a domain name to another domain name. DNS uses the second name to look up the address. Users still see the first domain name in their

browser. As an example, you could map `contoso.com` to your `webapp.azurewebsites.net` URL.

- If the IP address changes, a `CNAME` entry is still valid, whereas an `A` record must be updated.
- Some domain registrars don't allow `CNAME` records for the root domain or for wildcard domains. In such cases, you must use an `A` record.

3. **Enable the custom domain.** After you have your domain and create your DNS record, use the Azure portal to validate your custom domain and add it to your web app. Be sure to test your domain before publishing.

Back up and restore your App Service app

The [Backup and Restore feature](#) in Azure App Service lets you easily create backups manually or on a schedule. You can configure the backups to be retained for a specific or indefinite amount of time. You can restore your app or site to a snapshot of a previous state by overwriting the existing content or restoring to another app or site.

Things to know about Backup and Restore

- Back up and restore is supported in the Basic, Standard, Premium, and Isolated tiers. *For the Basic tier, you can only back up and restore the production slot.*
- You need an Azure storage account and container in the same subscription as the app to back up.
- Azure App Service can back up the following information to the Azure storage account and container you configured for your app:
 - App configuration settings
 - File content
 - Any database connected to your app (SQL Database, Azure Database for MySQL, Azure Database for PostgreSQL, MySQL in-app)
- In your storage account, each backup consists of a Zip file and XML file:
 - The Zip file contains the back-up data for your app or site.
 - The XML file contains a manifest of the Zip file contents.
- You can configure backups manually or on a schedule.
- Full backups are the default.
- Partial backups are supported. You can specify files and folders to exclude from a backup.
- You restore partial backups of your app or site the same way you restore a regular backup.
- Backups can hold up to 10 GB of app and database content.
- Backups for your app or site are visible on the **Containers** page of your storage account and app (or site) in the Azure portal.

Things to consider when creating backups and restoring backups

- **Consider full backups.** Do a full backup to easily save all configuration settings, all file content, and all database content connected with your app or site. When you restore a full backup, all content on the site is replaced with whatever is in the backup. If a file is on the site, but not in the backup, the file is deleted.
- **Consider partial backups.** Specify a partial backup so you can choose exactly which files to back up. When you restore a partial backup, any content located in an excluded folder or file is left as-is.
- **Consider browsing back-up files.** Unzip and browse the Zip and XML files associated with your backup to access your backups. This option lets you view the content without actually

performing an app or site restore.

- **Consider firewall on back-up destination.** If your storage account is enabled with a firewall, you can't use the storage account as the destination for your backups.

Use Azure Application Insights

[Azure Application Insights](#) is a feature of Azure Monitor that lets you monitor your live applications. You can integrate Application Insights with your App Service configure to automatically detect performance anomalies in your apps.

Things to know about Application Insights

- Application Insights works on various platforms including .NET, Node.js, and Java EE.
- The feature can be used for configurations that are hosted on-premises, in a hybrid environment, or in any public cloud.
- Application Insights integrates with your Azure Pipeline processes, and has connection points to many development tools.

Things to consider when using Application Insights

Consider monitoring the following items in your App Service configuration scenario.

- **Consider Request rates, response times, and failure rates.** Find out which pages are most popular, at what times of day, and where your users are. See which pages perform best. If your response times and failure rates go high when there are more requests, then perhaps you have a resourcing problem.
- **Consider Dependency rates, response times, and failure rates.** Use Application Insights to discover if external services are degrading your app performance.
- **Consider Exceptions.** Analyze the aggregated statistics, or pick specific instances and drill into the stack trace and related requests. Both server and browser exceptions are reported.
- **Consider Page views and load performance.** Collect the number of page views reported by your users' browsers and analyze the load performance.
- **Consider User and session counts.** Application Insights can help you keep track of the number of users and sessions connected to your app.
- **Consider Performance counters.** Add Application Insights performance counters from your Windows or Linux server machines. Monitor performance output for the CPU, memory, network usage, and so on.
- **Consider Host diagnostics.** Integrate diagnostics from Docker or Azure into your app Application Insights.
- **Consider Diagnostic trace logs.** Implement trace logs from your app to help correlate trace events with requests and diagnose issues.
- **Consider Custom events and metrics.** Write your own custom events and metric tracking algorithms as client or server code. Track business events such as number of items sold, or number of games won.

The main takeaways for this module

- Azure App Service lets you develop and deploy web, mobile, and API apps.
- Azure App Service configuration settings include runtime stack, operating system, region, and App Service plan.
- Deployment slots help you manage different app stages. For example, development, test, stage, and production.
- The default Azure App Service domain name can be customized for your organization.

- Azure Application Insights is a feature of Azure Monitor that lets you monitor your live applications. You can integrate Application Insights with your App Service configure to automatically detect performance anomalies in your apps.
- Application Insights lets you continuously monitor the performance and usability of your apps.

5 - Configure Azure Container Instances

Containers and virtual machines are both forms of virtualization, but there are some key differences between them.

Compare containers to virtual machines

Hardware virtualization makes it possible to run multiple isolated instances of operating systems concurrently on the same physical hardware. Containers represent the next stage in the virtualization of computing resources.

Container-based virtualization allows you to virtualize the operating system. This approach lets you run multiple applications within the same instance of an operating system, while maintaining isolation between the applications.

Things to know about containers versus virtual machines

Compare	Containers	Virtual machines
Isolation	A container typically provides lightweight isolation from the host and other containers, but a container doesn't provide as strong a security boundary as a virtual machine.	A virtual machine provides complete isolation from the host operating system and other virtual machines. This separation is useful when a strong security boundary is critical, such as hosting apps from competing companies on the same server or cluster.
Operating system	Containers run the user mode portion of an operating system and can be tailored to contain just the needed services for your app. This approach helps you use fewer system resources.	Virtual machines run a complete operating system including the kernel, which requires more system resources (CPU, memory, and storage).
Deployment	You can deploy individual containers by using Docker via the command line. You can deploy multiple containers by using an orchestrator such as Azure Kubernetes Service.	You can deploy individual virtual machines by using Windows Admin Center or Hyper-V Manager. You can deploy multiple virtual machines by using PowerShell or System Center Virtual Machine Manager.
Persistent storage	Containers use Azure Disks for local storage for a single node, or Azure Files (SMB shares) for storage shared by multiple nodes or servers.	Virtual machines use a virtual hard disk (VHD) for local storage for a single machine, or an SMB file share for storage shared by multiple servers.
Fault tolerance	If a cluster node fails, the orchestrator on another cluster node rapidly recreates any containers running on the node.	Virtual machines can fail over to another server in a cluster, where the virtual machine's operating system restarts on the new server.

Things to consider when using containers

- **Consider flexibility and speed.** Gain increased flexibility and speed when developing and sharing your containerized application code.
- **Consider testing.** Choose containers for your configuration to allow for simplified testing of your apps.
- **Consider app deployment.** Implement containers to gain streamlined and accelerated deployment of your apps.
- **Consider workload density.** Support higher workload density and improve your resource utilization by working with containers.

Review Azure Container Instances

Containers are becoming the preferred way to package, deploy, and manage cloud applications. There are many options for teams to build and deploy cloud native and containerized applications on Azure.

Azure Container Instances offers the fastest and simplest way to run a container in Azure, without having to manage any virtual machines and without having to adopt a higher-level service. Azure Container Instances is a great solution for any scenario that can operate in isolated containers.

Understand container images

All containers are created from container images. A container image is a lightweight, standalone, executable package of software that encapsulates everything needed to run an application.

It includes the following components:

- **Code:** The application's source code.
- **Runtime:** The environment required to execute the application.
- **System tools:** Utilities necessary for the application to function.
- **System libraries:** Shared libraries used by the application.
- **Settings:** Configuration parameters specific to the application.

Things to know about Azure Container Instances

- **Fast startup times.** Containers can start in seconds without the need to deploy and manage virtual machines.
- **Public IP connectivity and DNS names.** Containers can be directly exposed to the internet with an IP address and FQDN (fully qualified domain name).
- **Custom sizes.** You specify CPU cores (from 0.1 to 4 vCPU) and memory (from 0.1 to 16 GB) for each container at deployment time. Resource allocation is fixed for the lifetime of the container group.
- **Persistent storage.** Containers support direct mounting of Azure Files file shares.
- **Linux and Windows containers.** Container Instances can schedule both Windows and Linux containers. Specify the operating system type when you create your container groups.
- **Coscheduled groups.** Container Instances supports scheduling of multi-container groups that share host machine resources.
- **Virtual network deployment.** Linux container groups can be deployed into an Azure virtual network for private communication with other Azure resources. Virtual network deployed containers receive no public IP address and communicate only within the virtual network or peered networks.

Implement container groups

The top-level resource in Azure Container Instances is the **container group**. A [container group](#) is a collection of containers that get scheduled on the same host machine. The containers share a lifecycle, resources, local network, and storage volumes.

Things to know about container groups

- A container group is similar to a pod in Kubernetes. A pod typically has a 1:1 mapping with a container, but a pod can contain multiple containers. The containers in a multi-container pod can share related resources.
- Azure Container Instances allocates resources to a multi-container group by adding together the resource requests of all containers in the group. Resources can include items such as CPUs, memory, and GPUs.
- There are three common ways to deploy a multi-container group.
 - **Azure Resource Manager template.** JSON-based infrastructure as code, ideal when deploying alongside other Azure resources.
 - **Bicep.** Microsoft's recommended infrastructure as code language, more concise than Azure Resource Manager templates. Bicep includes full IntelliSense support.
 - **YAML files.** Container-focused format, ideal for deployments that include only container instances.
- Container groups can share an external-facing IP address, one or more ports on the IP address, and a DNS label with an FQDN.
 - **External client access.** You must expose the port on the IP address and from the container to enable external clients to reach a container in your group.
 - **Port mapping.** Port mapping isn't supported because containers in a group share a port namespace.
 - **Deleted groups.** When a container group is deleted, its IP address and FQDN are released.

The multi-container group has the following characteristics and configuration:

- The container group is scheduled on a single host machine, and is assigned a DNS name label.
- The container group exposes a single public IP address with one exposed port.
- One container in the group listens on port 80. The other container listens on port 1433.
- The group includes two Azure Files file shares as volume mounts. Each container in the group mounts one of the file shares locally.

Things to consider when using container groups

Multi-container groups are useful when you want to divide a single functional task into a few container images. Different teams can deliver the images, and the images can have separate resource requirements.

- **Consider web app updates.** Support updates to your web apps by implementing a multi-container group. One container in the group serves the web app and another container pulls the latest content from source control.
- **Consider log data collection.** Use a multi-container group to capture logging and metrics data about your app. Your application container outputs logs and metrics. A logging container collects the output data and writes the data to long-term storage.
- **Consider app monitoring.** Enable monitoring for your app with a multi-container group. A monitoring container periodically makes a request to your application container to ensure your app is running and responding correctly. The monitoring container raises an alert if it identifies possible issues with your app.

- **Consider front-end and back-end support.** Create a multi-container group to hold your front-end container and back-end container. The front-end container can serve a web app. The back-end container can run a service to retrieve data.

Review Azure Container Apps

There are many options for teams to build and deploy cloud native and containerized applications on Azure. Let's

Things to know about Azure Container Apps

[Azure Container Apps](#) is a serverless platform that allows you to maintain less infrastructure and save costs while running containerized applications. Instead of worrying about server configuration, container orchestration, and deployment details, Container Apps provides all the up-to-date server resources required to keep your applications stable and secure.

Common uses of Azure Container Apps include:

- Deploying API endpoints
- Hosting background processing jobs
- Handling event-driven processing
- Running microservices

Applications built on Azure Container Apps can dynamically scale based on the following characteristics:

- HTTP traffic
- Event-driven processing
- CPU or memory load
- Any KEDA-supported scaler

Things to consider when using Azure Container Apps

- Optimized for running general purpose containers, especially for applications that span many microservices deployed in containers.
- Powered by Kubernetes and open-source technologies like Dapr, KEDA, and envoy.
- Supports Kubernetes-style apps and microservices with features like service discovery and traffic splitting.
- Enables event-driven application architectures by supporting scale based on traffic and pulling from event sources like queues, including scale to zero.
- Supports running on demand, scheduled, and event-driven jobs.

Compare container management solutions

Azure offers several container platforms for different scenarios. Azure Container Instances (ACI) is best for isolated, short-lived tasks. Azure Container Apps (ACA) serves serverless microservices. Azure Kubernetes Service (AKS) provides full Kubernetes control for complex orchestration needs.

Feature	Azure Container Apps (ACA)	Azure Kubernetes Service (AKS)
Overview	ACA is a serverless container platform that simplifies the deployment and management of microservices-based applications by abstracting away the underlying infrastructure.	AKS simplifies deploying a managed Kubernetes cluster in Azure by offloading the operational overhead to Azure. It’s suitable for complex applications that require orchestration.

Feature	Azure Container Apps (ACA)	Azure Kubernetes Service (AKS)
Deployment	ACA provides a PaaS experience with quick deployment and management capabilities.	AKS offers more control and customization options for Kubernetes environments, making it suitable for complex applications and microservices.
Management	ACA builds upon AKS and offers a simplified PaaS experience for running containers.	AKS provides a more granular control over the Kubernetes environment, suitable for teams with Kubernetes expertise.
Scalability	ACA supports both HTTP-based autoscaling and event-driven scaling, making it ideal for applications that need to respond quickly to changes in demand.	AKS offers horizontal pod autoscaling and cluster autoscaling, providing robust scalability options for containerized applications.
Use Cases	ACA is designed for microservices and serverless applications that benefit from rapid scaling and simplified management.	AKS is best for complex, long-running applications. These applications require full Kubernetes features and tight integration with other Azure services.

1 - Introduction to Azure Backup

Azure Backup can also address scenarios for your Azure environments, with support for:

- Azure VMs
- Azure Managed Disks
- Azure Files
- SQL Server in Azure VMs
- SAP HANA databases in Azure VMs
- Azure Database for PostgreSQL servers
- Azure Blobs
- Azure Database for PostgreSQL - Flexible servers
- Azure Database for MySQL - Flexible servers
- Azure Kubernetes cluster

What is Azure Backup?

The Azure Backup service provides simple, secure, and cost-effective solutions to back up your data and recover it from the Microsoft Azure cloud.

Azure Backup definition

Azure Backup is an Azure service that provides cost effective, secure, and zero-infrastructure backup solutions for all Azure-managed data assets.

When to use Azure Backup

Azure Backup can provide backup services for the following data assets:

- On-premises files, folders, and system state
- Azure Virtual Machines (VMs)
- Azure Managed Disks
- Azure Files Shares

- SQL Server in Azure VMs
- SAP HANA (High-performance Analytic Appliance) databases in Azure VMs
- Azure Database for PostgreSQL servers
- Azure Blobs
- Azure Database for PostgreSQL - Flexible servers
- Azure Database for MySQL - Flexible servers
- Azure Kubernetes cluster

Key features

Feature	Description	Usage
Zero-infrastructure backup solution	Unlike conventional backup solutions, no backup server or infrastructure is needed. Similarly, no backup storage needs to be deployed, because Azure Backup automatically manages and scales it.	Zero-infrastructure solution eliminates capital expenses and reduces operational expenses. It increases ease of use by automating storage management.
At-scale management	Natively manage your entire backup estate from a central console called Backup Center. Use APIs, PowerShell, and Azure CLI to automate Backup policy and security configurations.	Backup center simplifies data protection management at-scale by enabling you to discover, govern, monitor, operate, and optimize backup management from one unified console, which helps you to drive operational efficiency with Azure.
Security	Azure Backup provides built-in security to your backup environment, both when your data is in transit and at rest by using capabilities encryption, private endpoints, alerts, and so on.	Your backups are automatically secured against ransomware, malicious admins, and accidental deletions.

Recovery Time Objective (RTO) is the target time within which a business process must be restored after a disaster occurs to avoid unacceptable consequences.

Recovery Point Objective (RPO) is the maximum amount of data loss, measured in time, that your organization can sustain during an event.

How Azure Backup works

- **Workload integration layer - Backup Extension:** Integration with the actual workload, such as Azure virtual machines (VMs) or Azure Blobs, happens at this layer.
- **Data Plane - Access Tiers:** There are three access tiers where the backups could be stored:
 - Snapshot tier
 - Standard tier
 - Archive tier
- **Data Plane - Availability and Security:** The backup data is replicated across zones or regions, based on the redundancy the user specifies.
- **Management Plane – Recovery Services vault/Backup vault and Backup center:** The vault provides an interface for the user to interact with the backup service.

What data is backed up and how?

The simplest explanation of Azure Backup is that it backs up data, machine state, and workloads running on on-premises machines and VM instances to the Azure cloud. Azure Backup stores the backed-up data in Recovery Services vaults and Backup vaults.

For on-premises Windows machines, you can back up directly to Azure with the Azure Backup Microsoft Azure Recovery Services (MARS) agent. Alternatively, you can back up these Windows machines to a backup server, perhaps a System Center Data Protection Manager (DPM) or Microsoft Azure Backup Server (MABS).

Supported backup types

Azure Backup supports full backups and incremental backups. Your initial backup is a full backup. DPM/MABS use the incremental backup for disk backups, and all backups to Azure also use incremental backups.

The following table outlines the support for SQL Server type backups:

Type	Description	Usage
Full	A full database backup backs up the entire database. It contains all the data in a specific database or in a set of filegroups or files. A full backup also contains enough logs to recover that data.	At most, you can trigger one full backup per day. You can choose to make a full backup on a daily or weekly interval.
Differential	A differential backup is based on the most recent full-data backup. It captures only the data that changed since the full backup.	At most, you can trigger one differential backup per day. You can't configure a full backup and a differential backup on the same day.
Multiple backups per day	Back up Azure VMs hourly with a minimum recovery point objective (RPO) of 4 hours and a maximum of 24 hours.	You can use Enhanced backup policy to set the backup schedule to 4, 6, 8, 12, and 24 hours (respectively) for new Azure offerings, such as Trusted Launch VM.
Selective disk backup	Selectively back up a subset of the data disks that are attached to your VM, then restore a subset of the disks that are available in a recovery point, both from instant restore and vault tier. Selective disk backup helps you manage critical data in a subset of the VM disks and use database backup solutions when you want to back up only their OS disk to reduce cost.	Azure Backup provides Selective Disk backup and restore capability using Enhanced backup policy.
Transaction Log	A log backup enables point-in-time restoration up to a specific second.	At most, you can configure transactional log backups every 15 minutes.

Data Plane - Access Tiers

There are three access tiers where the backups can be stored:

- Snapshot tier:** (Workload-specific term) In the first phase of a virtual machine backup, the snapshot is taken and stored along with the disk. This form of storage is referred to as a snapshot tier. Restoring a snapshot tier is faster than restoring from a vault, because it eliminates the wait time for snapshots to be copied from the vault before triggering the restore operation. The snapshots of the VM/Azure Files/Azure Blobs/and so on are retained in

the customer's subscription in a specified resource group. This container ensures that restores are quick, because the backup/snapshot is available locally to the customer.

- **Vault-standard tier:** Backup data for all workloads supported by Azure Backup is stored in vaults, which hold backup storage, an autoscaling set of storage accounts managed by Azure Backup. The Vault-standard tier is an online storage tier that allows you to store an isolated copy of backup data in a Microsoft-managed tenant, thus creating an extra layer of protection. For workloads where snapshot tier is supported, there's a copy of the backup data in both the snapshot tier and the Vault-standard tier. The Vault-standard tier ensures that backup data is available even if the data source being backed up is deleted or compromised.
- **Archive tier:** Customers rely on Azure Backup for storing backup data, including their Long-Term Retention (LTR) backup data, with retention needs defined in the organization's compliance rules. In most cases, the older backup data is rarely accessed and is only stored for compliance needs.

Data Plane - Availability and Security

The backup data is replicated across zones or regions, based on the redundancy you specify. You can choose from locally redundant storage (LRS), Geo-redundant storage (GRS), or zone-redundant storage (ZRS).

The data is kept safe by encrypting it and implementing Azure role-based access control (RBAC). You choose who can perform backup and restore operations. Azure Backup also provides protection against malicious deletion of your backup by using soft-delete operations. A deleted backup is stored for 14 days, free of charge, which allows you to recover the backup if needed.

Management Plane – Recovery Services vault/Backup vault and Backup center

Azure Backup uses Recovery Services vaults and Backup vaults to orchestrate and manage backups. It also uses vaults to store backed-up data. The vault provides an interface for the user to interact with the backup service. Azure Backup Policies within each vault define when the backups should get triggered and how long they need to be retained.

When to use Azure Backup

The table contains some key areas where Azure Backup can provide services to you for data protection.

Criteria	Consideration
Azure workloads	Azure VMs, Azure Disks, SQL Server in Azure VMs, SAP HANA databases in Azure VMs, Azure Blobs, Azure Files shares, Azure Database for PostgreSQL.
Compliance	Customer-defined backup policy with long-term retention across multiple zones or regions.
Operational recoveries	With self-service backup and restores, the application administrator can take care of issues that might arise such as accidental deletion or data corruption.

Additionally, Azure Backup provides the following advantages specifically for SQL Server:

- Workload aware backups that support all backup types: full, differential, and log
- 15-minute recovery point objective (RPO) with frequent log backups
- Point-in-time recovery up to a second
- Individual database-level backup and restore

Does Azure Backup help with compliance?

You can implement required access-control mechanisms for your backups. Vaults (Recovery Services and Backup vaults) provide the management capabilities and are accessible via the Azure portal, Backup Center, Vault dashboards, SDK, CLI, and even REST APIs. It's also an Azure role-based access control (Azure RBAC) boundary, providing you with the option to restrict access to backups only to authorized Backup Admins.

Long-term retention can be:

- **Planned (compliance requirements):** If you know in advance that data is required years from the current time, use Long-term retention.
- **Unplanned (on-demand requirement):** If you don't know in advance, then you can use on-demand backup with specific custom retention settings. Your policy settings don't impact these custom retention settings.
- **On-demand backup with custom retention:** If you need to take a backup not scheduled via backup policy, then you can use an on-demand backup. It can be useful for taking backups that don't fit your scheduled backup or for taking granular backup (for example, multiple IaaS VM backups per day since scheduled backup permits only one backup per day). It's important to note that the retention policy defined in scheduled policy doesn't apply to on-demand backups.

2 - Protect your virtual machines by using Azure Backup

What is Azure Backup?

Azure Backup is a built-in Azure service that provides secure backup for all Azure-managed data assets. It uses zero-infrastructure solutions to enable self-service backups and restores, with at-scale management at a lower and predictable cost. Azure Backup currently offers specialized backup solutions for Azure and on-premises virtual machines (VMs). Azure Backup also gives workloads like SQL Server or SAP HANA (High-performance Analytic Appliance) running in Azure VMs enterprise-class backup and restore options.

Backups are used in cases of accidental data loss, data corruption, or ransomware attacks.

Additionally, the choice of a recovery approach depends on the criticality of the application, recovery point objective (RPO) and recovery time objective (RTO) requirements, and the cost implications.

Why use Azure Backup?

Azure Backup has several benefits over more traditional backup solutions:

Zero-infrastructure backup: Azure Backup eliminates the need to deploy and manage any backup infrastructure or storage. There's no overhead in maintaining backup servers or scaling the storage up or down as the needs vary.

Long-term retention: Meet rigorous compliance and audit needs by retaining backups for many years, after which the built-in lifecycle management capability prunes the recovery points automatically.

Security: Azure Backup provides security to your backup environment, both when your data is in transit and at rest:

- **Azure role-based access control:** Role-based access control allows you to segregate duties within your team and grant only the amount of access to users necessary to do their jobs.
- **Encryption of backups:** Backup data is automatically encrypted using Microsoft-managed keys. Alternatively, you can encrypt your backed-up data using customer-managed keys

stored in the Azure Key Vault.

- **No internet connectivity required:** When you use Azure VMs, all the data transfer happens only on the Azure backbone network without needing to access your virtual network. So no access to any IPs or fully qualified domain names (FQDNs) is required.
- **Soft delete:** With soft delete, the backup data is retained for 14 more days even after the deletion of the backup item. This retention protects against accidental deletion or malicious deletion scenarios, allowing the recovery of those backups with no data loss. Azure Backup also provides **Enhanced soft delete** that enables you to retain a deleted item in the *soft deleted* state for a longer duration.

Azure Backup also offers the ability to back up VMs encrypted with Azure Disk Encryption.

High availability: Azure Backup offers three types of replication:

- **Locally redundant storage (LRS):** The lowest-cost option with basic protection against server rack and drive failures. We recommend it for noncritical scenarios.
- **Geo-redundant storage (GRS):** The intermediate option has failover capabilities in a secondary region. We recommend it for backup scenarios.
- **Zone-redundant storage (ZRS):** This option protects against datacenter-level failures by replicating your storage account synchronously across three Azure availability zones. We recommend it for high-availability scenarios.

Azure Backup supported scenarios

Azure Backup supports the following scenarios:

- **Azure VMs** - Back up Windows or Linux Azure VMs
Azure Backup provides independent and isolated backups to guard against unintended destruction of the data on your VMs. Backups are stored in a Recovery Services vault with built-in management of recovery points. Configuration and scaling are simple, backups are optimized, and you can easily restore as needed.
- **On-premises** - Back up files, folders, and system state using the [Microsoft Azure Recovery Services \(MARS\) agent](#). Or use [Microsoft Azure Backup Server \(MABS\)](#) or [Data Protection Manager \(DPM\) server](#) to protect on-premises VMs (Hyper-V and VMware) and other on-premises workloads.
- **Azure Files shares** - Azure Files provides snapshot management by Azure Backup.
- **SQL Server in Azure VMs and SAP HANA databases in Azure VMs** - Azure Backup offers stream-based, specialized solutions to back up SQL Server, or SAP HANA running in Azure VMs. These solutions take workload-aware backups that support different backup types such as full, differential and log, 15-minute RPO, and point-in-time recovery.

Back up an Azure virtual machine by using Azure Backup

Azure VMs are backed up by taking snapshots of the underlying disks at user-defined intervals and transferring those snapshots to the Recovery Services vault as per the customer-defined policy.

Recovery Services vault

Azure Backup uses a Recovery Services vault to manage and store the backup data. A vault is a storage-management entity, which provides a simple experience to carry out and monitor backup and restore operations. With Azure Backup, you don't need to worry about deploying or managing storage accounts.

Snapshots

A snapshot is a point-in-time backup of all disks on the VM. For Azure VMs, Azure Backup uses different extensions for each supporting operating system:

Extension	OS	Description
VM Snapshot	Windows	The extension works with Volume Shadow Copy Service (VSS) to take a copy of the data on disk and in memory.
VM SnapshotLinux	Linux	The snapshot is a copy of the disk.

Depending on how the snapshot is taken and what it includes, you can achieve different levels of consistency:

- **Application consistent**
 - The snapshot captures the VM as a whole. It uses VSS writers to capture the content of the machine memory and any pending I/O operations.
 - For Linux machines, you need to write custom pre or post scripts per app to capture the application state.
 - You can get complete consistency for the VM and all running applications.
- **File system consistent**
 - If VSS fails on Windows, or the pre and post scripts fail on Linux, Azure Backup still creates a file-system-consistent snapshot.
 - During a recovery, no corruption occurs within the machine. But installed applications need to do their own cleanup during startup to become consistent.
- **Crash consistent**
 - This level of consistency typically occurs if the VM is shut down at the time of the backup.
 - No I/O operations or memory contents are captured during this type of backup. This method doesn't guarantee data consistency for the OS or app.

Backup policy

The backup policy supports two access tiers: *snapshot tier* and the *vault tier*. By using the Enhanced policy, you can trigger hourly backups.

Selective disk backup: Azure Backup provides **Selective Disk backup and restore** capability using **Enhanced policy**. By using this capability, you can selectively back up a subset of the data disks that are attached to your VM. Then, you can restore a subset of the disks that are available in a recovery point, both from instant restore and vault tier. It helps you manage critical data in a subset of the VM disks and use database backup solutions when you want to back up only their OS disk to reduce cost.

Snapshot tier: All the snapshots are stored locally for a maximum period of five days, in what is called the snapshot tier. For all types of operation recoveries, we recommended that you restore from the snapshots because it's faster to do so. This capability is called **instant restore**.

Vault tier: All snapshots are additionally transferred to the vault for more security and longer retention. At this point, the recovery point type changes to "snapshot and vault."

Backup process for an Azure virtual machine

Here's how Azure Backup completes a backup for Azure VMs:

1. For Azure VMs that are selected for backup, Azure Backup starts a backup job according to the backup frequency you specify in the backup policy.
2. During the first backup, a backup extension is installed on the VM, if the VM is running:

- For Windows VMs, the VM Snapshot extension is installed.
 - For Linux VMs, the VM SnapshotLinux extension is installed.
3. After the snapshot is taken, the data is stored locally and transferred to the vault.
- The backup is optimized by backing up each VM disk in parallel.
 - For each disk that's being backed up, Azure Backup reads the blocks on the disk and identifies and transfers only the data blocks that changed (the delta) since the previous backup.
 - Snapshot data might not be immediately copied to the vault. It might take several hours at peak times. Total backup time for a VM is less than 24 hours for daily backup policies.

Restore virtual machine data

Companies that have a business continuity and disaster recovery (BCDR) plan typically schedule test runs to ensure that the business can successfully recover from disasters.

Restore types

Azure Backup provides many ways to restore a VM. As explained earlier, you can either instantly restore from the snapshot tier (optimal for operational recoveries) or from the vault tier.

Restore option	Details
Create a new VM	Quickly creates and gets a basic VM up and running from a restore point. The new VM must be created in the same region as the source VM.
Restore disk	Restores a VM disk, which can then be used to create a new VM. The disks are copied to the resource group you specify. Azure Backup provides a template to help you customize and create a VM. Alternatively, you can attach the disk to an existing VM, or create a new VM. This option is useful if you want to customize the VM, add configuration settings that weren't there at the time of backup. Or, add settings that must be configured using the template or PowerShell.
Replace existing	You can restore a disk and use it to replace a disk on the existing VM. Azure Backup takes a snapshot of the existing VM before replacing the disk and stores it in the staging location you specify. Existing disks connected to the VM are replaced with the selected restore point. The current VM must exist. You can't use this option if the VM is deleted.
Cross region (secondary region)	Cross region restore can be used to restore Azure VMs in the secondary region, which is an Azure paired region. This feature is available for the following options: - Create a VM - Restore Disks We don't currently support the Replace existing disks option.

Restore option	Details
Cross Subscription Restore	Backup Admins and App admins can perform the restore operation on secondary regions. Cross Subscription Restore: - Allows you to restore Azure Virtual Machines or disks to a different subscription within the same tenant as the source subscription. As per the Azure role-based access control capabilities from restore points. - Allowed only if the Cross Subscription Restore property is enabled for your Recovery Services vault. - Works with Cross Region Restore and Cross Zonal Restore. - You can trigger Cross Subscription Restore for managed virtual machines only. - Cross Subscription Restore is supported for Restore with Managed System Identities (MSI). - It's unsupported for snapshots tier recovery points. - It's unsupported for unmanaged VMs and VMs encrypted with Advanced Digital Encryption (ADE).
Cross Zonal Restore	Allows you to restore Azure Virtual Machines or disks pinned to any zone to different available zones (as per the Azure Role-based access control capabilities) from restore points. When you select a zone to restore, it selects the logical zone (and not the physical zone) as per the Azure subscription you use to restore to. - You can trigger Cross Zonal Restore for managed virtual machines only. - Cross Zonal Restore is supported for Restore with Managed System Identities (MSI). - Cross Zonal Restore supports restore of an Azure zone pinned/non-zone pinned VM from a vault with Zone-redundant storage (ZRS) enabled. Learn how to set Storage Redundancy. - You can only use Cross Zonal Restore to restore a VM pinned to an Azure zone from a vault with Cross Region Restore (CRR) under these conditions: The secondary region supports zones, or Zone Redundant Storage (ZRS) is enabled. - Cross Zonal Restore is supported from secondary regions. - It's unsupported from snapshots restore point. - It's unsupported for Encrypted Azure VMs.
Selective disk backup	Allows you to back up and restore selective VM disks through Enhanced policy. Using this capability, you can selectively back up a subset of the data disks that are attached to your VM. Then, you can restore a subset of the disks that are available in a recovery point, both from instant restore and vault tier. Selective disk backup is useful when you: - Manage critical data in a subset of the VM disks. - Use database backup solutions and want to back up only their OS disk to reduce cost.

Restore an encrypted virtual machine

Azure Backup supports the backup and restore of machines encrypted through Azure Disk Encryption. Disk Encryption works with Azure Key Vault to manage the relevant secrets that are associated with the encrypted disk. For an extra layer of security, you can use key vault encryption keys (KEKs) to encrypt the secrets before they're written to the key vault.

Certain limitations apply when you restore encrypted VMs:

- Azure Backup supports only standalone key encryption. Any key that's part of a certificate isn't currently supported.
- File-level or folder-level restores aren't supported with encrypted VMs. To restore to that level of granularity, the entire VM has to be restored. You can then manually copy the file or folders.
- The **Replace existing VM** option isn't available for encrypted VMs.

Restore Azure virtual machine data

A backup can't be restored if the VM is allocated and running.

3 - Monitor your Azure virtual machines with Azure Monitor

Azure Monitor provides built-in and customizable monitoring abilities. You can use these to track the health, performance, and behavior of the VM host and the operating system, workloads, and applications running on your VM

Monitoring for Azure VMs

Azure Monitor is a comprehensive monitoring solution for collecting, analyzing, and responding to monitoring data from Azure and non-Azure resources, including VMs. Azure Monitor has two main monitoring features: Azure Monitor Metrics and Azure Monitor Logs.

Metrics are numerical values collected at predetermined intervals to describe some aspect of a system. Metrics can measure VM performance, resource utilization, error counts, user responses, or any other aspect of the system that you can quantify. Azure Monitor Metrics automatically monitors a predefined set of metrics for every Azure VM, and retains the data for 93 days with some exceptions.

Logs are recorded system events that contain a timestamp and different types of structured or free-form data. Azure automatically records activity logs for all Azure resources. This data is available at the resource level. Azure Monitor doesn't collect logs by default, but you can configure Azure Monitor Logs to collect from any Azure resource. Azure Monitor Logs stores log data in a Log Analytics workspace for querying and analysis.

Azure VMs have several layers that require monitoring. Each of the following layers has a distinct set of telemetry and monitoring requirements.

- Host VM
- Guest operating system (OS)
- Client workloads
- Applications that run on the VM

The VM host represents the compute, storage, and network resources that Azure allocates to the VM.

VM host metrics

VM host metrics measure technical aspects of the VM such as processor utilization and whether the machine is running. You can use VM host metrics to:

- Trigger an alert when your VM is reaching its disk or CPU limits.
- Identify trends or patterns.
- Control your operational costs by sizing VMs according to usage and demand.

You can use Azure Monitor Metrics Explorer to plot more metrics graphs, investigate changes, and visually correlate metrics trends for your VMs.

With Metrics Explorer you can:

- Plot multiple metrics on a graph to see how much traffic hits your VM and how the VM performs.
- Track the same metric over multiple VMs in a resource group or other scope, and use splitting to show each VM on the graph.
- Select flexible time ranges and granularity.
- Specify many other settings such as chart type and value ranges.
- Send graphs to workbooks or pin them to dashboards for quickly viewing health and performance.
- Group metrics by time intervals, geographic regions, server clusters, or application components.

Recommended alert rules

Alerts proactively notify you of specified occurrences and patterns in your VM host metrics. *Recommended alert rules* are a predefined set of alert rules based on commonly monitored host metrics. These rules define recommended CPU, memory, disk, and network usage levels to alert on. The rules also include VM availability, which alerts you when the VM stops running.

You can quickly enable and configure recommended alert rules when you create an Azure VM, or afterwards from the VM's portal page. You can also view, configure, and create custom alerts by using Azure Monitor Alerts.

Activity logs

You can create diagnostic settings to send activity logs to the following destinations:

- **Azure Monitor Logs:** For more complex querying and alerting, and for longer retention up to two years.
- **Azure Storage:** For cheaper, long-term archiving.
- **Azure Event Hubs:** To forward outside of Azure.

Boot diagnostics

Boot diagnostics are host logs you can use to help troubleshoot boot issues with your VMs. You can enable boot diagnostics by default when you create a VM, or afterwards for existing VMs.

Azure Monitor Metrics can store only metrics data, but Azure Monitor Logs can store both metrics and event logs.

VM insights

VM insights is an Azure Monitor feature that helps get you started monitoring your VM clients. VM insights is especially useful for exploring overall VM usage and performance when you don't yet know the metric of primary interest. VM insights provides:

- Simplified Azure Monitor Agent onboarding to enable monitoring a VM's guest OS and workloads.
- A preconfigured DCR that monitors and collects the most common performance counters for Windows and Linux.
- Predefined trending performance metrics charts and workbooks from the VM's guest OS.
- A set of predefined workbooks that show collected VM client metrics over time.

- Optionally, a collection of processes running on the VM, dependencies with other services, and a dependency map that displays interconnected components with other VMs and external sources.

Client event log data

VM insights creates a DCR that collects a specific set of performance counters. To collect other data, such as event logs, you can create a separate DCR that specifies the data you want to collect from the VM and where to send it. Azure Monitor stores collected log data in a Log Analytics workspace. From there, you can access and analyze the data by using log queries written in Kusto Query Language (KQL).

Monitor VM host data

Use Metrics Explorer to view detailed host metrics

Azure Monitor Metrics Explorer offers a UI for exploring and analyzing VM metrics. You can use Metrics Explorer to view and create custom charts for many VM host metrics in addition to the metrics shown on the built-in graphs.

Understand Metrics Explorer

To open Metrics Explorer, you can:

- Select **Metrics** from the VM's left navigation menu under **Monitoring**.
- Select the **See all Metrics** link next to **Platform metrics** on the **Monitoring** tab of the VM's **Overview** page.
- Select **Metrics** from the left navigation menu on the Azure Monitor **Overview** page.

In Metrics Explorer, you can select the following values from the dropdown fields:

- **Scope:** If you open Metrics Explorer from a VM, this field is prepopulated with the VM name. You can add more items with the same resource type (VMs) and location.
- **Metric Namespace:** Most resource types have only one namespace, but for some types, you must pick a namespace. For example, storage accounts have separate namespaces for files, tables, blobs, and queues.
- **Metric:** Each metrics namespace has many metrics available to choose from.
- **Aggregation:** For each metric, Metrics Explorer applies a default aggregation. You can use a different aggregation to get different information about the metric.

You can apply the following aggregation functions to metrics:

- **Count:** Counts the number of data points.
- **Average (Avg):** Calculates the arithmetic mean of values.
- **Maximum (Max):** Identifies the highest value.
- **Minimum (Min):** Identifies the lowest value.
- **Sum:** Adds up all the values.

Create a metrics graph

To create a Metrics Explorer graph that shows host VM maximum percentage CPU and inbound flows together for the past 30 minutes:

1. Open **Metrics Explorer** by selecting **See all Metrics** on the VM's **Monitoring** tab or selecting **Metrics** from the VM's left navigation menu.

2. **Scope** and **Metric Namespace** are already populated for the host VM. Select **Percentage CPU** from the **Metrics** dropdown list.
3. **Aggregation** is automatically populated with **Avg**, but change it to **Max**.
4. Select **Add metric** at upper left.
5. Under **Metric**, select **Inbound Flows**. Leave **Aggregation** at **Avg**.
6. At upper right, select **Local Time: Last 24 hours (Automatic - 15 minutes)**, change it to **Last 30 minutes**, and select **Apply**.

Collect client performance counters by using VM insights

The VM client includes the operating system and other workloads and applications. To monitor the software running on your VM, you install the Azure Monitor Agent, which collects data from inside the VM. VM insights:

- Installs Azure Monitor Agent on your VM.
- Creates a DCR that collects and sends a predefined set of client performance data to a Log Analytics workspace.
- Presents the data in curated workbooks.

VM insights provides you with a basis for monitoring the performance of your VM client and mapping the processes running on your machine.

Enable VM insights

1. In the Azure portal, on your VM's **Overview** page, select **Insights** from the left navigation menu under **Monitoring**.
2. On the **Insights** page, select **Enable**.
3. Under **Data collection rule**, note the properties of the DCR that VM insights creates. In the DCR description, **Processes and dependencies (Map)** is set to **Disabled**, which you can change to **Enabled** or review [this article on how-to](#) if greyed out. Also a default **Log Analytics workspace** is created or assigned.
4. Select **Configure**.
5. Configuration of the workspace and the agent installation typically takes 5 to 10 minutes. It can take another 5 to 10 minutes for data to become available to view in the portal.
6. When the deployment finishes, confirm that the Azure Monitor Agent is installed by looking on the **Properties** tab of the VM's **Overview** page under **Extensions + applications**.

On the **Monitoring** tab of the **Overview** page, under **Performance and utilization**, you can see that **Guest OS metrics** are now being collected.

View VM insights

To view the VM insights performance graphs and maps:

1. Select **Insights** from the VM's left navigation menu under **Monitoring**.
2. Near the top of the **Insights** page, select the **Performance** tab. The prebuilt VM insights Performance workbook shows charts and graphs with performance-related data for the current VM.
 1. You can customize the view by specifying a different **Time range** at the top of the page and different aggregations at the top of each graph.
 2. Select **View Workbooks** to select from other available prebuilt VM insights workbooks. Select **Go To Gallery** to select from a gallery of other VM insights workbooks and workbook templates, or to edit and create your own workbooks

3. If enabled earlier, select the **Map** tab on the **Insights** page to see the workbook for the Map feature. The map visualizes the VM's dependencies by discovering running process groups and processes that have active network connections over a specified time range

Collect VM client event logs

Azure Monitor Metrics and VM insights performance counters help you identify performance anomalies, and alert when thresholds are reached. But to analyze the root causes of issues you detect, you need to analyze log data to see which system events caused or contributed to the issues.

Create a Data Collection Endpoint

You must have a data collection endpoint to send log data to. To create an endpoint:

1. In the Azure Monitor left navigation menu under **Settings**, select **Data Collection Endpoints**.
2. On the **Data Collection Endpoints** page, select **Create**.
3. On the **Create data collection endpoint** page, for **Name**, enter *linux-logs-endpoint*.
4. Select the same **Subscription**, **Resource group**, and **Region** as your VM uses.
5. Select **Review + create**, and when validation passes, select **Create**.

Create the Data Collection Rule

To create the DCR to collect the event logs:

1. In the Monitor left navigation menu under **Settings**, select **Data Collection Rules**.
2. On the **Data Collection Rules** page, you can see the DCR that VM insights created. Select **Create** to create a new data collection rule.
3. On the **Basics** tab of the **Create Data Collection Rule** screen, provide the following information:
 - **Rule name:** Enter *collect-events-linux*.
 - **Subscription, Resource Group**, and **Region:** Select the same as for your VM.
 - **Platform Type:** Select **Linux**.
4. Select **Next: Resources** or the **Resources** tab.
5. On the **Resources** screen, select **Add resources**.
6. On the **Select a scope** screen, select the **monitored-linux-vm** VM, and then select **Apply**.
7. On the **Resources** screen, select **Enable Data Collection Endpoints**.
8. Under **Data collection endpoint** for the **monitored-linux-vm**, select the **linux-logs-endpoint** you created.
9. Select **Next: Collect and deliver**, or the **Collect and deliver** tab.
10. On the **Collect and deliver** tab, select **Add data source**.
11. On the **Add data source** screen, under **Data source type**, select **Linux Syslog**.
12. On the **Add data source** screen, select **Next: Destination** or the **Destination** tab, and make sure the **Account or namespace** matches the Log Analytics workspace that you want to use. You can use the default Log Analytics workspace that VM insights set up, or create or use another Log Analytics workspace.
13. On the **Add data source** screen, select **Add data source**.
14. On the **Create Data Collection Rule** screen, select **Review + create**, and when validation passes, select **Create**.

View log data

You can view and analyze the log data collected by your DCR by using KQL log queries. A set of sample KQL queries is available for VMs, but you can write a query to look at the events your DCR is collecting.

Azure Monitor helps you collect, analyze, and alert on various types of host and client monitoring data from your Azure VMs.

- Azure Monitor provides a set of VM host logs and performance and usage metrics for all Azure VMs.
- You can enable recommended alert rules when you create VMs or afterwards to alert on important VM host metrics.
- Azure Monitor Metrics Explorer lets you graph and analyze metrics for Azure VMs and other resources.
- VM insights provides a simple way to monitor important VM client performance counters and processes running on your VM.
- You can create data collection rules to collect other metrics and logs from your VM client.
- You can use Log Analytics to query and analyze log data.